# Architecture Diagram Collection

From first principles to expert practice

Built from the manuscript sources · 0 chapter(s) in this edition

**Appendix D — Architecture Diagram Collection**

Chapter 1 — What Is System Design?
Chapter 2 — How to Think Like a System Designer
Chapter 3 — The System Design Learning Method
Chapter 4 — Quantitative Foundations for System Design
Chapter 5 — Computer Architecture
Chapter 6 — Operating Systems
Chapter 7 — Networking Fundamentals
Chapter 8 — Web Application Fundamentals
Chapter 9 — Real-Time Communication
Chapter 10 — Security Fundamentals for System Designers

# Appendix D — Architecture Diagram Collection

> Generated by `build/build.py` from every Mermaid diagram in the manuscript. Sources are in `diagrams/chNN/*.mmd`. Each figure links back to the chapter that explains it.

## Chapter 1 — What Is System Design?

**Figure 1.1 — PhotoShare v0: a single-server design.**

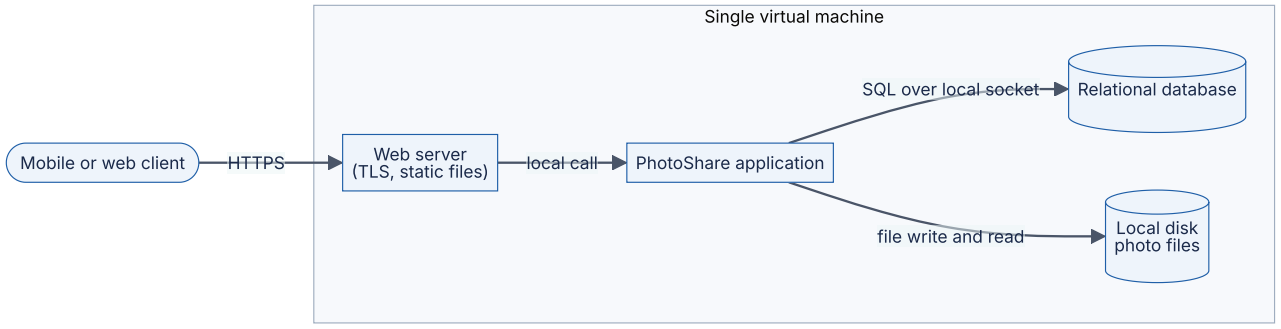



**Figure 1.2 — High-level design of the PhotoShare upload path.**

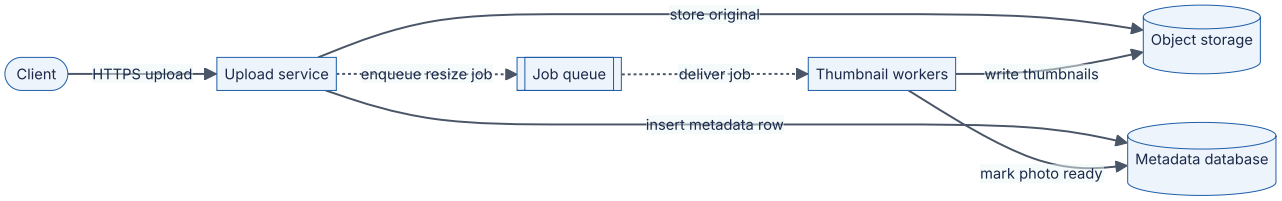



**Figure 1.3 — Low-level design of the upload component (class view).**

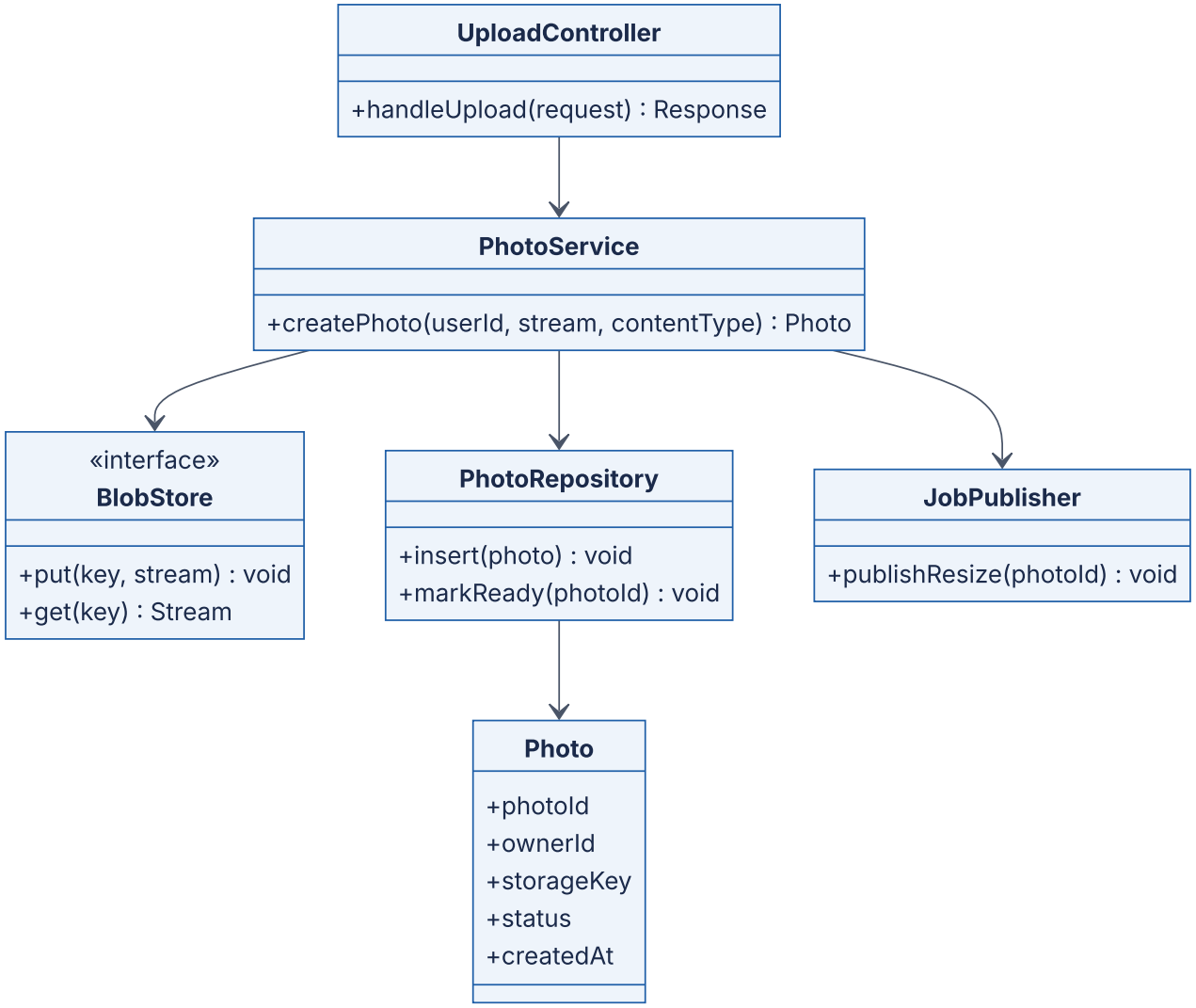



**Figure 1.4 — ADR lifecycle.**

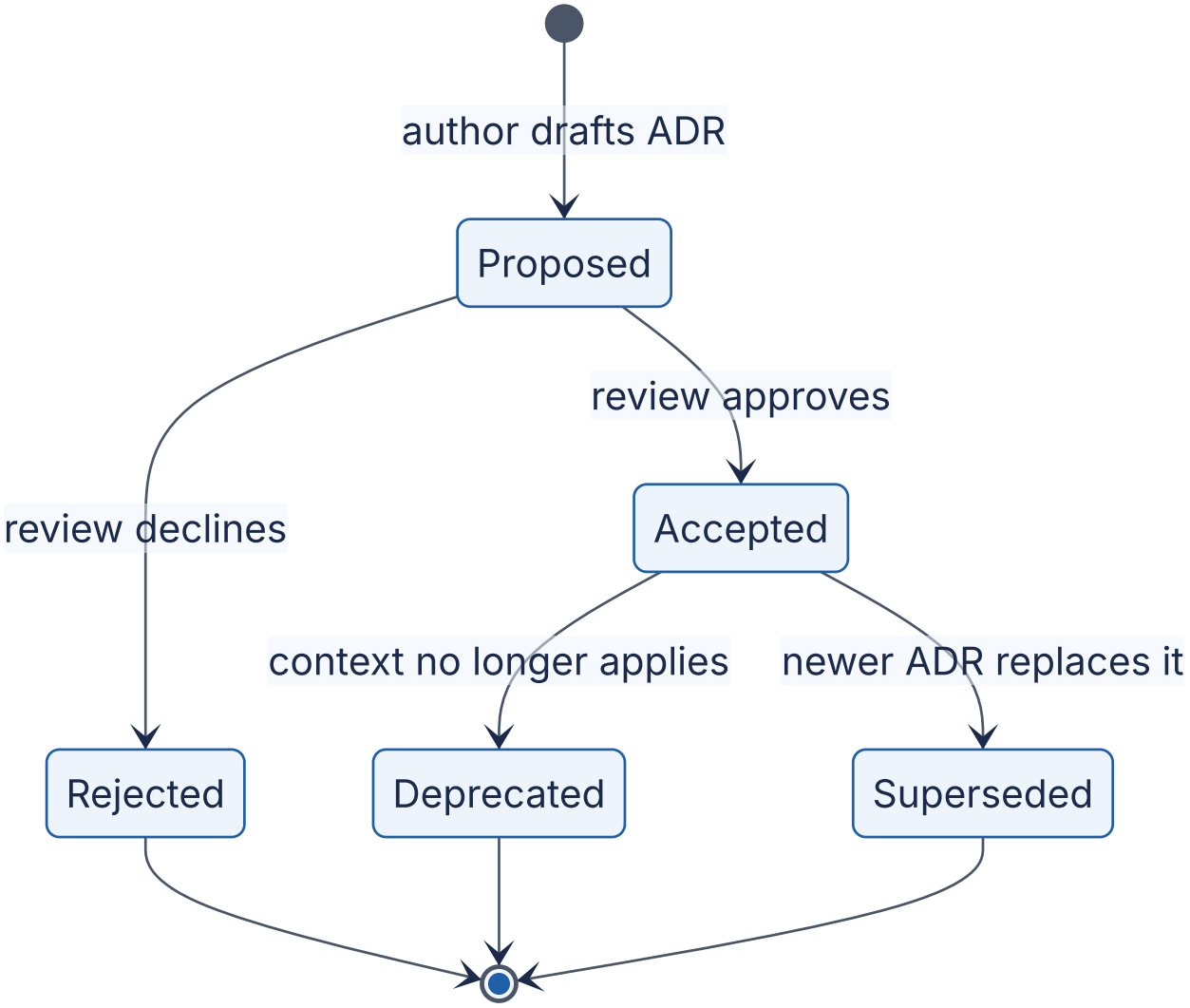



**Figure 1.5 — PhotoShare v1: stateless application tier and managed storage.**

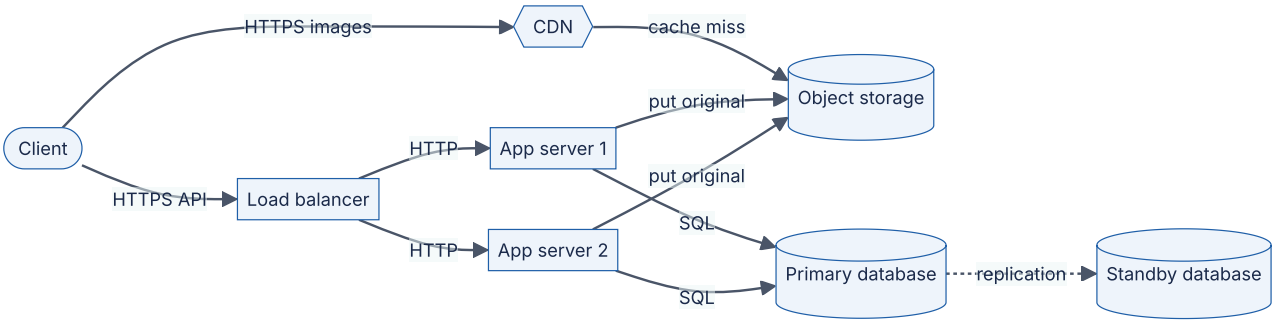



**Figure 1.6 — PhotoShare v2: cache, read replicas and asynchronous processing.**

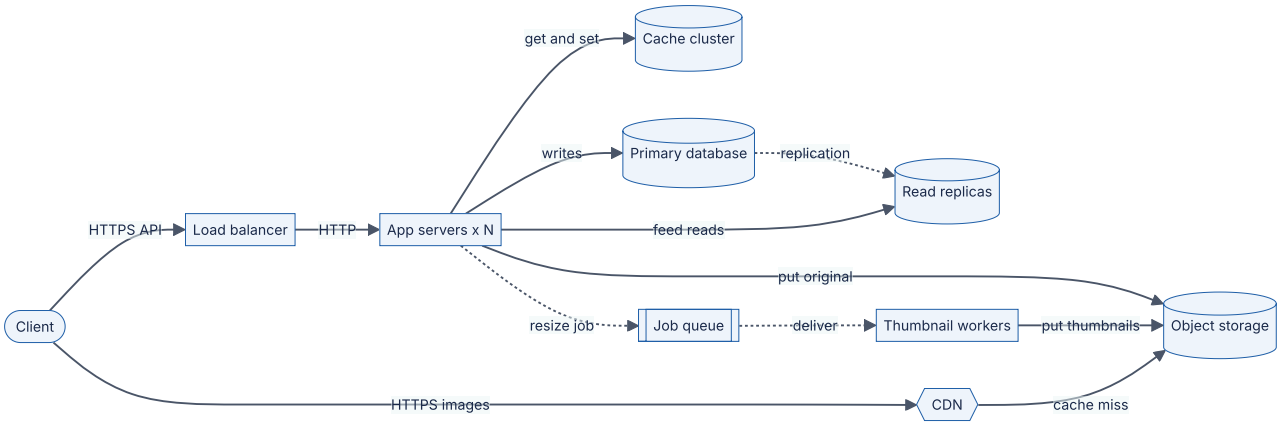



**Figure 1.7 — PhotoShare v3: services, partitioned data and multi-zone deployment.**

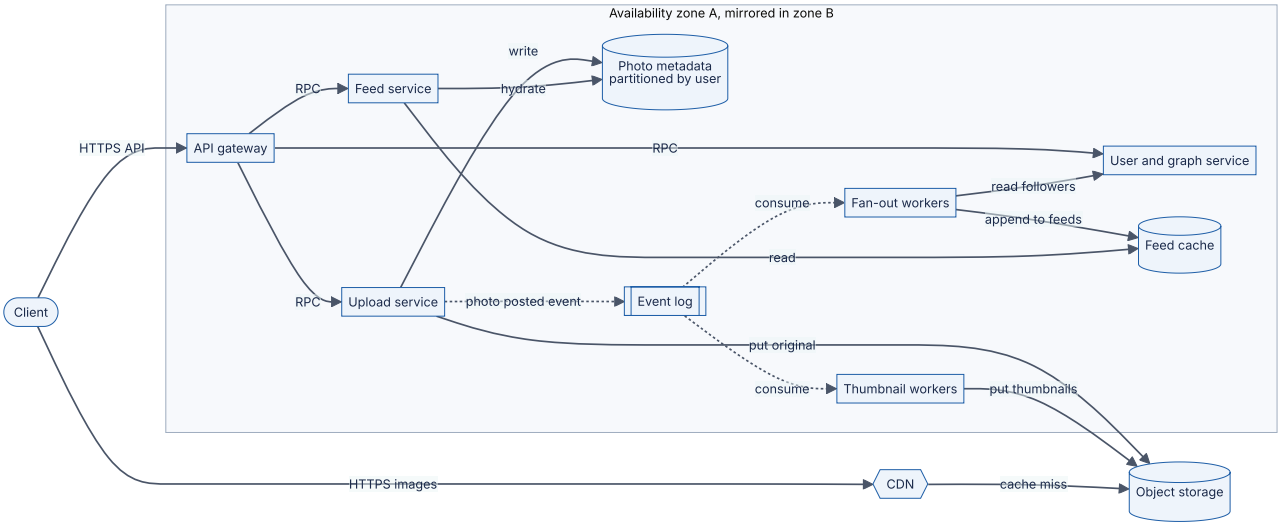



**Figure 1.8 — The life of a design in a typical engineering organization.**

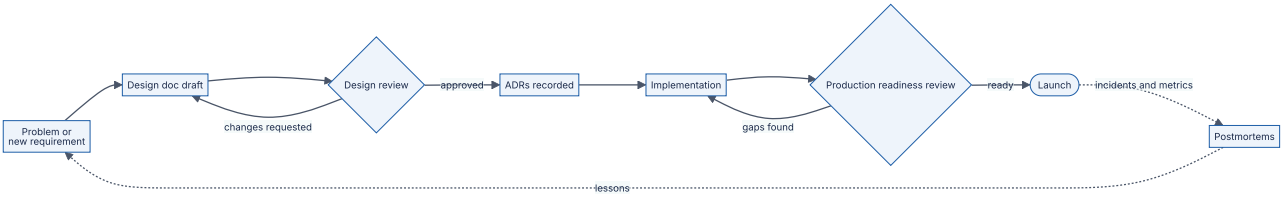
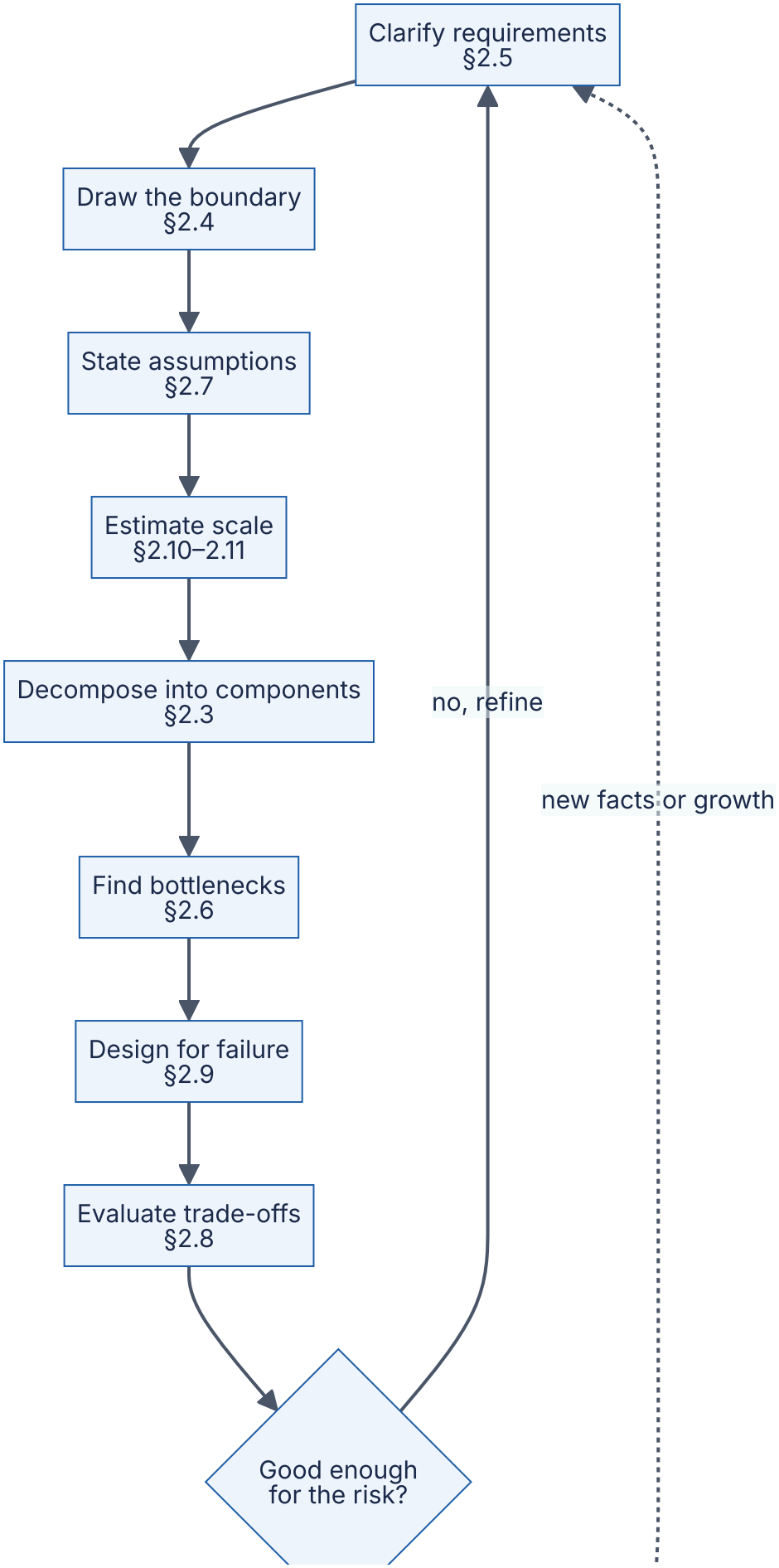



**Figure 1.9 — A typical 45-minute system design interview.**




# Chapter 2 — How to Think Like a System Designer

**Figure 2.1 — The system designer’s thinking loop.**


Clarify requirements §2.5
Draw the boundary §2.4
State assumptions §2.7
Estimate scale §2.10–2.11
Decompose into components §2.3
Find bottlenecks §2.6
Design for failure §2.9
Evaluate trade-offs §2.8
Good enough for the risk?
no, refine
new facts or growth

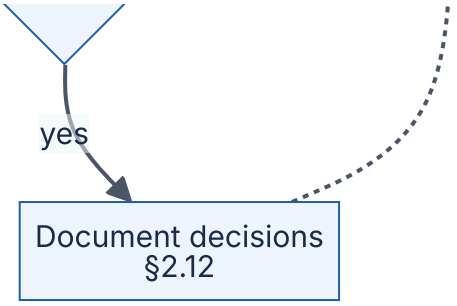



**Figure 2.2 — QuickBite first-pass decomposition.**

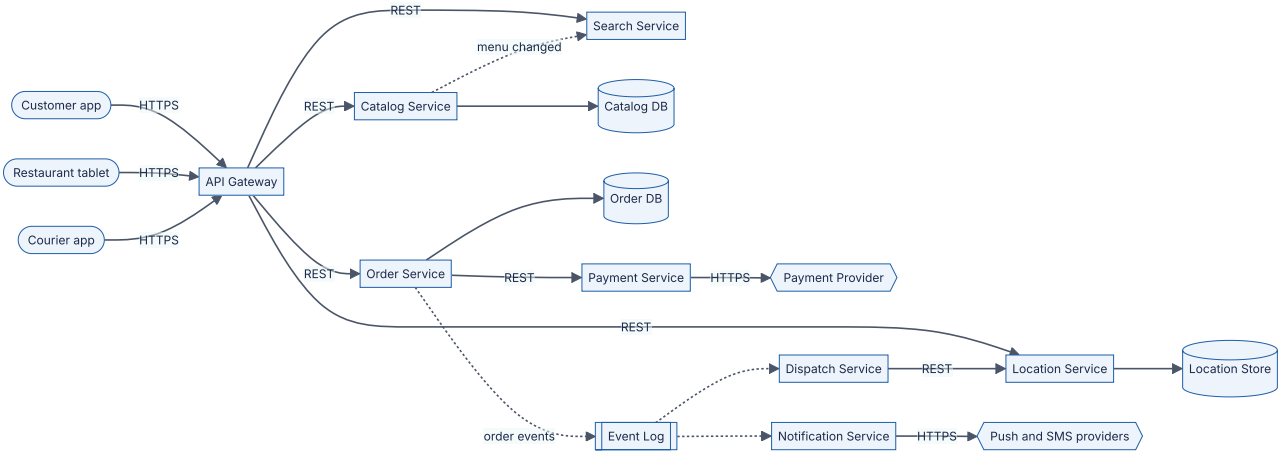



**Figure 2.3 — QuickBite system-context diagram.**

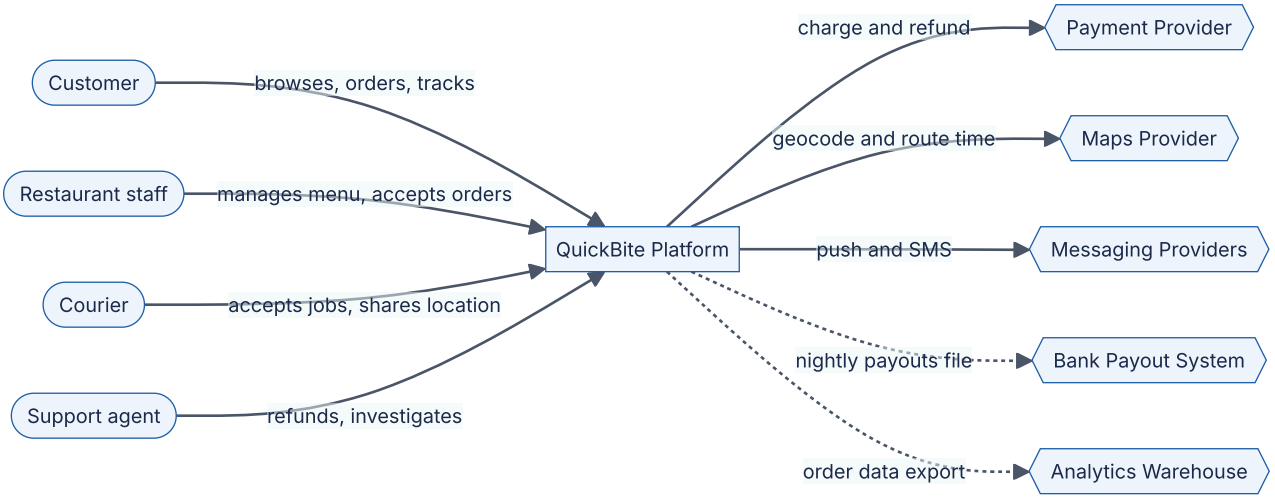



**Figure 2.4 — Finding the bottleneck with the USE method.**

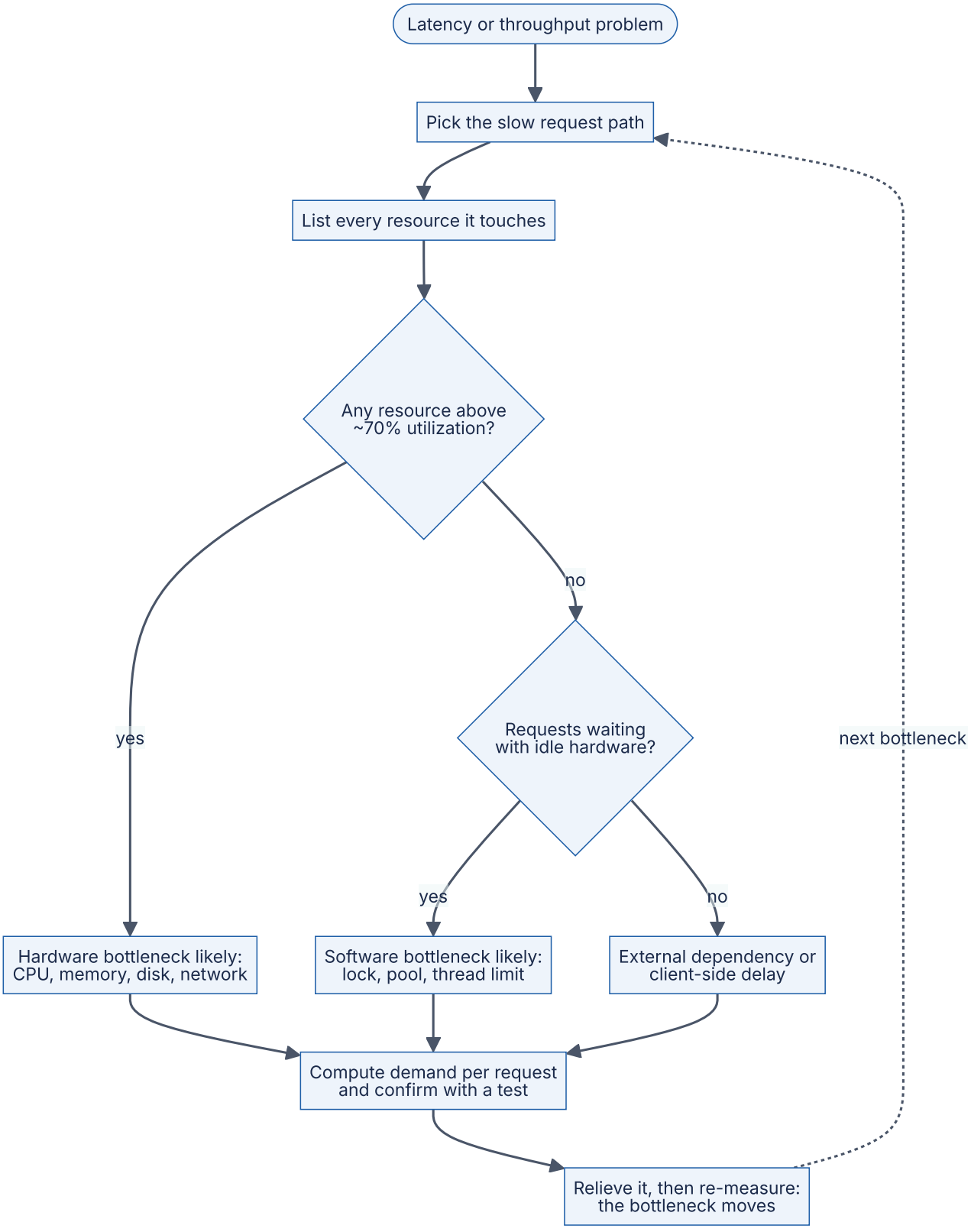



**Figure 2.5 — Choosing the decision process by reversibility.**

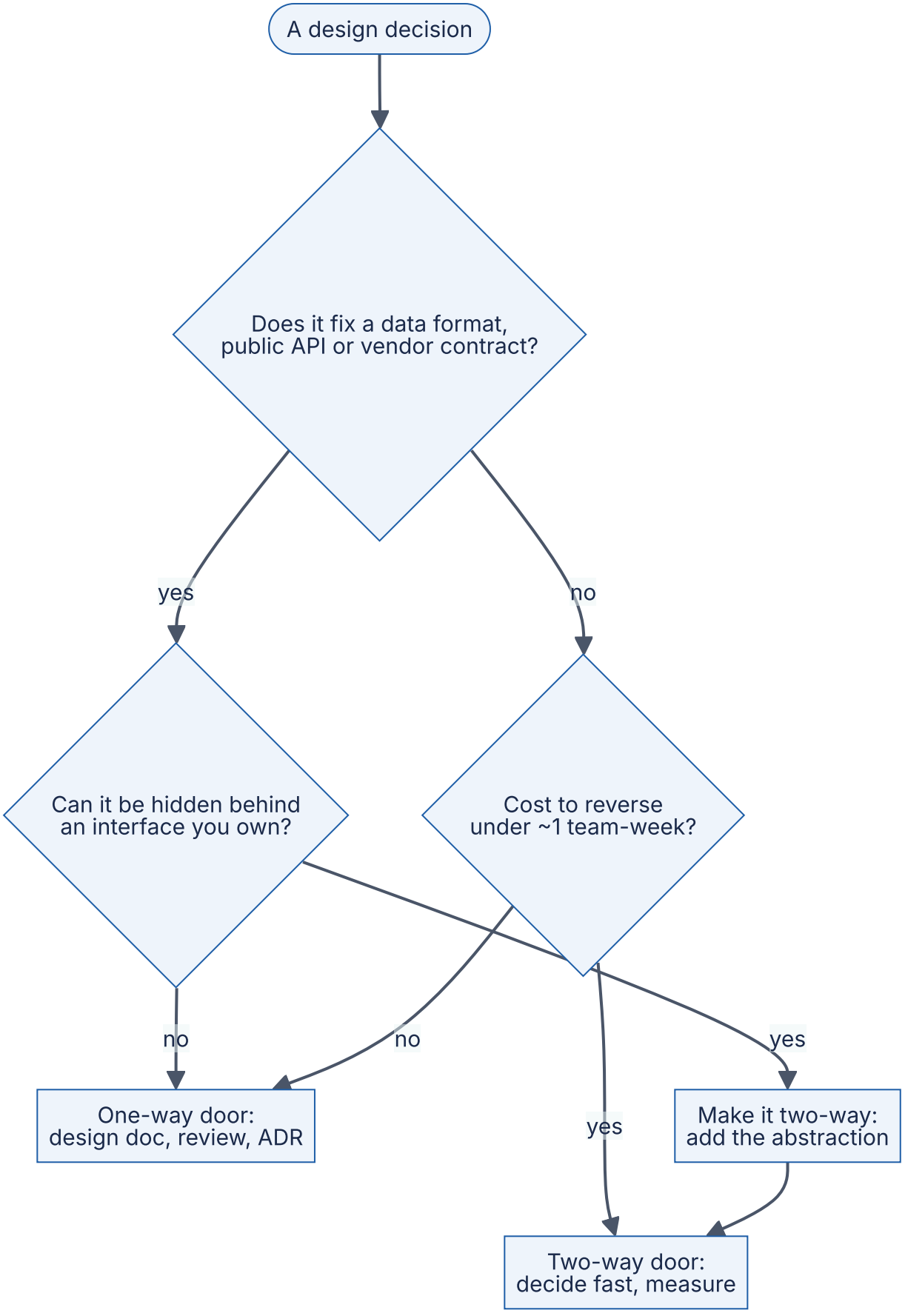



**Figure 2.6 — QuickBite checkout designed for failure.**

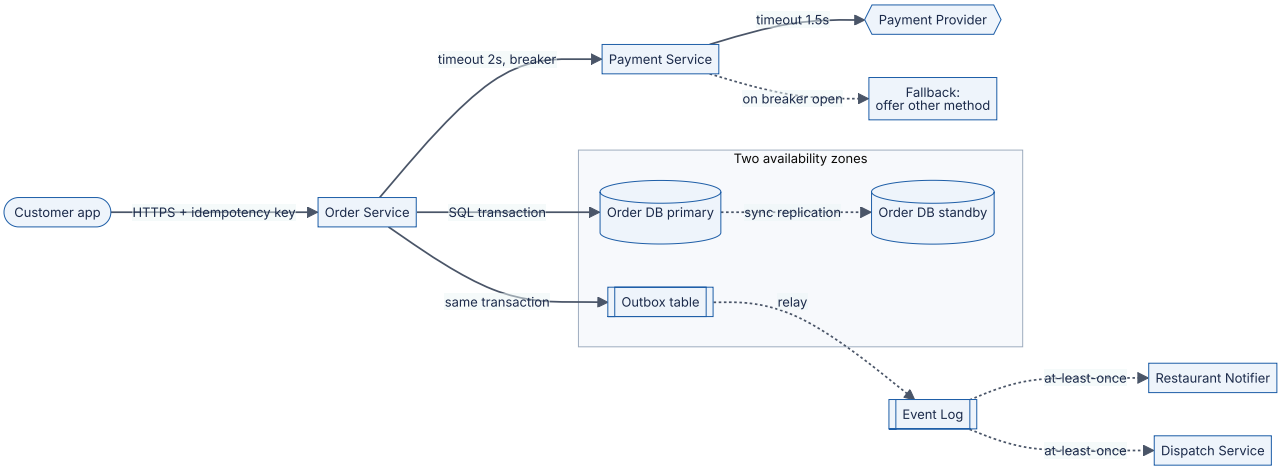



**Figure 2.7 — QuickBite C4 container view, after the §2.8 decision.**

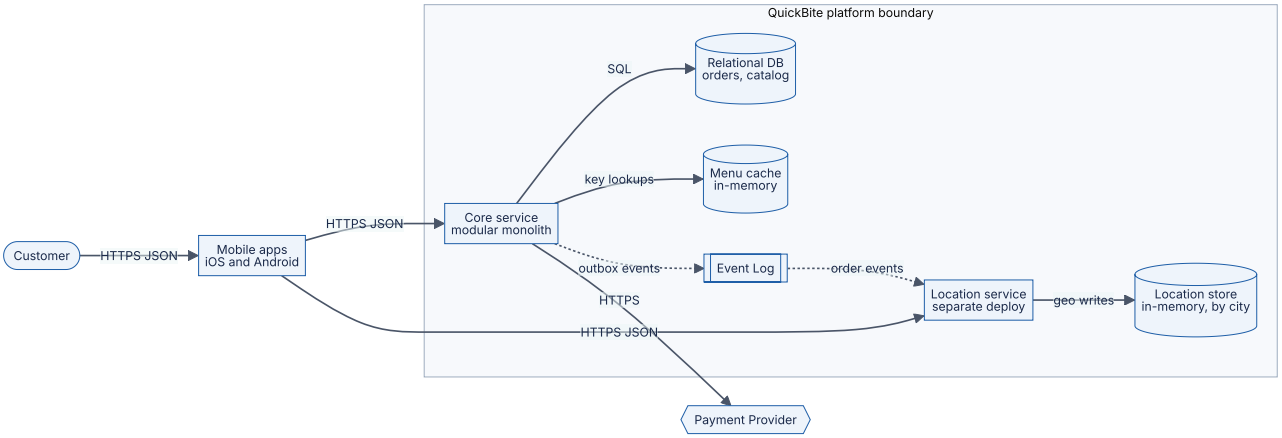
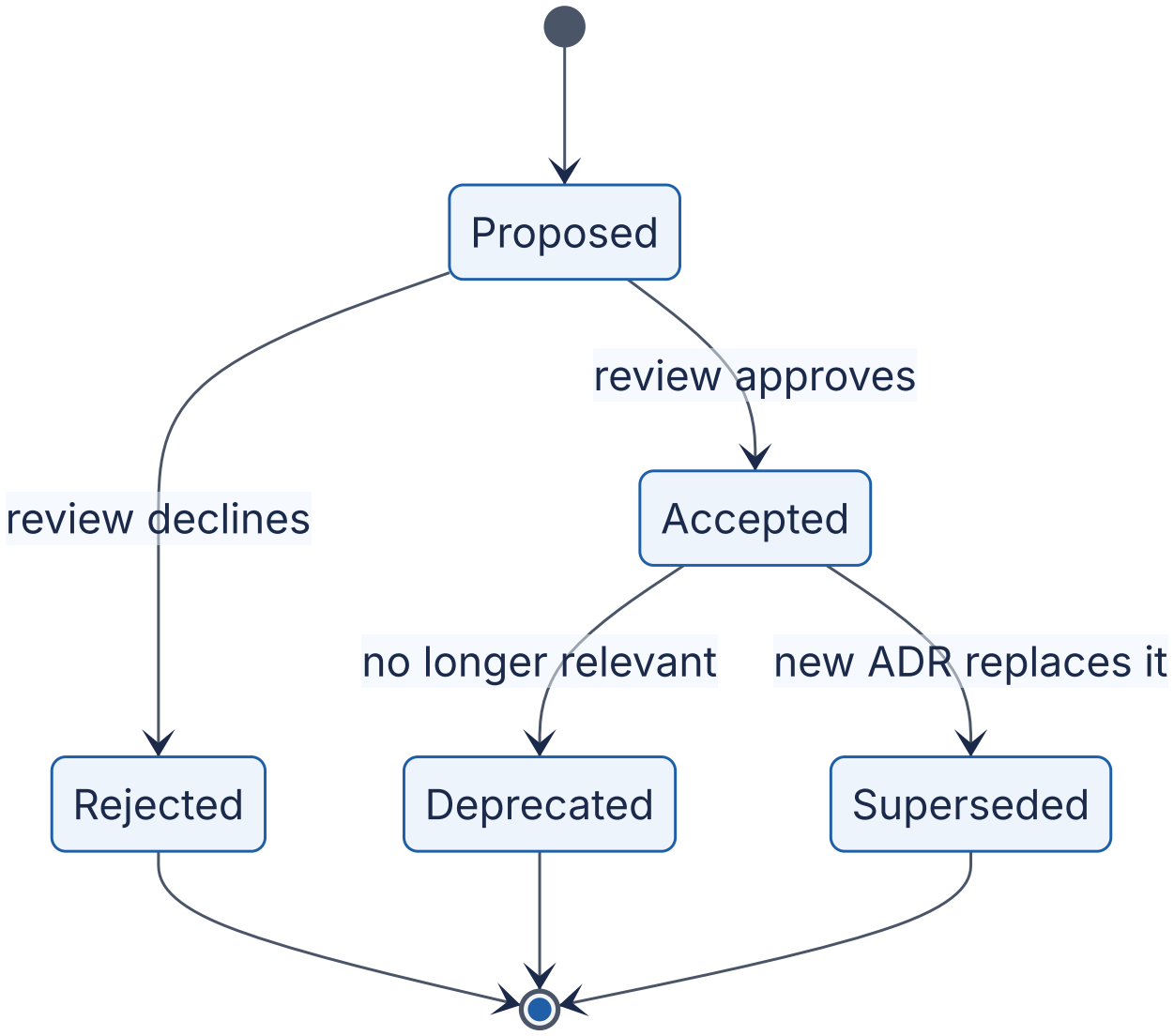
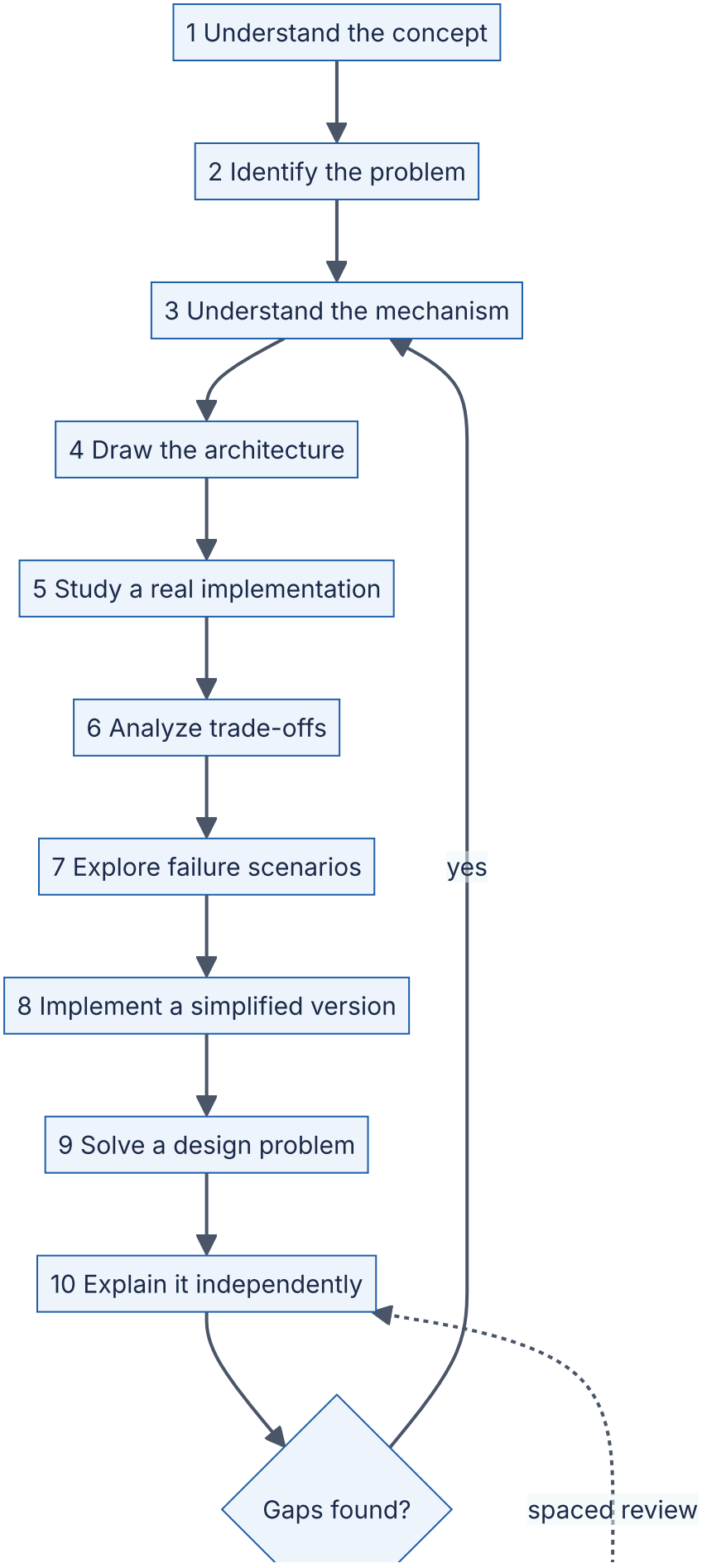



**Figure 2.8 — ADR life cycle.**




## Chapter 3 — The System Design Learning Method

**Figure 3.1 — The ten-step concept-mastery loop.**


1 Understand the concept
2 Identify the problem
3 Understand the mechanism
4 Draw the architecture
5 Study a real implementation
6 Analyze trade-offs
7 Explore failure scenarios
8 Implement a simplified version
9 Solve a design problem
10 Explain it independently
Gaps found?
yes
spaced review

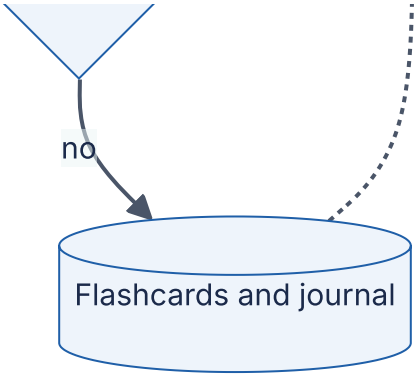



**Figure 3.2 — Consistent hashing in a client-routed distributed cache.**

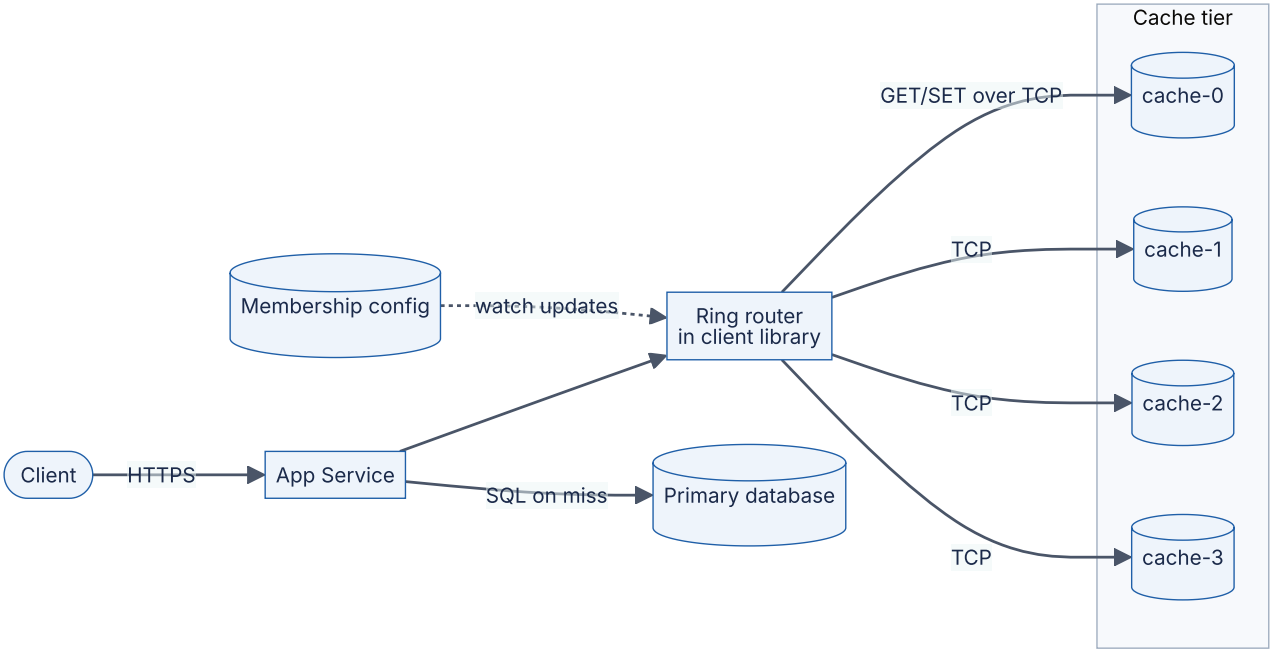



**Figure 3.3 — Session read when the primary owner has failed.**

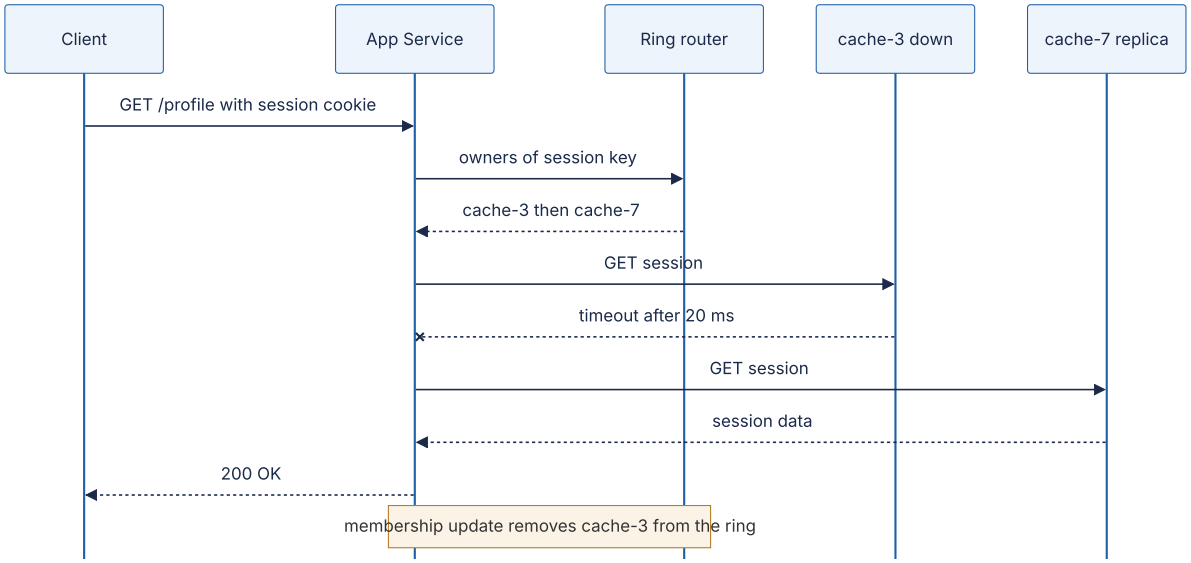


**Figure 3.4 — A five-box Leitner schedule for flashcards.**

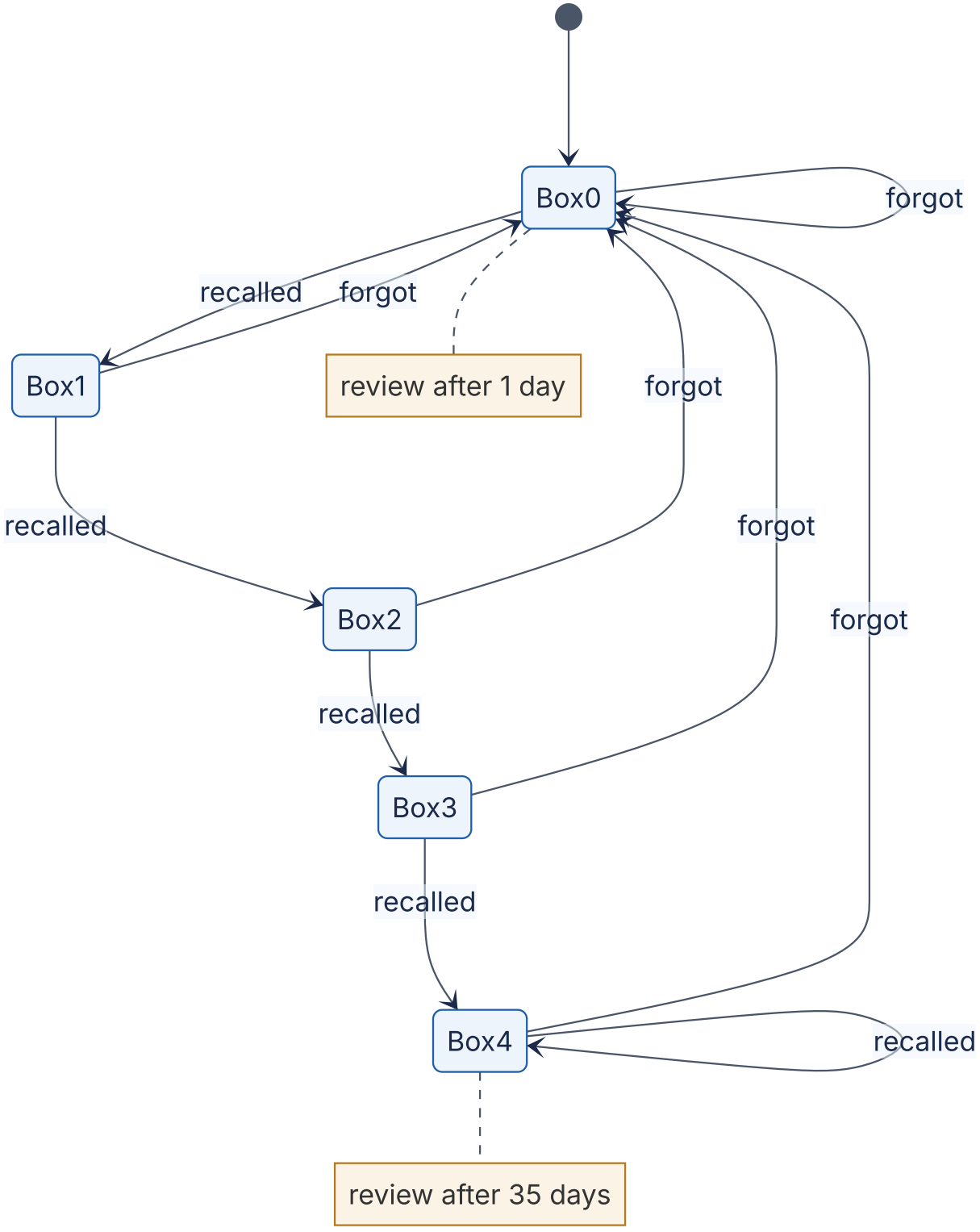
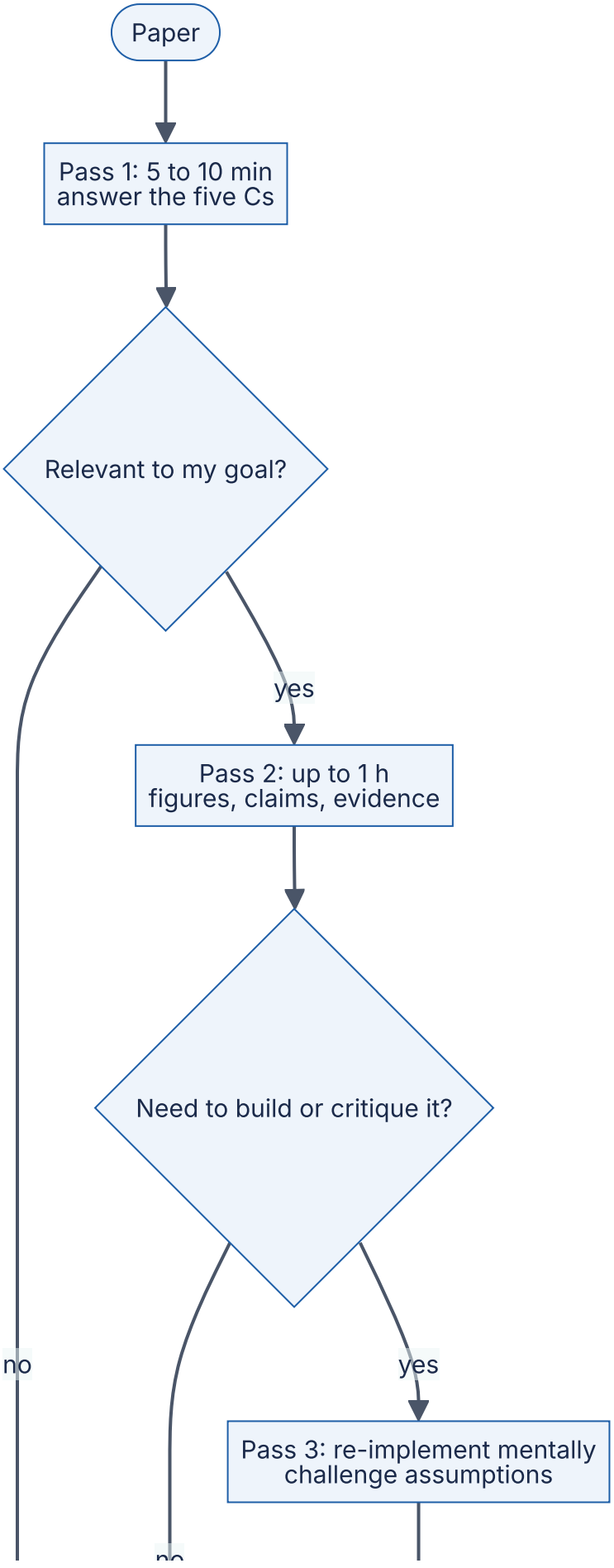



**Figure 3.5 — The three-pass reading decision flow.**


Paper
Pass 1: 5 to 10 min
answer the five Cs
Relevant to my goal?
yes
Pass 2: up to 1 h
figures, claims, evidence
Need to build or critique it?
no
yes
Pass 3: re-implement mentally
challenge assumptions

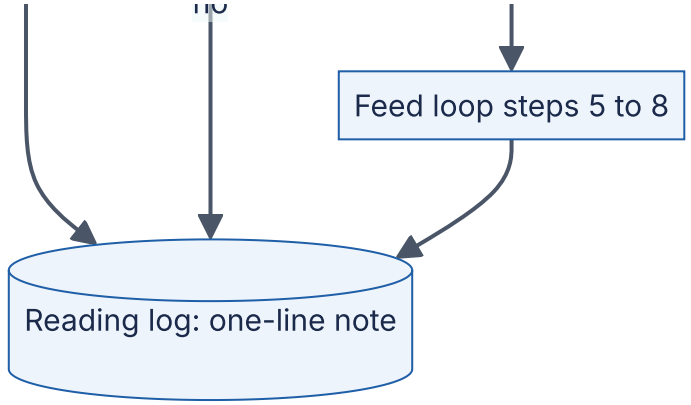



**Figure 3.6 — The 12-month beginner path (months shown as M01–M12).**

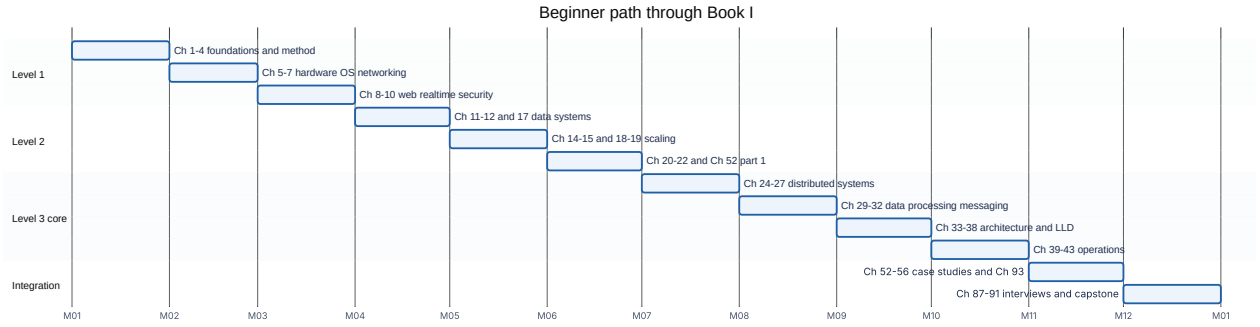



**Figure 3.7 — The postmortem study loop.**




# Chapter 4 — Quantitative Foundations for System Design

**Figure 4.1 — B-tree fan-out turns a billion keys into four levels.**

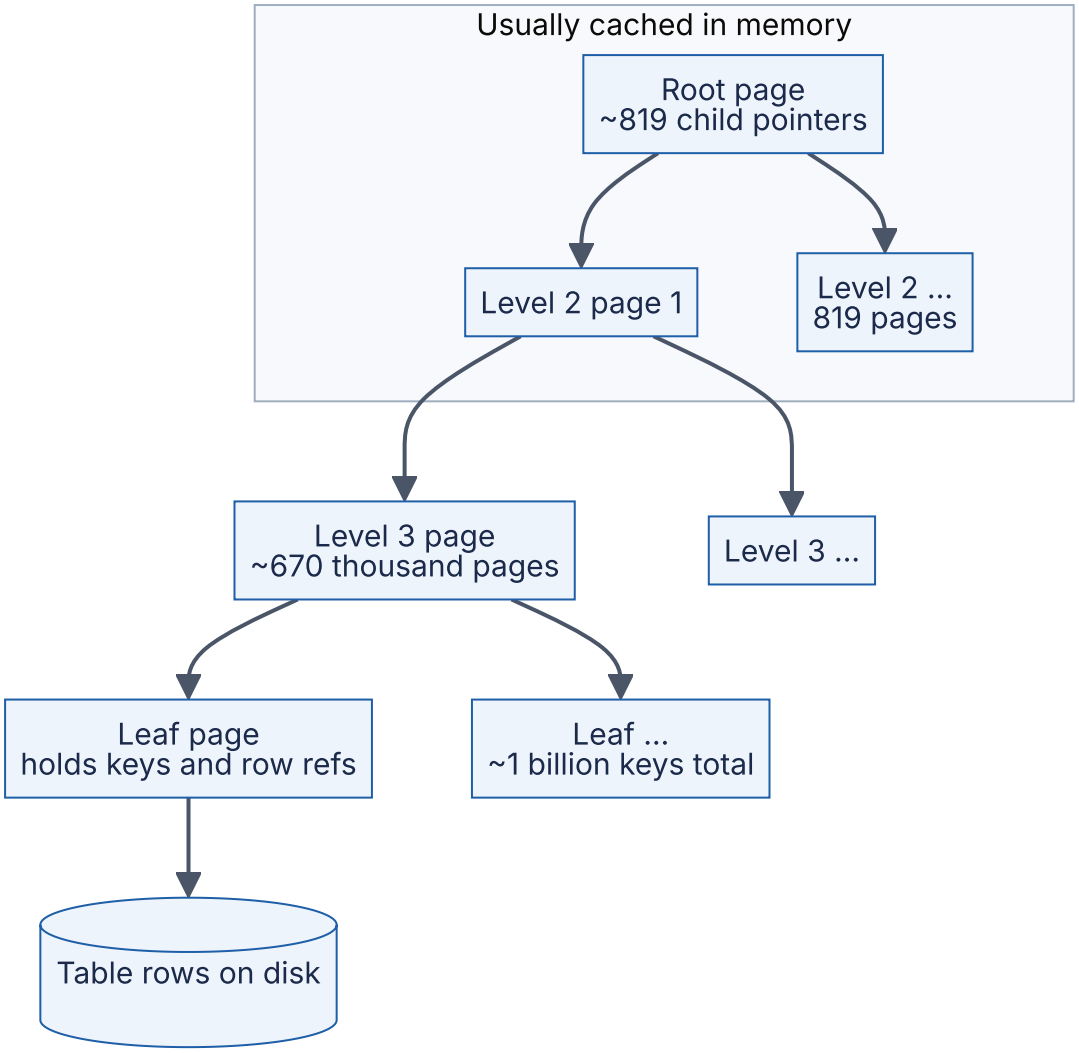



**Figure 4.2 — Fan-out: the slowest leaf determines the response time.**

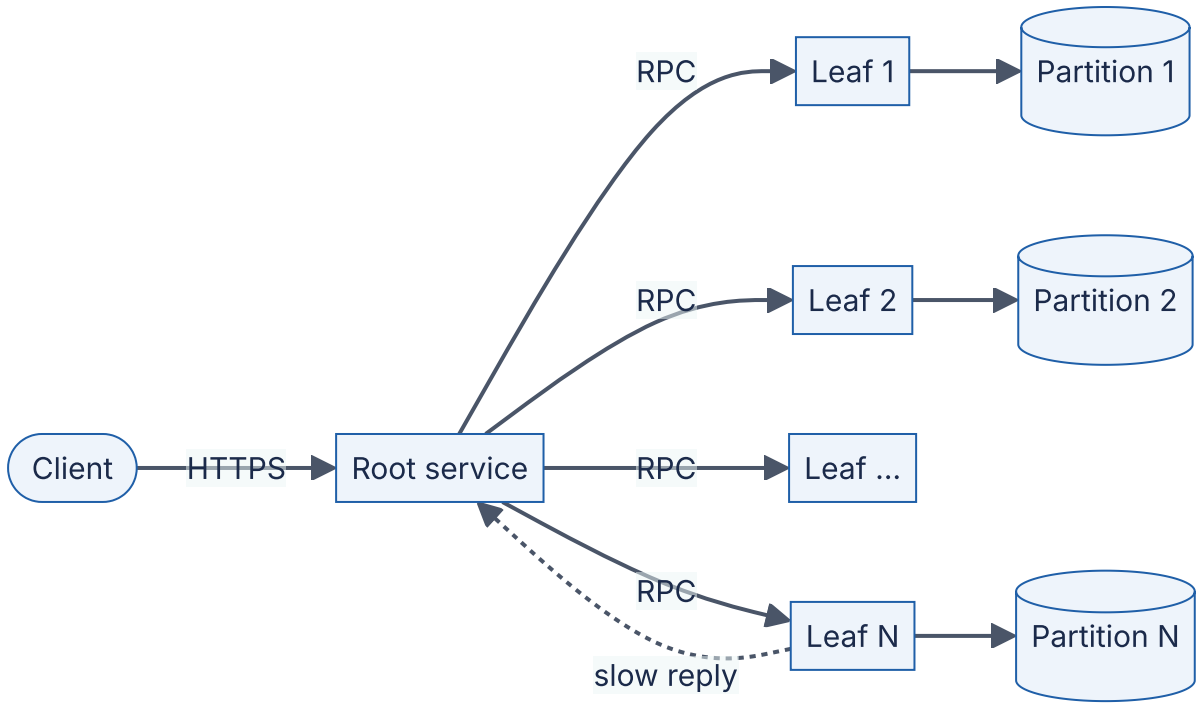



**Figure 4.3 — The M/M/1 queue.**




**Figure 4.4 — One shared queue versus separate queues.**

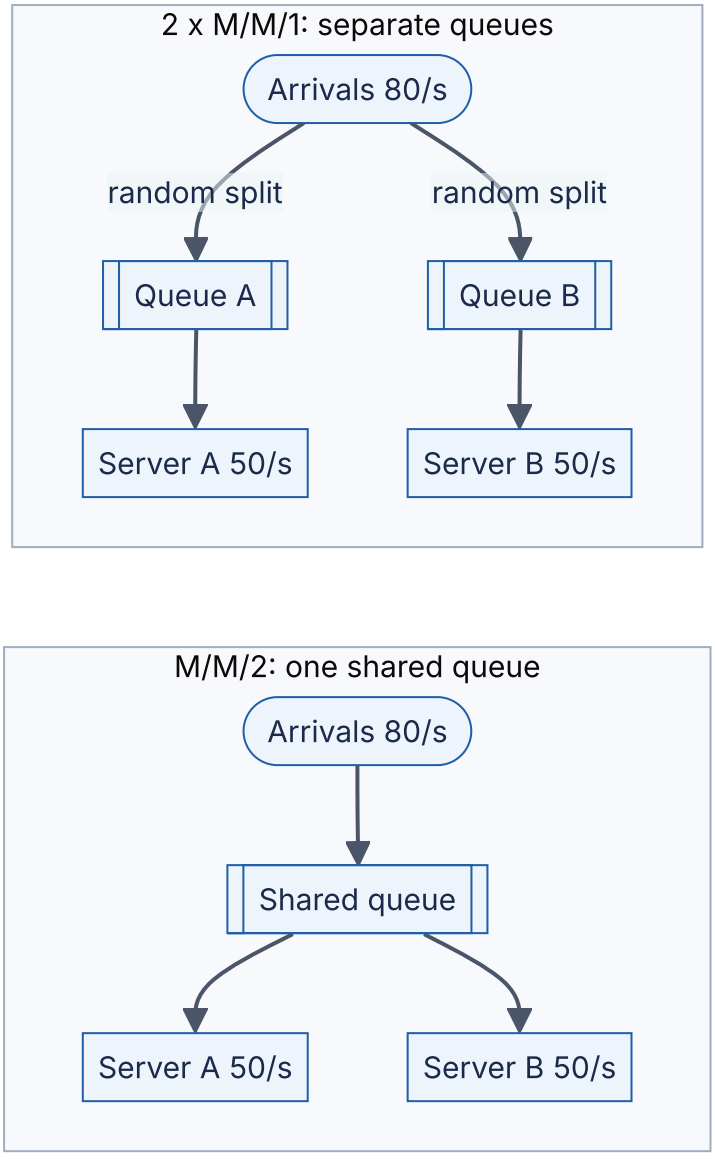



**Figure 4.5 — Choosing the boundary for Little’s Law.**

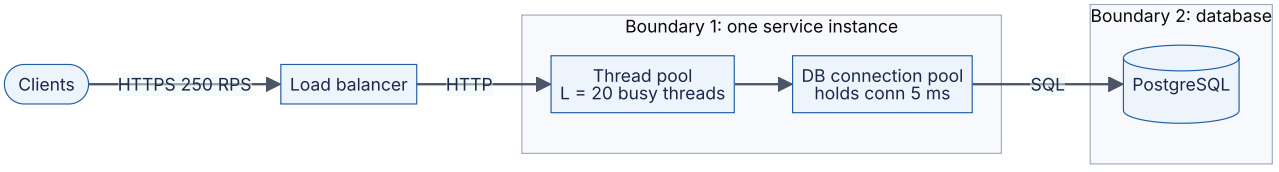



**Figure 4.6 — Series, parallel and k-of-n availability.**

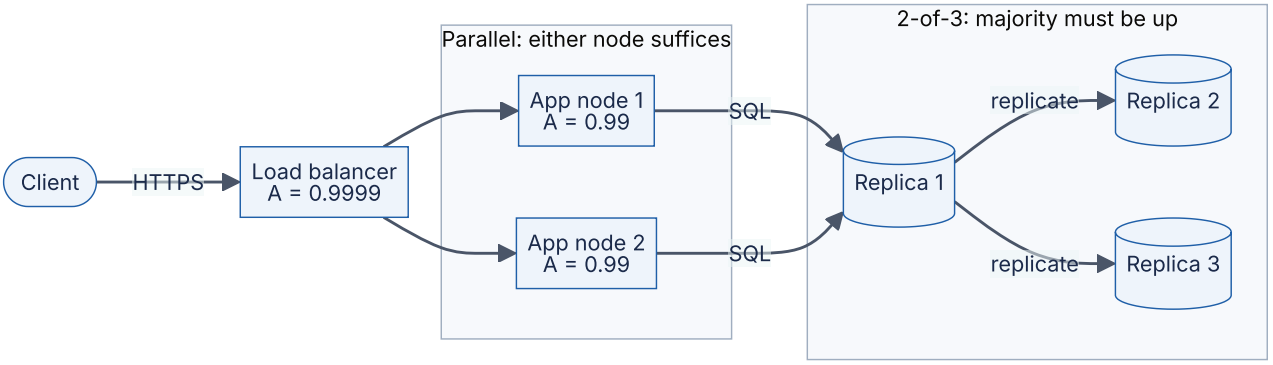



**Figure 4.7 — The up-down cycle of a repairable component.**

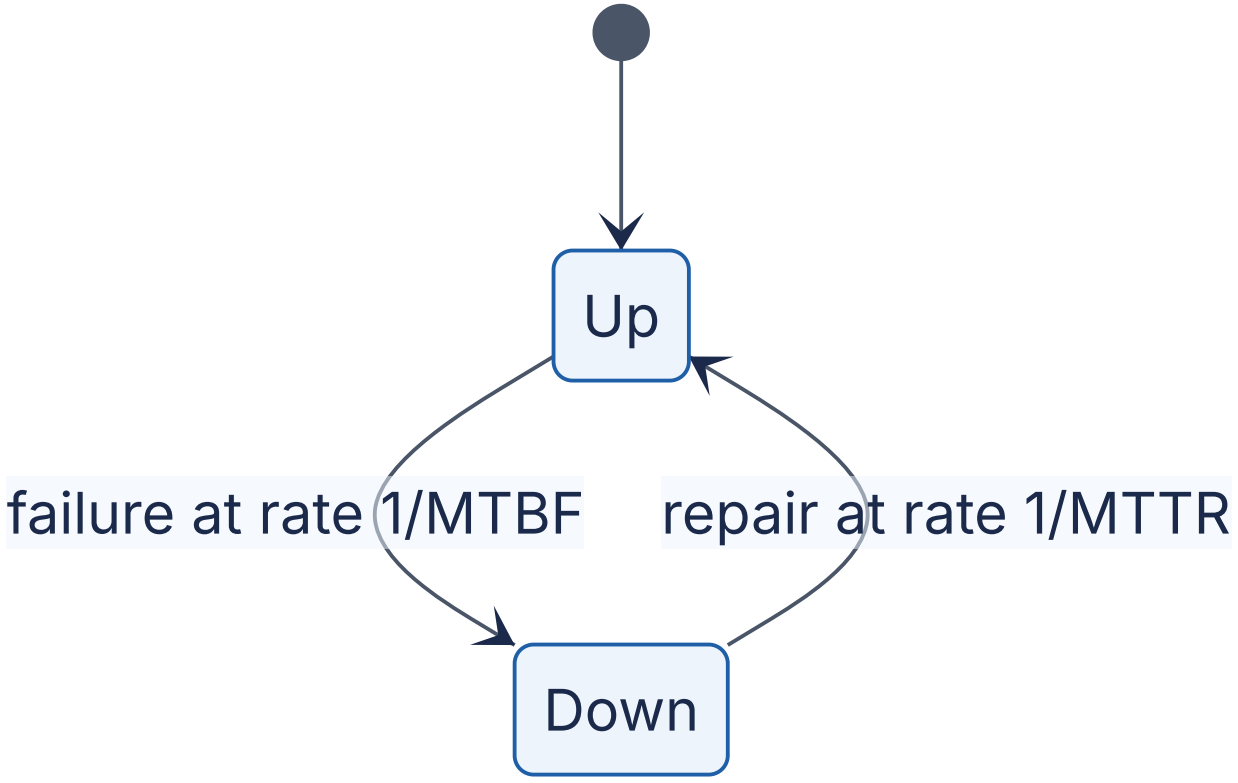



**Figure 4.8 — The replica-loss race for three-way replication.**

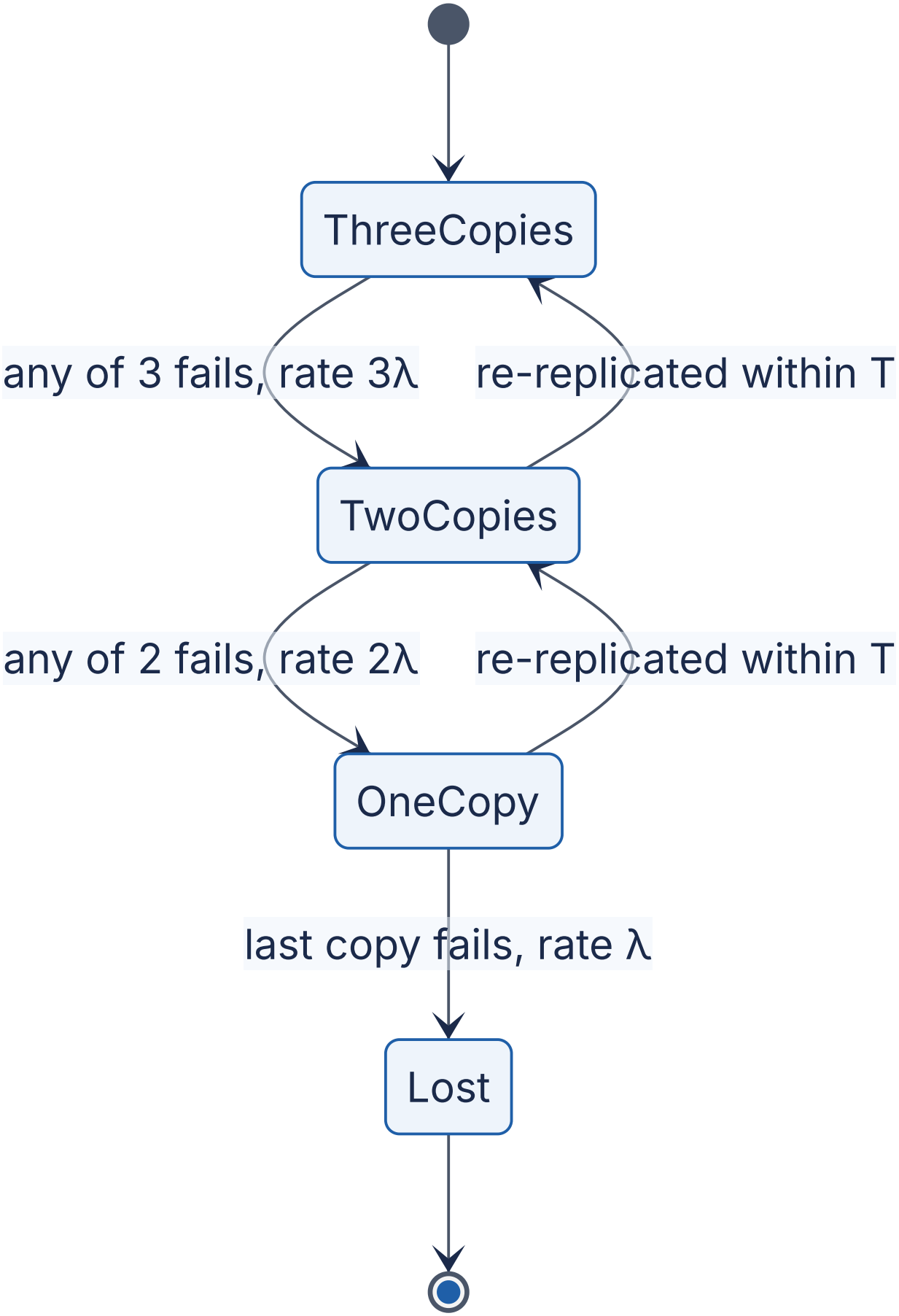
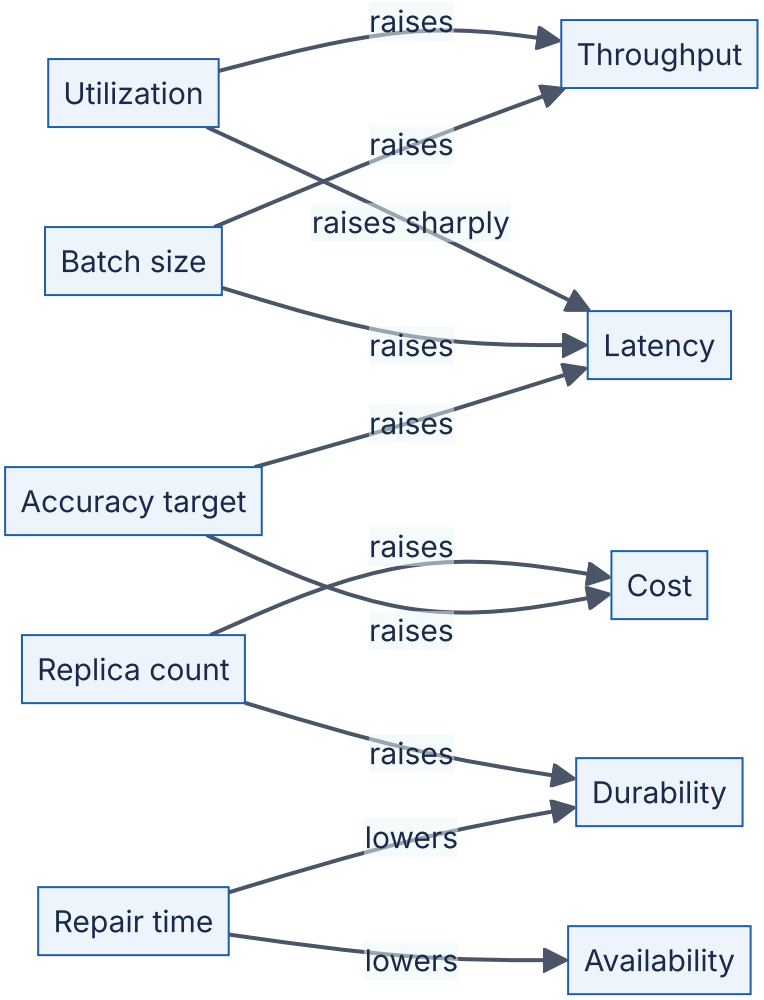

ThreeCopies
any of 3 fails, rate 3λ
re-replicated within T
TwoCopies
any of 2 fails, rate 2λ
re-replicated within T
OneCopy
last copy fails, rate λ
Lost

**Figure 4.9 — Six recurring quantitative relationships.**




# Chapter 5 — Computer Architecture

**Figure 5.1 — Inside a single-socket server.**

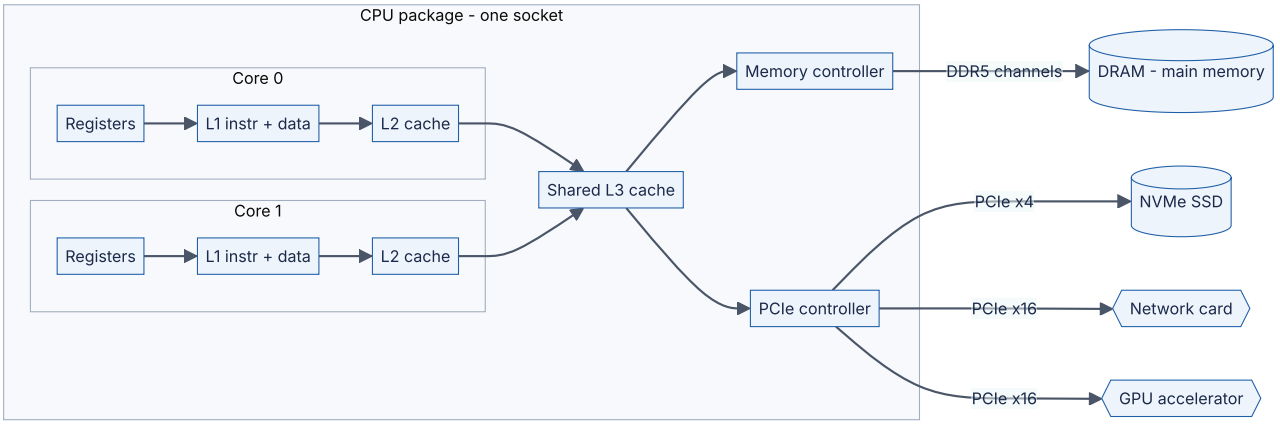



**Figure 5.2 — A pipelined core with a branch predictor.**

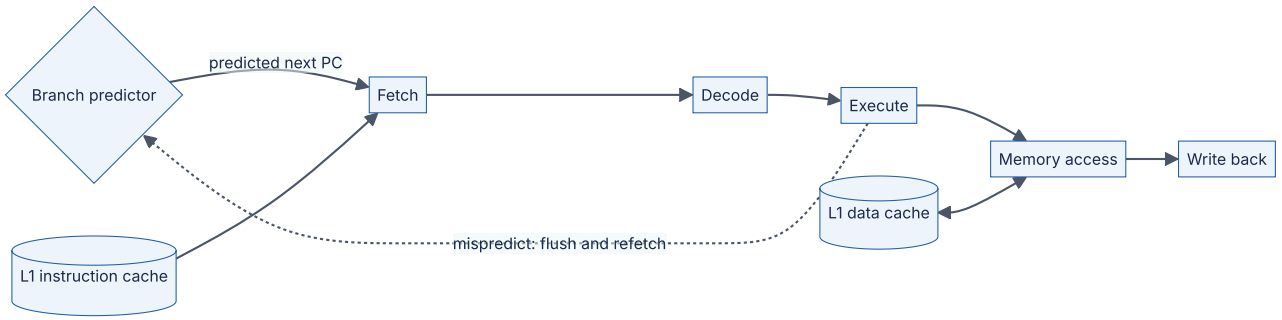



**Figure 5.3 — What happens on a load instruction.**

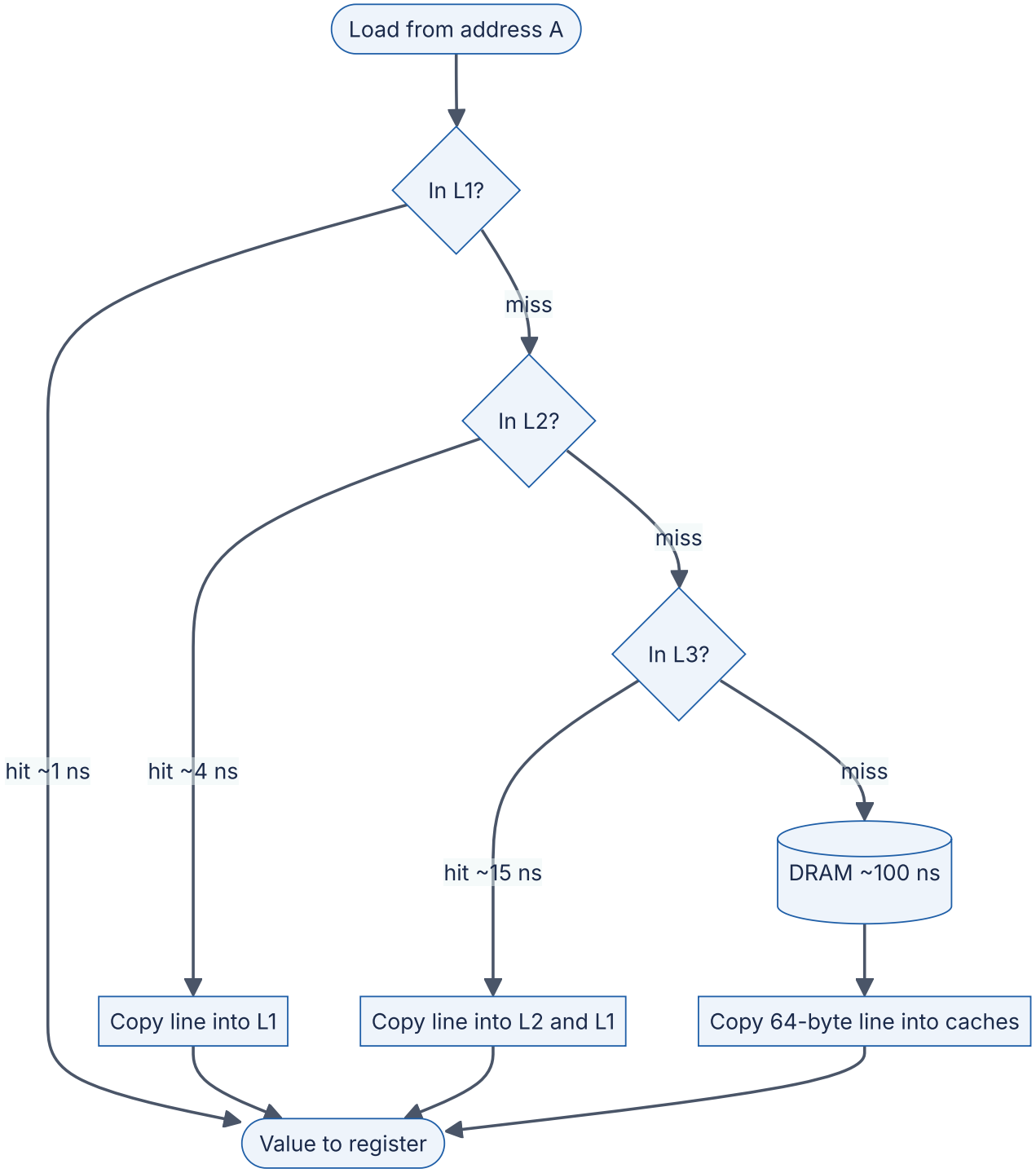



**Figure 5.4 — MESI state transitions for one cache line in one core’s cache.**

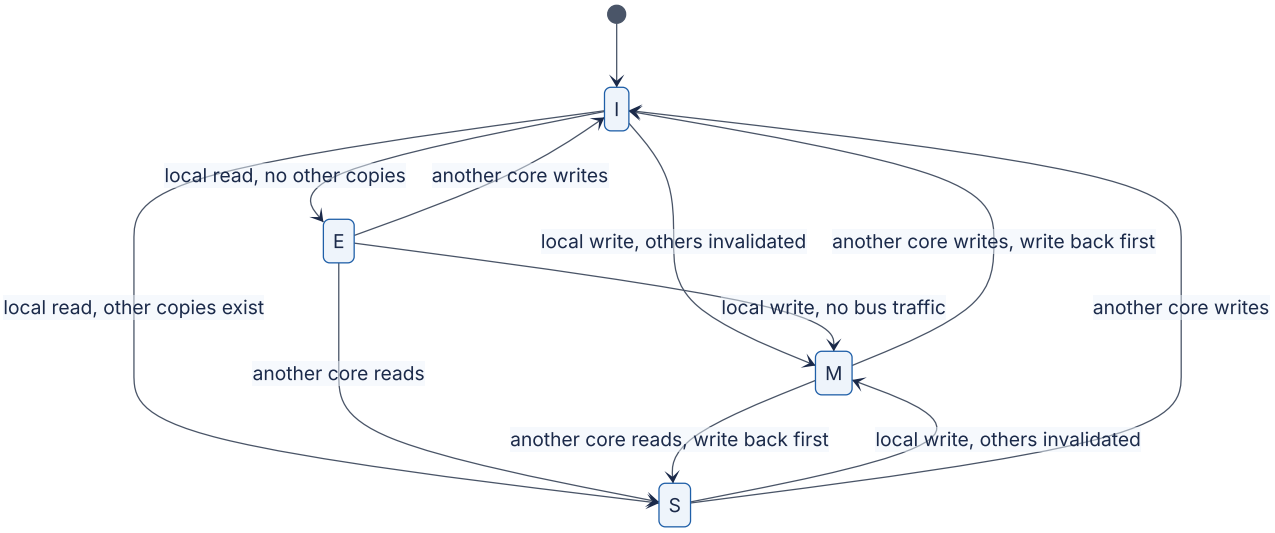



**Figure 5.5 — The memory and storage hierarchy (as of 2024–2025, approximate).**

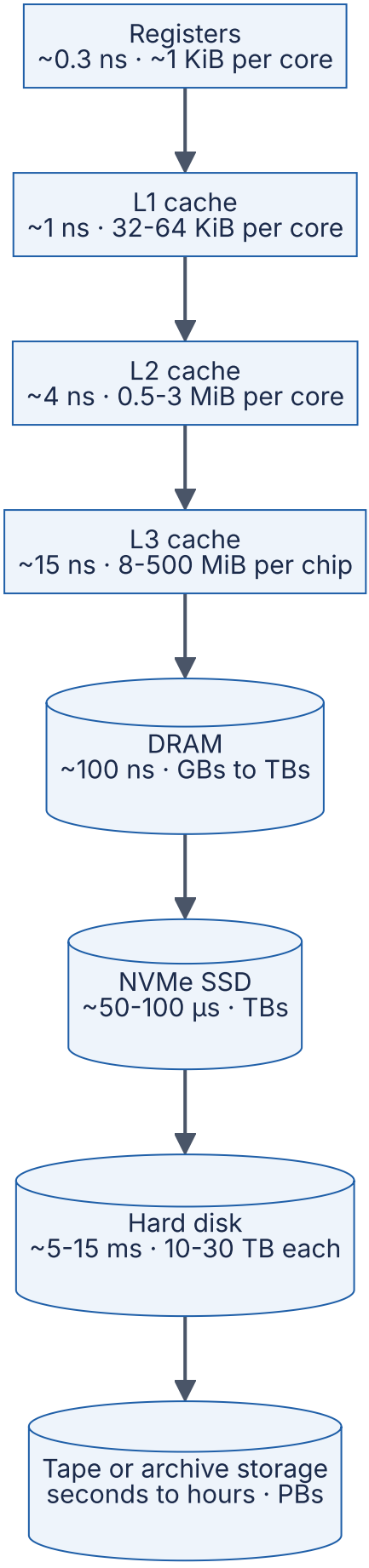

Registers
~0.3 ns · ~1 KiB per core
L1 cache
~1 ns · 32-64 KiB per core
L2 cache
~4 ns · 0.5-3 MiB per core
L3 cache
~15 ns · 8-500 MiB per chip
DRAM
~100 ns · GBs to TBs
NVMe SSD
~50-100 µs · TBs
Hard disk
~5-15 ms · 10-30 TB each
Tape or archive storage
seconds to hours · PBs

**Figure 5.6 — How an SSD handles an overwrite and garbage collection.**

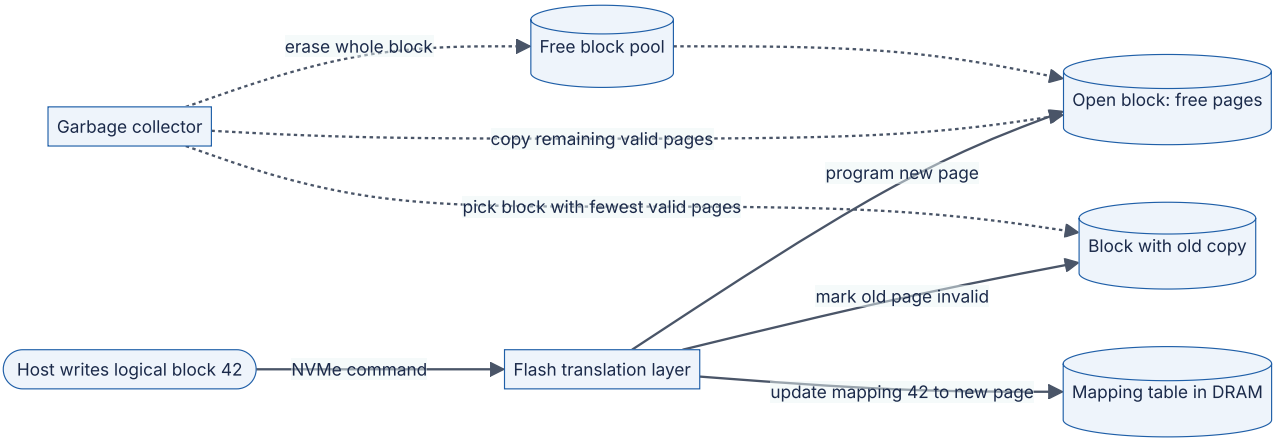



**Figure 5.7 — Reading a block from an NVMe SSD with DMA and an interrupt.**

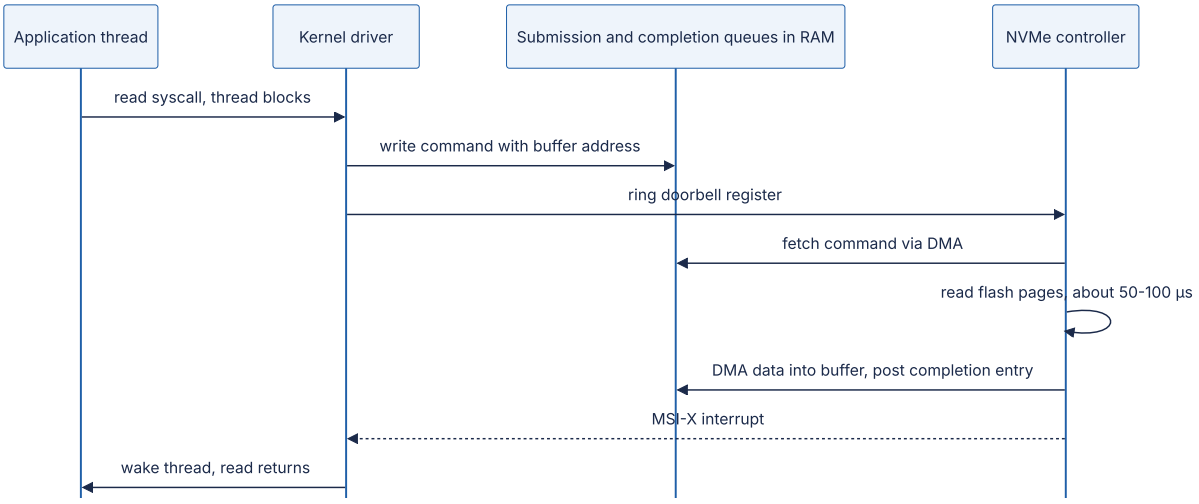



**Figure 5.8 — Diagnosing what limits a workload.**

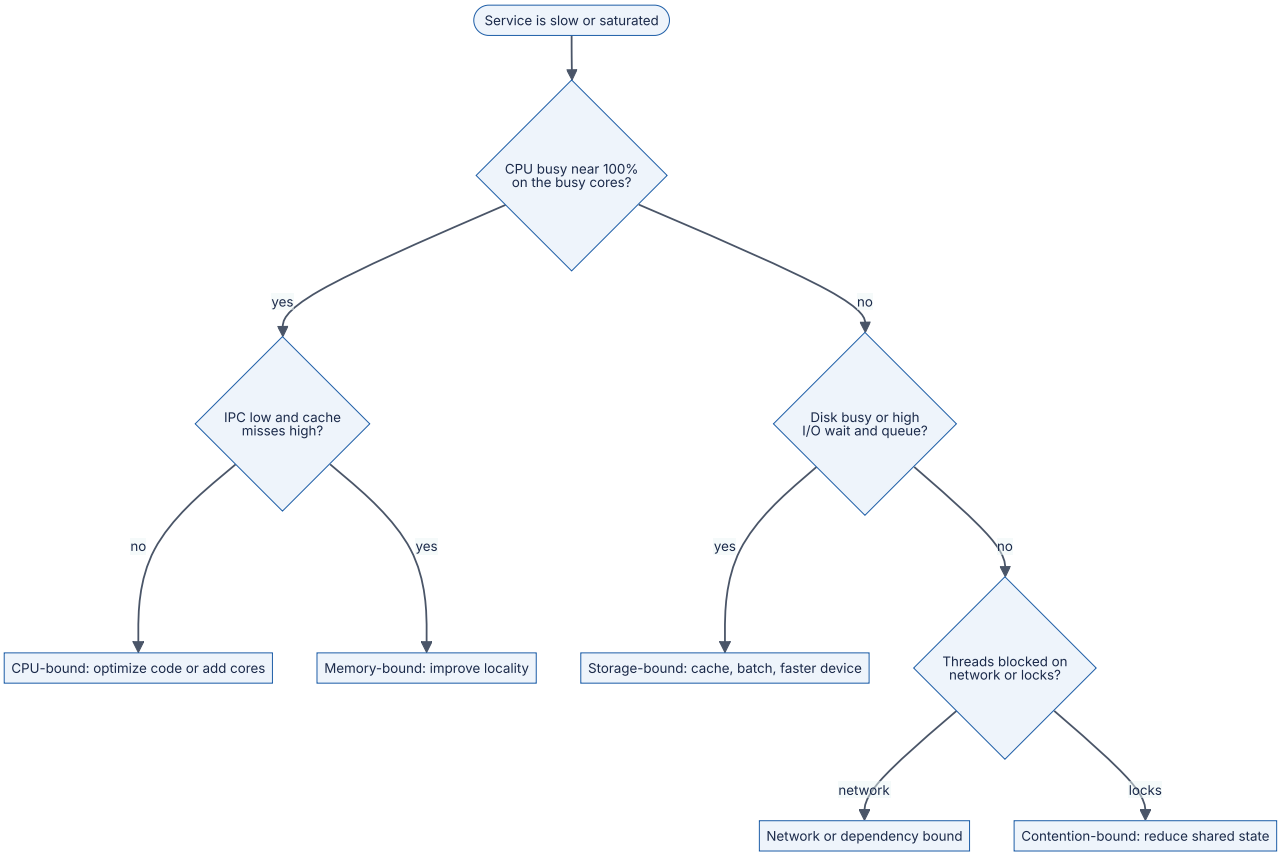



**Figure 5.9 — A two-socket NUMA server.**

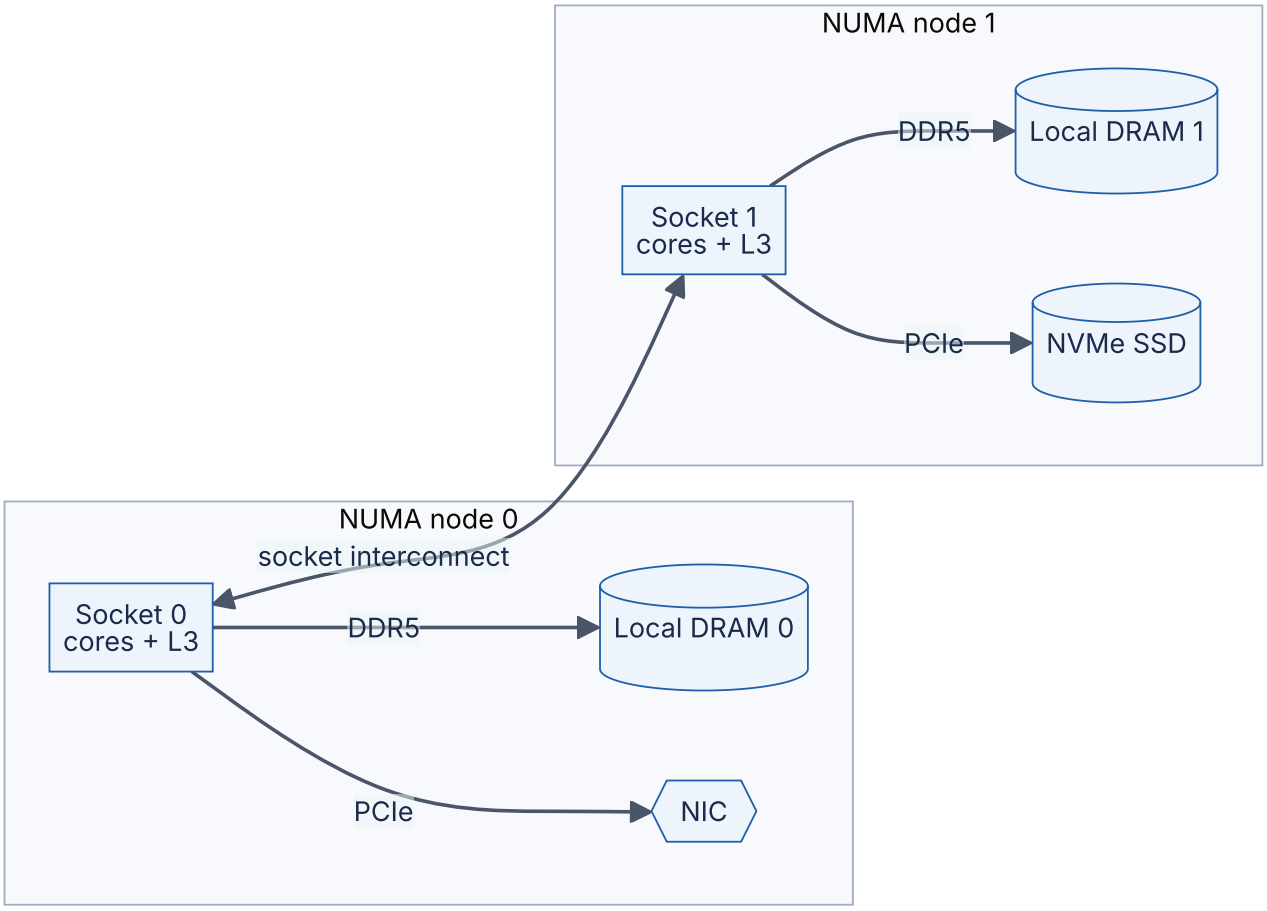
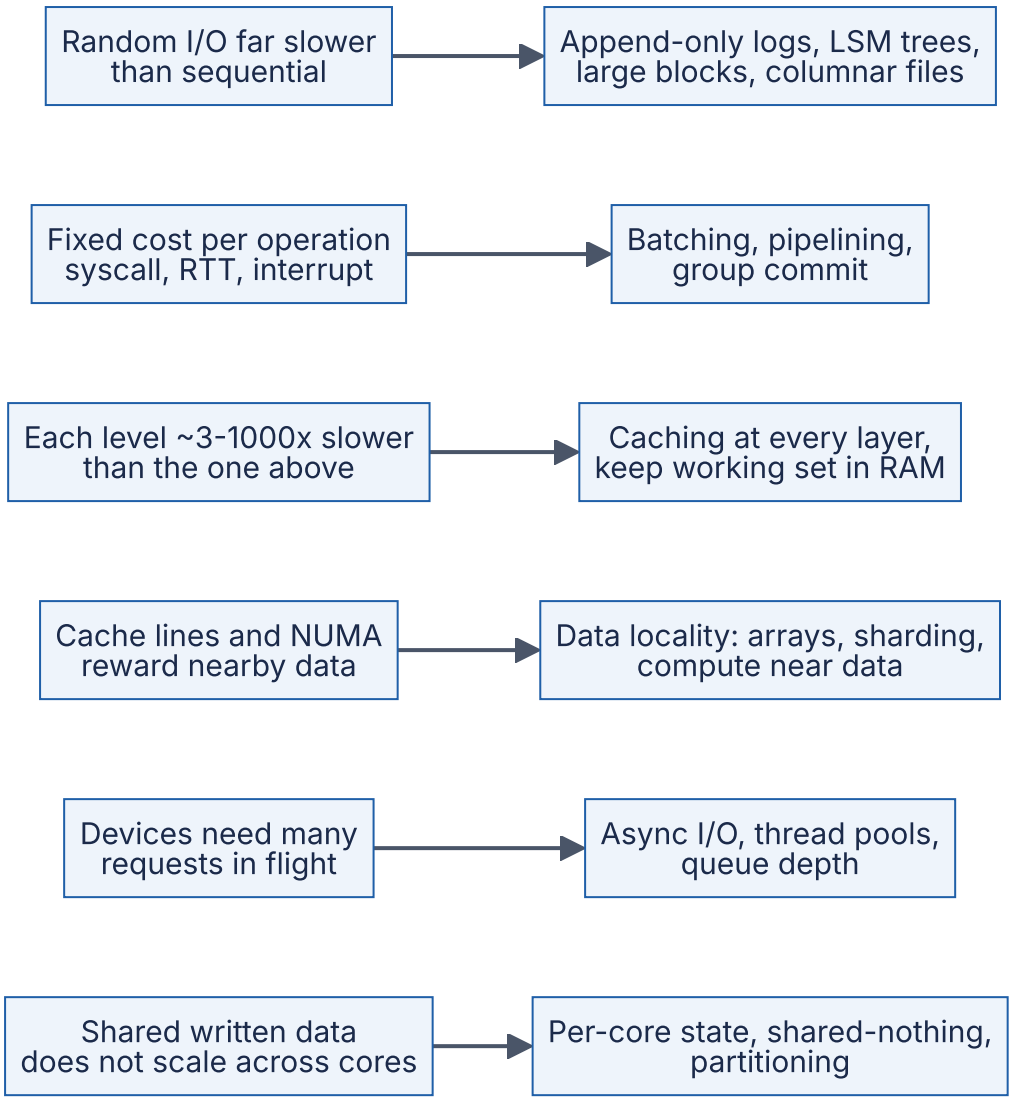



**Figure 5.10 — From hardware facts to design decisions.**




## Chapter 6 — Operating Systems

**Figure 6.1 — Layout of a process's virtual address space on Linux x86-64.**

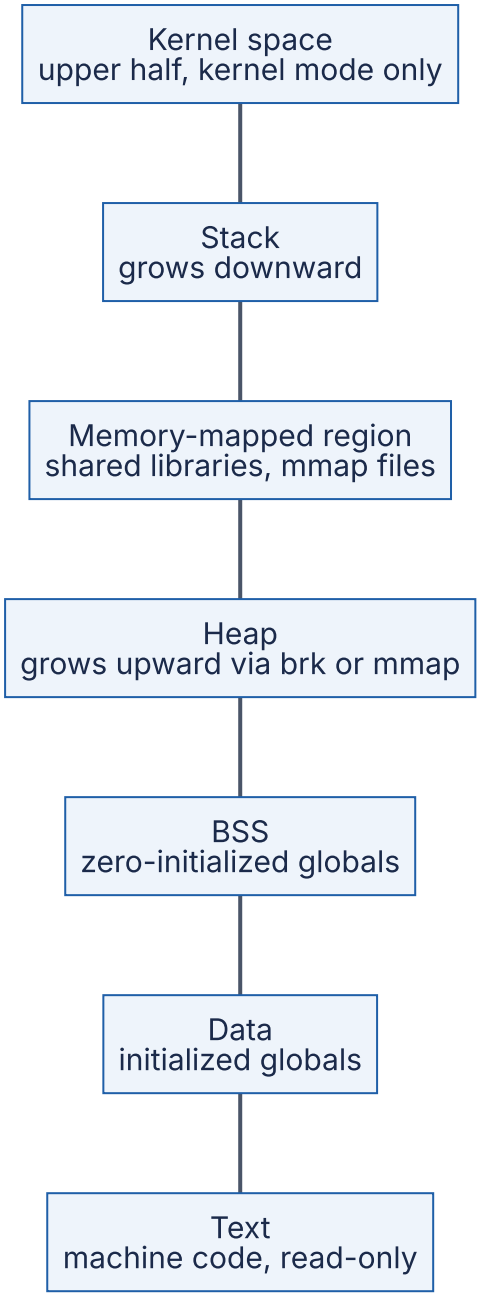



**Figure 6.2 — Process state diagram.**

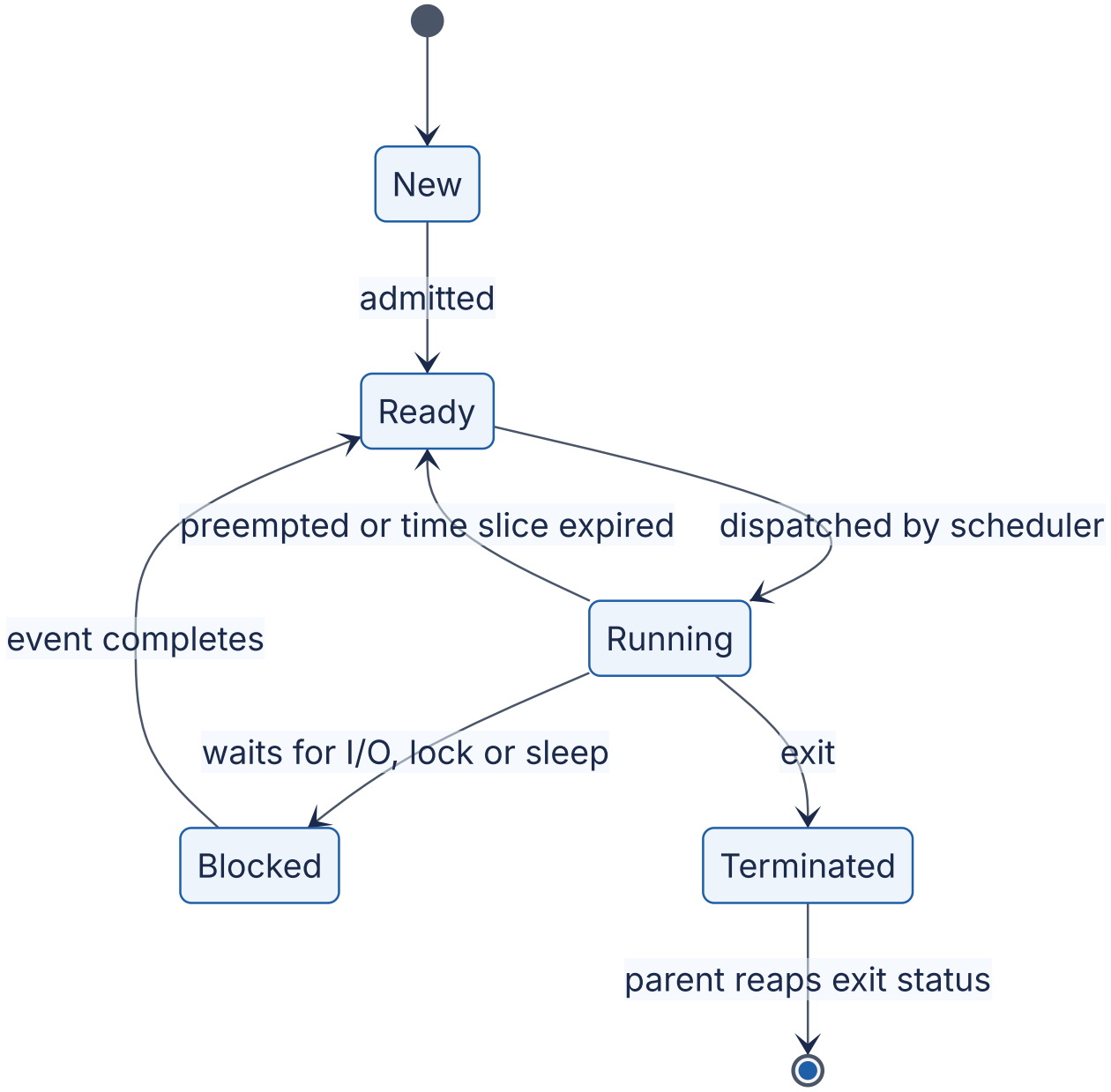



**Figure 6.3 — A context switch between two threads triggered by a timer interrupt.**

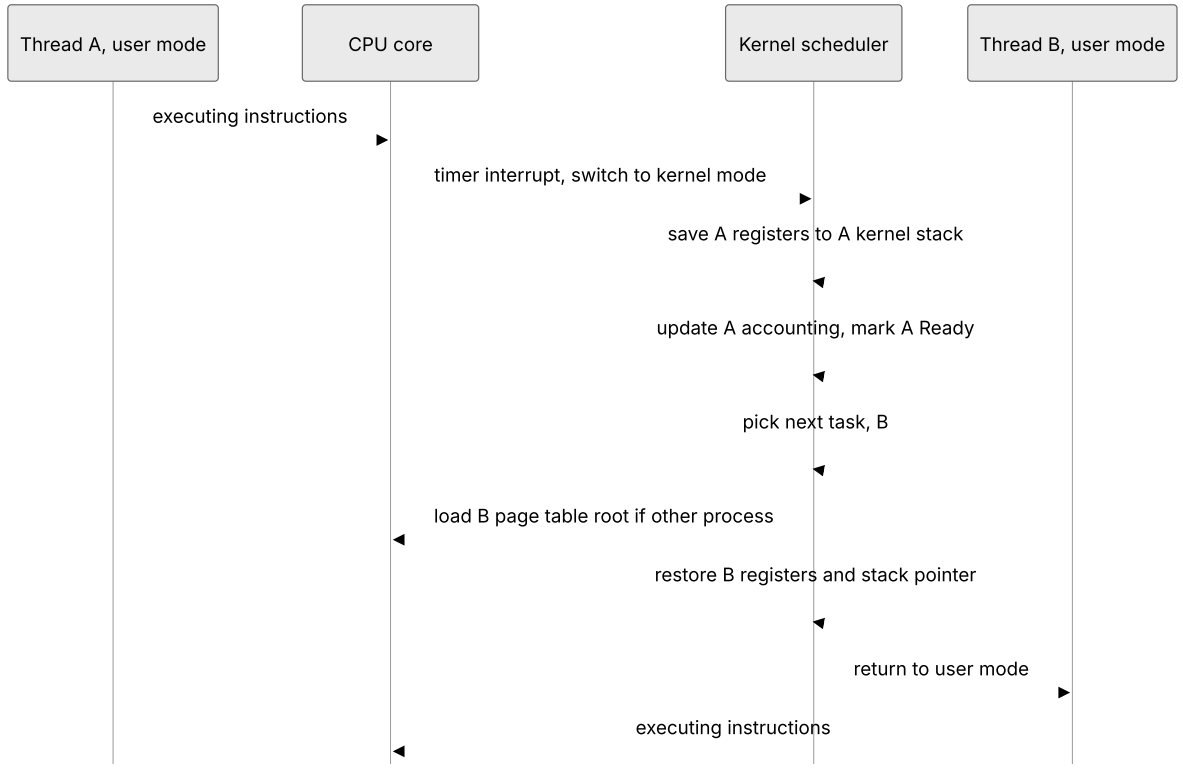



**Figure 6.4 — Multi-level feedback queue decisions.**

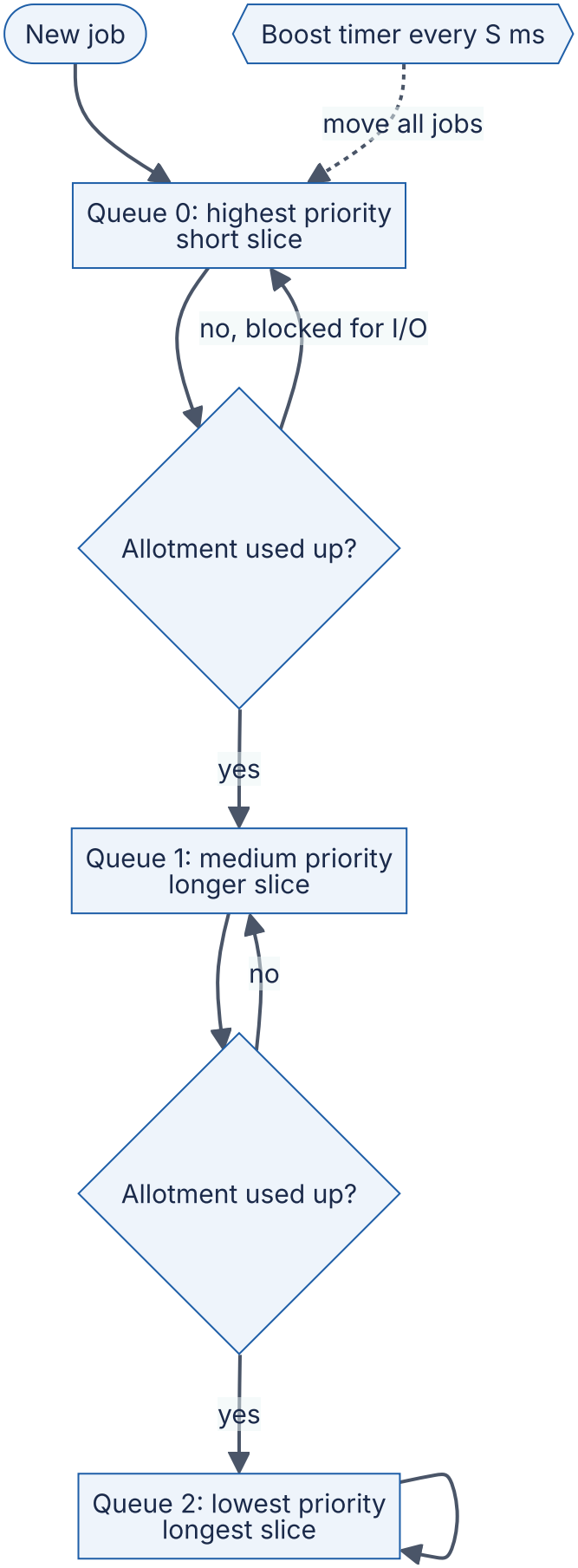
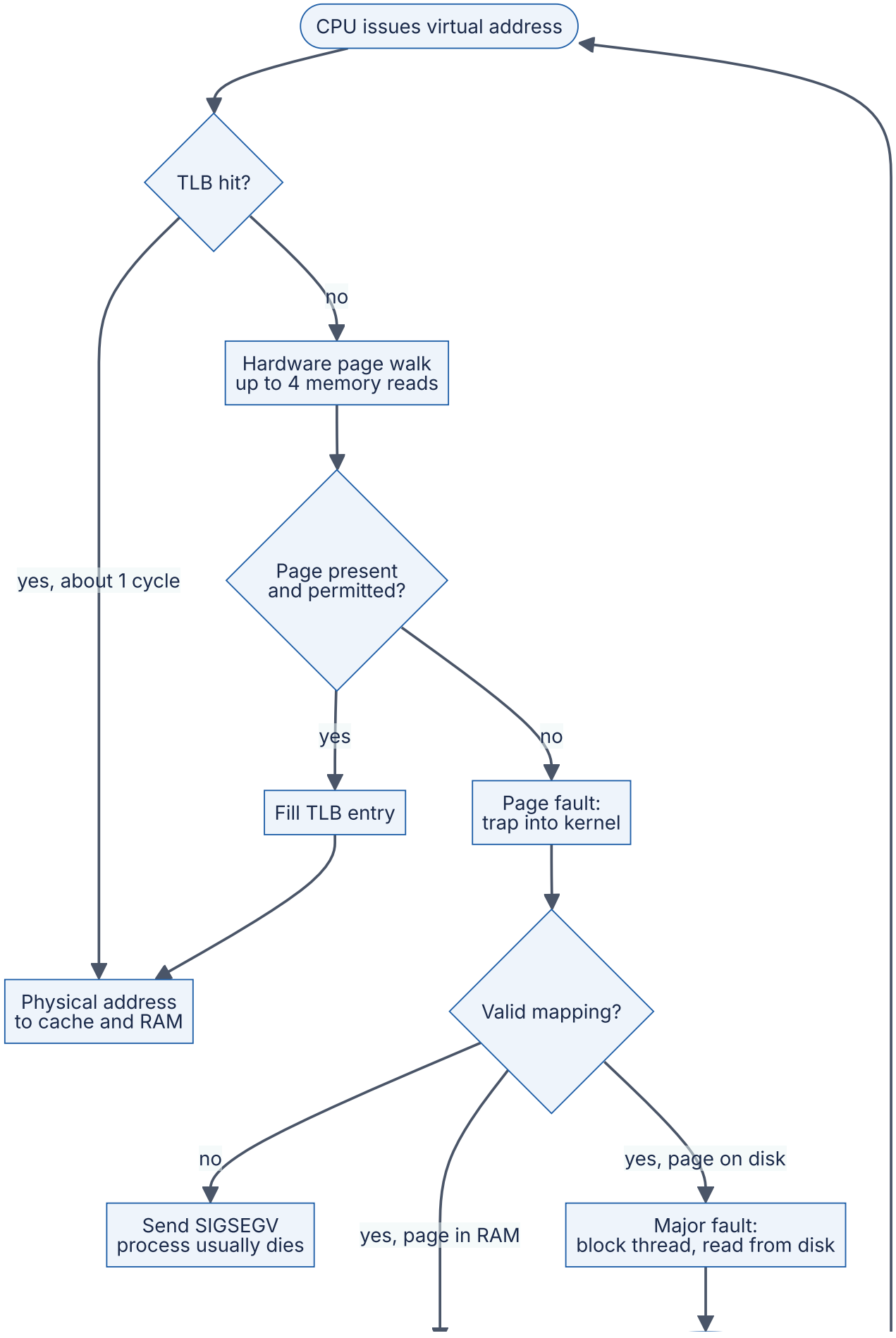

New job
Boost timer every S ms
move all jobs
Queue 0: highest priority short slice
no, blocked for I/O
Allotment used up?
yes
Queue 1: medium priority longer slice
no
Allotment used up?
yes
Queue 2: lowest priority longest slice

**Figure 6.5 — Translating a virtual address: TLB, page walk and page fault.**


CPU issues virtual address
TLB hit?
yes, about 1 cycle
no
Hardware page walk
up to 4 memory reads
Page present
and permitted?
yes
Fill TLB entry
no
Page fault:
trap into kernel
Physical address
to cache and RAM
Valid mapping?
no
Send SIGSEGV
process usually dies
yes, page in RAM
yes, page on disk
Major fault:
block thread, read from disk

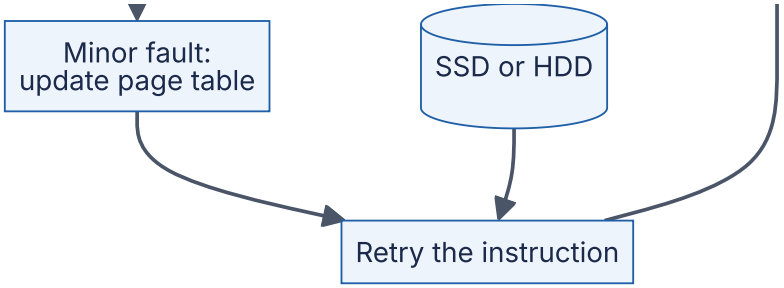



**Figure 6.6 — A `read()` system call that must wait for the disk.**

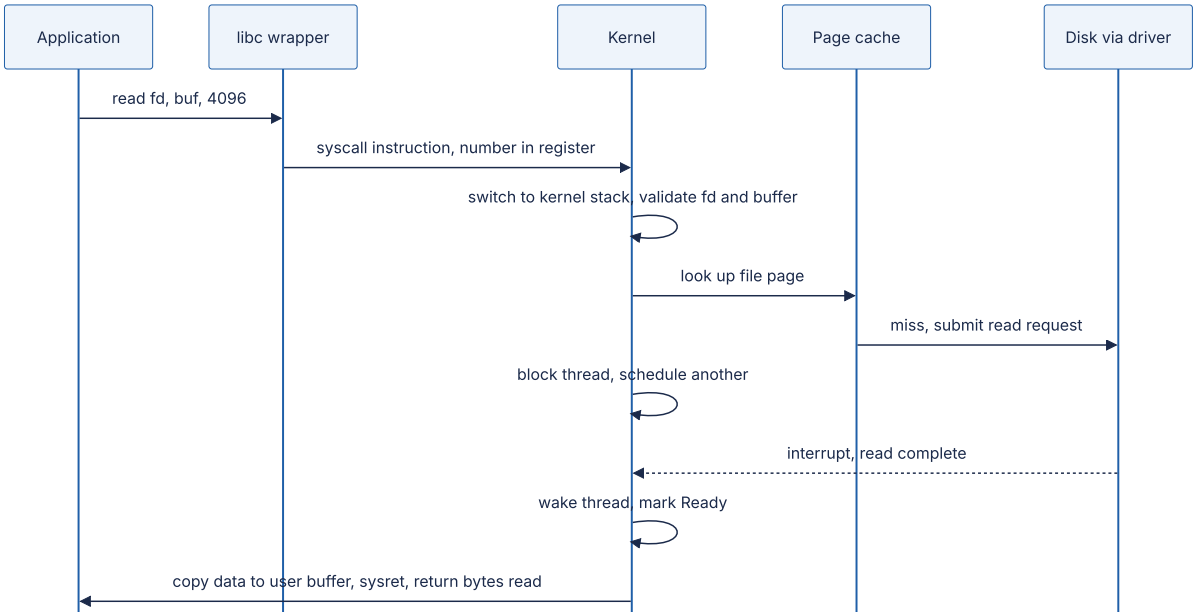



**Figure 6.7 — Two threads lose an update to a shared counter.**

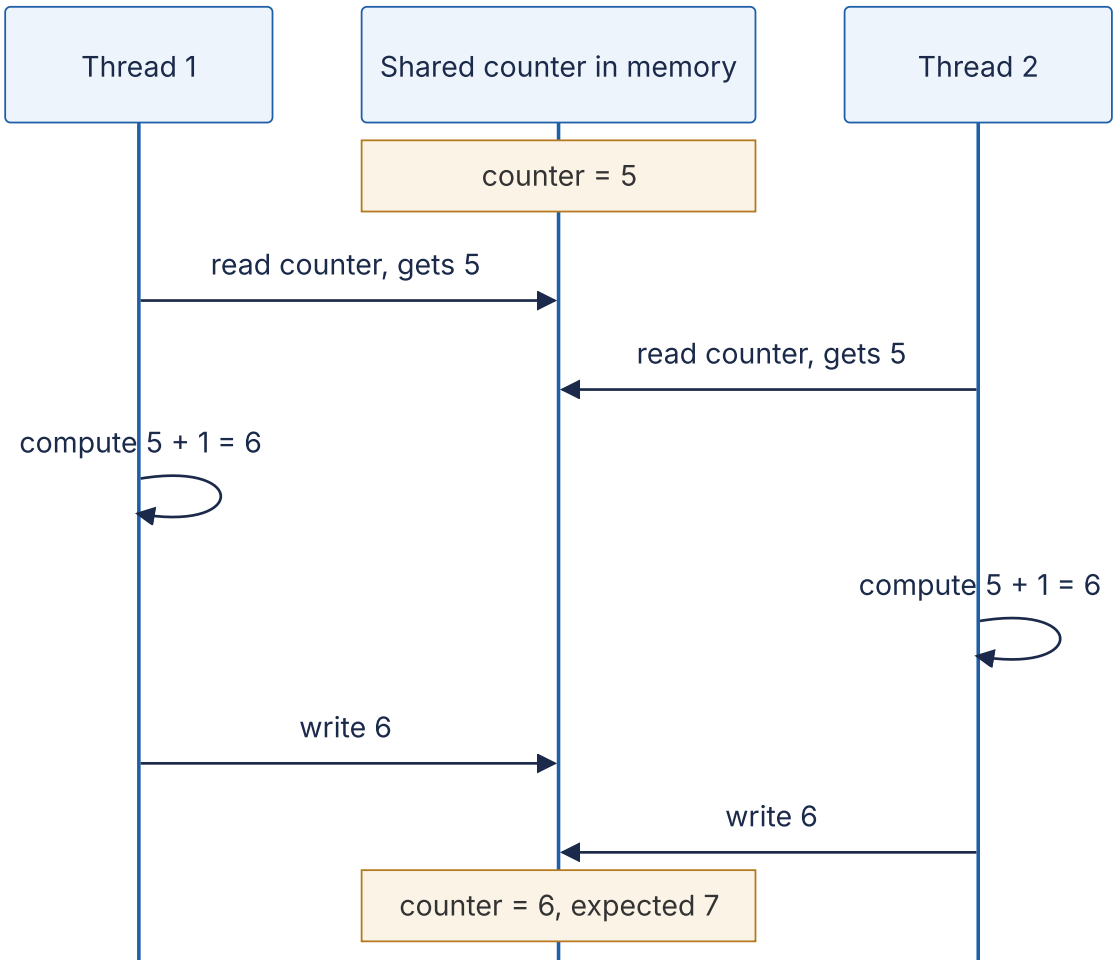



**Figure 6.8 — Resource-allocation graph with a deadlock cycle.**

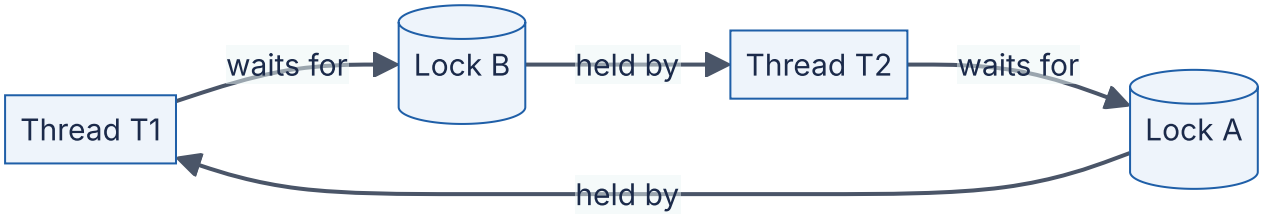



**Figure 6.9 — Priority inversion on Mars Pathfinder.**

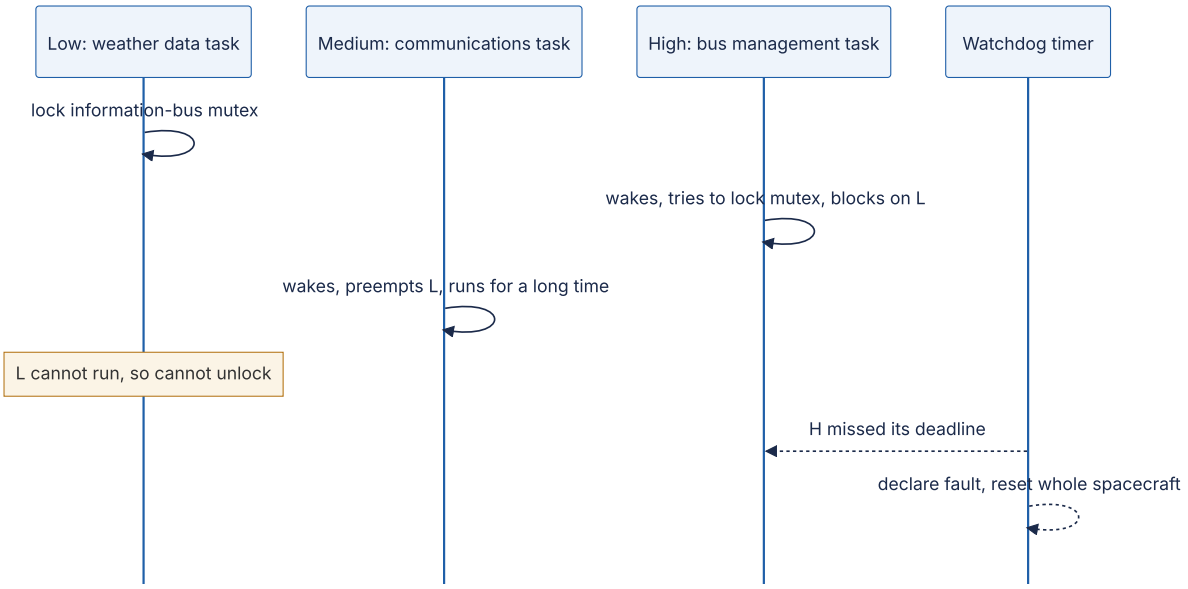



**Figure 6.10 — An event loop on epoll.**

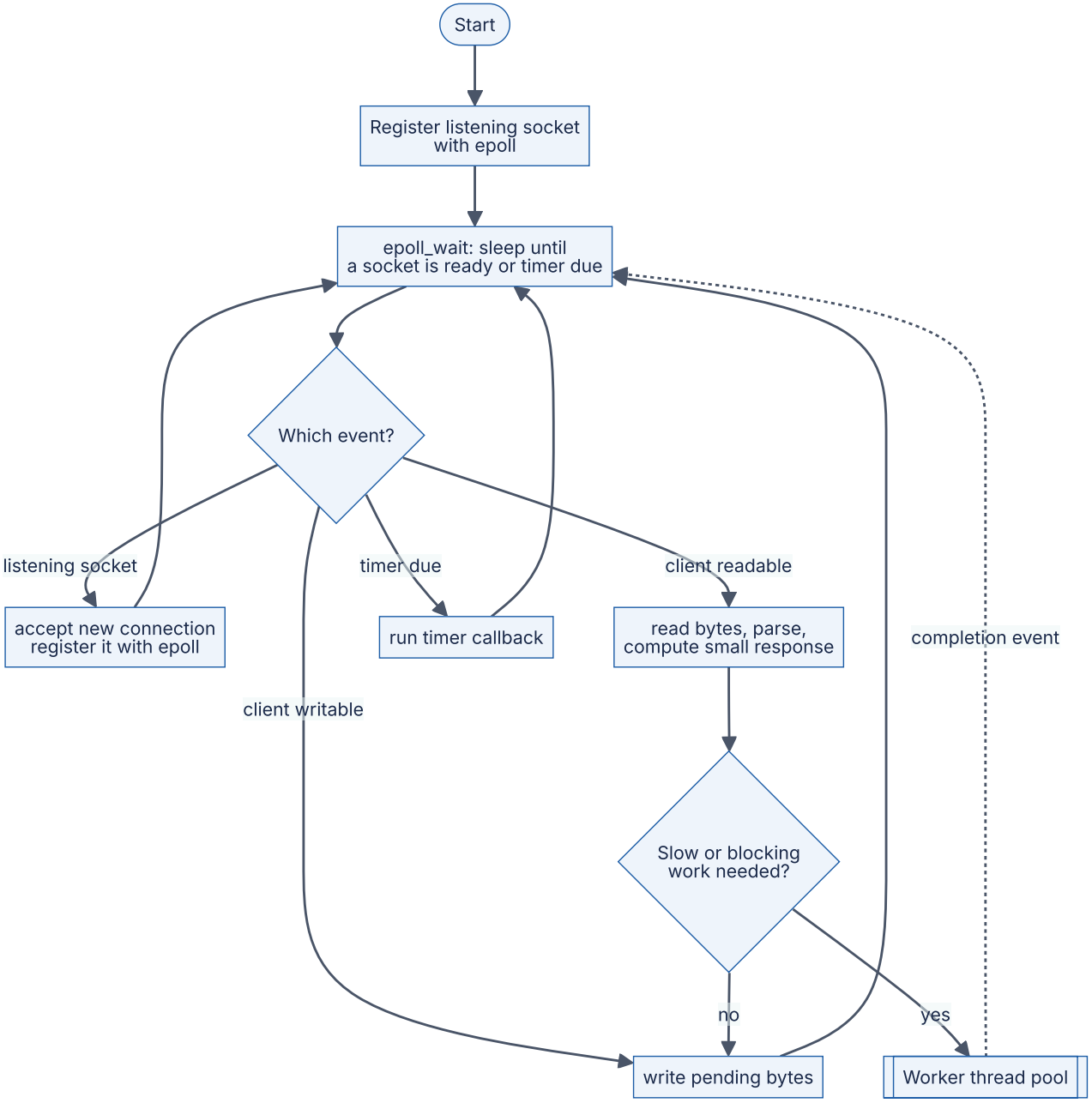



**Figure 6.11 — io_uring submission and completion rings.**

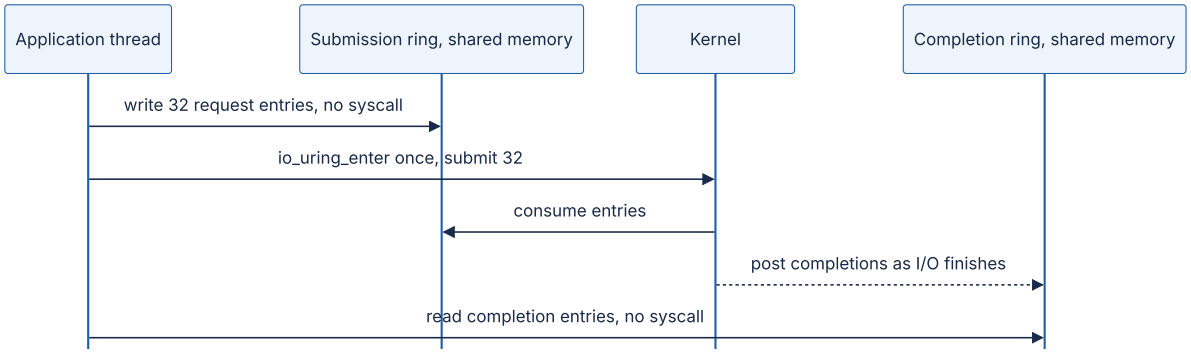
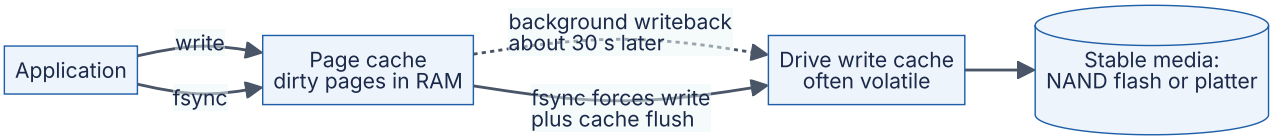



**Figure 6.12 — The write path from application to stable storage.**




# Chapter 7 — Networking Fundamentals

**Figure 7.1 — Encapsulation down the sender’s stack and decapsulation up the receiver’s.**

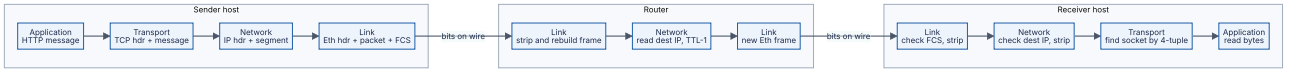



**Figure 7.2 — BGP routing between autonomous systems, with an anycast prefix announced from two sites.**

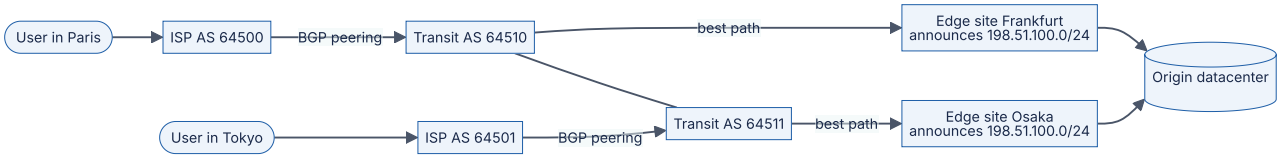



**Figure 7.3 — DNS resolution of www.example.com with an empty cache.**

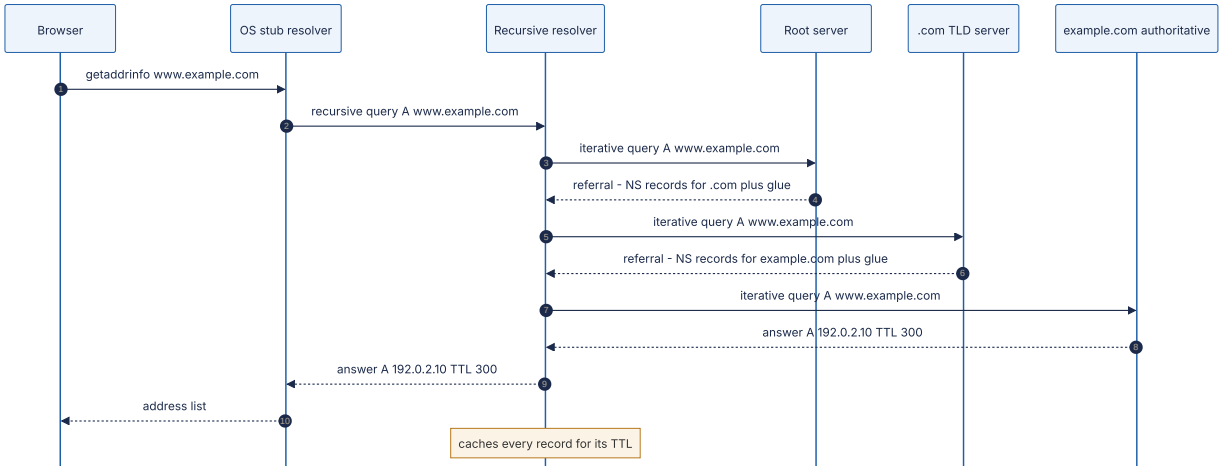



**Figure 7.4 — TCP three-way handshake followed by the first request.**

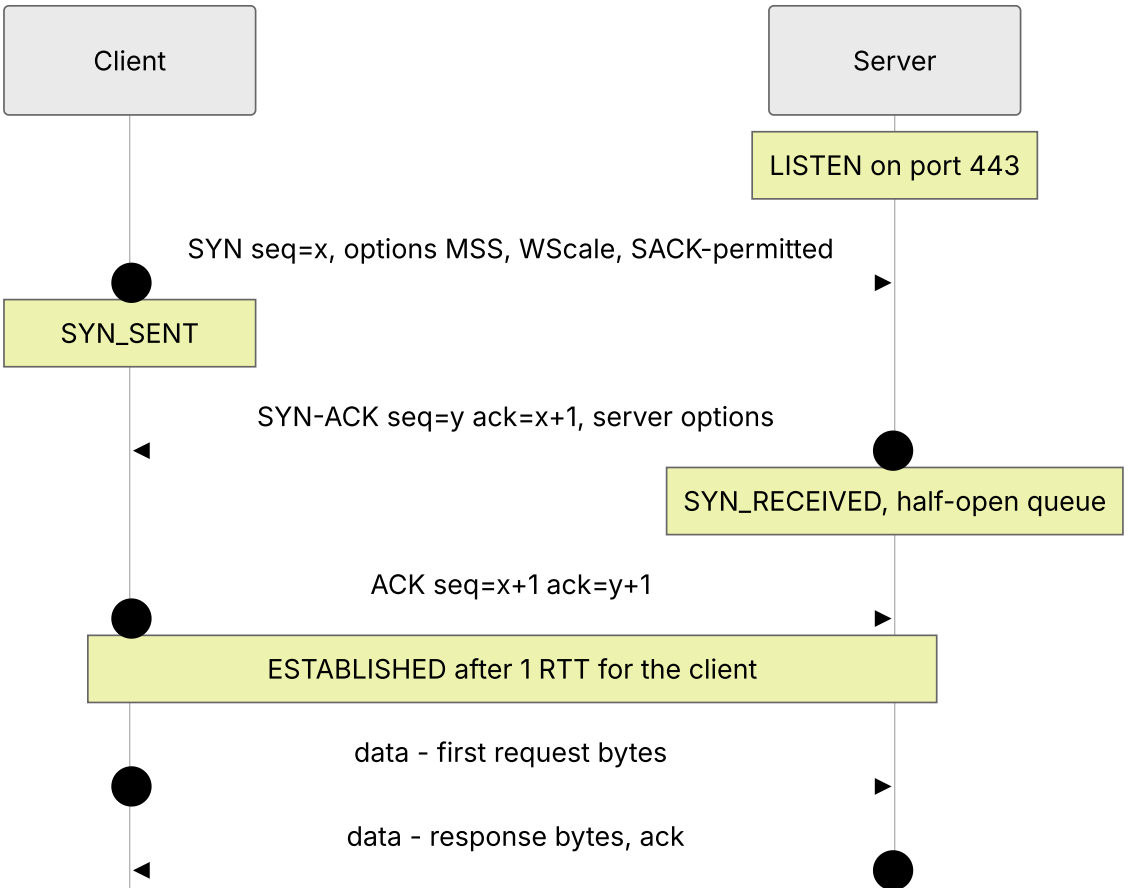



**Figure 7.5 — TCP four-way close and the TIME_WAIT state.**

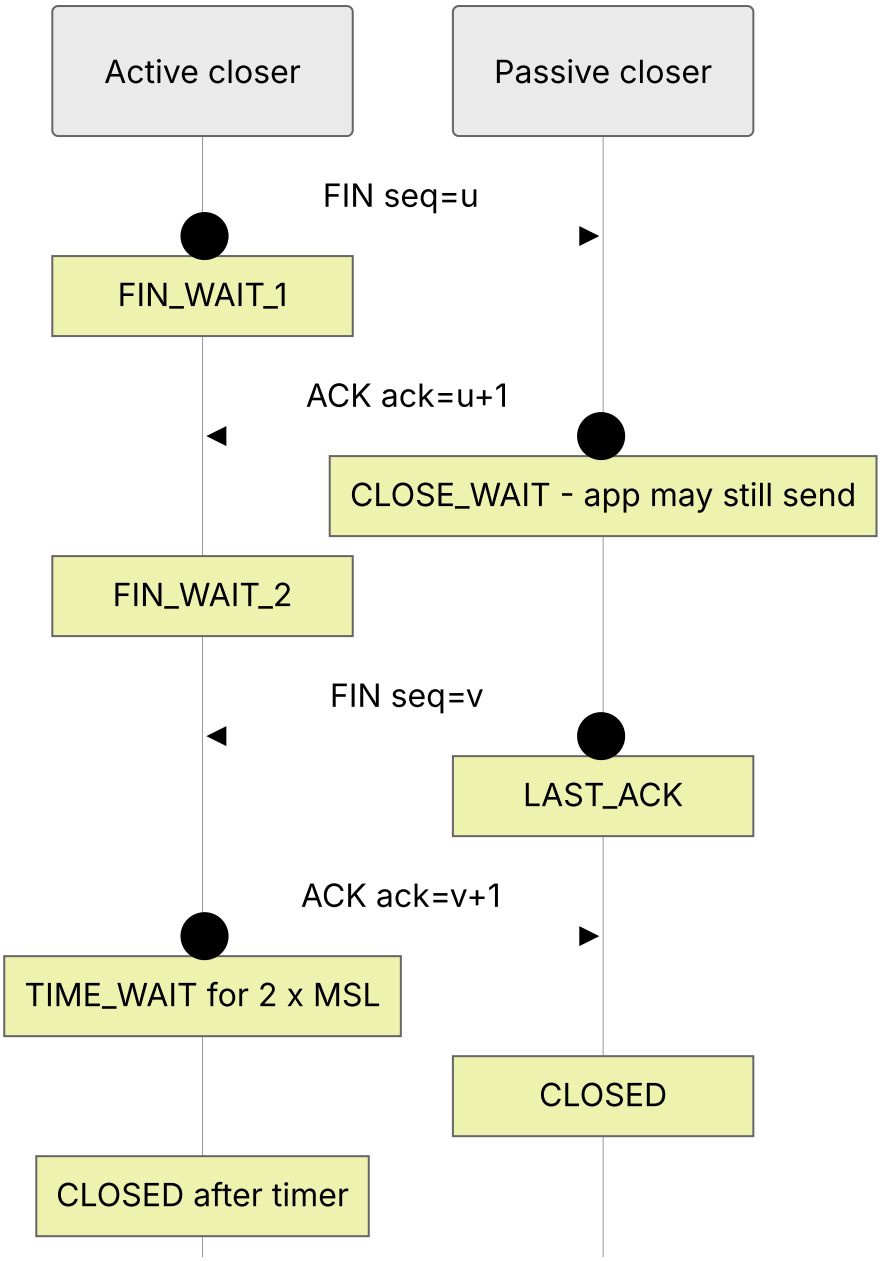



**Figure 7.6 — Loss-based TCP congestion control states (Reno-style).**

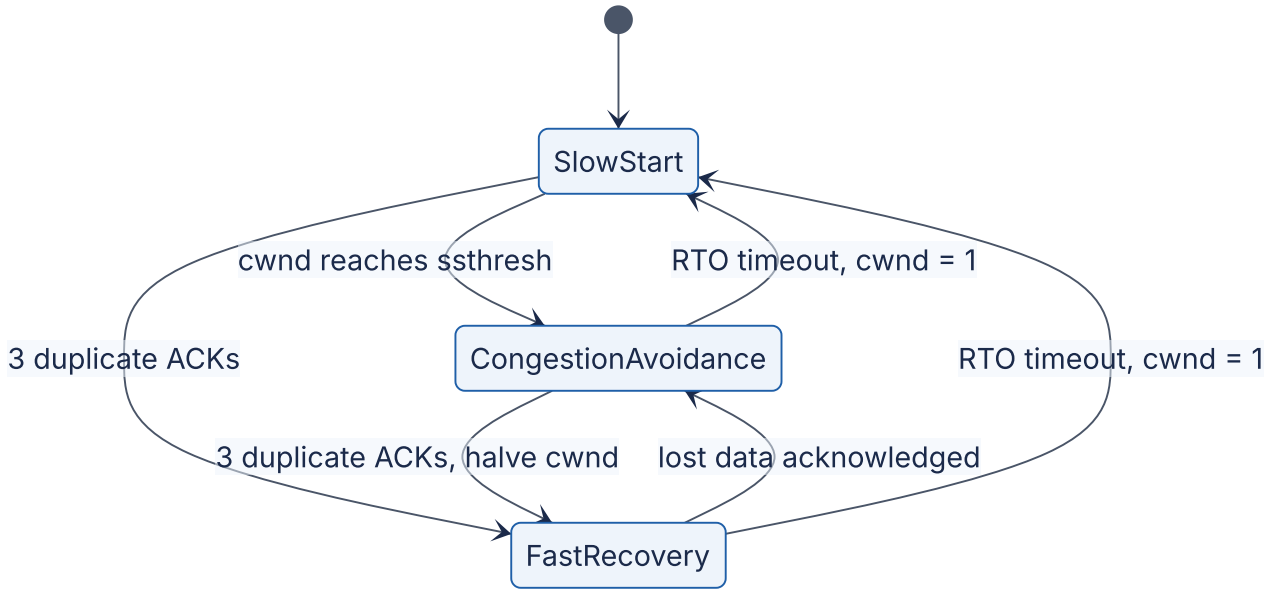



**Figure 7.7 — Protocol stacks for the three HTTP versions.**

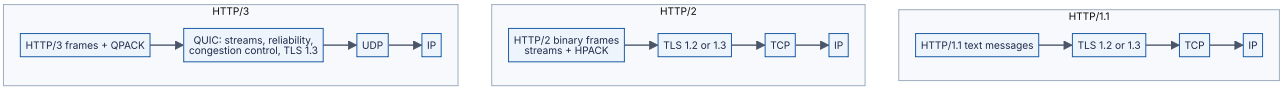



**Figure 7.8 — TLS 1.3 full handshake (after the TCP handshake).**

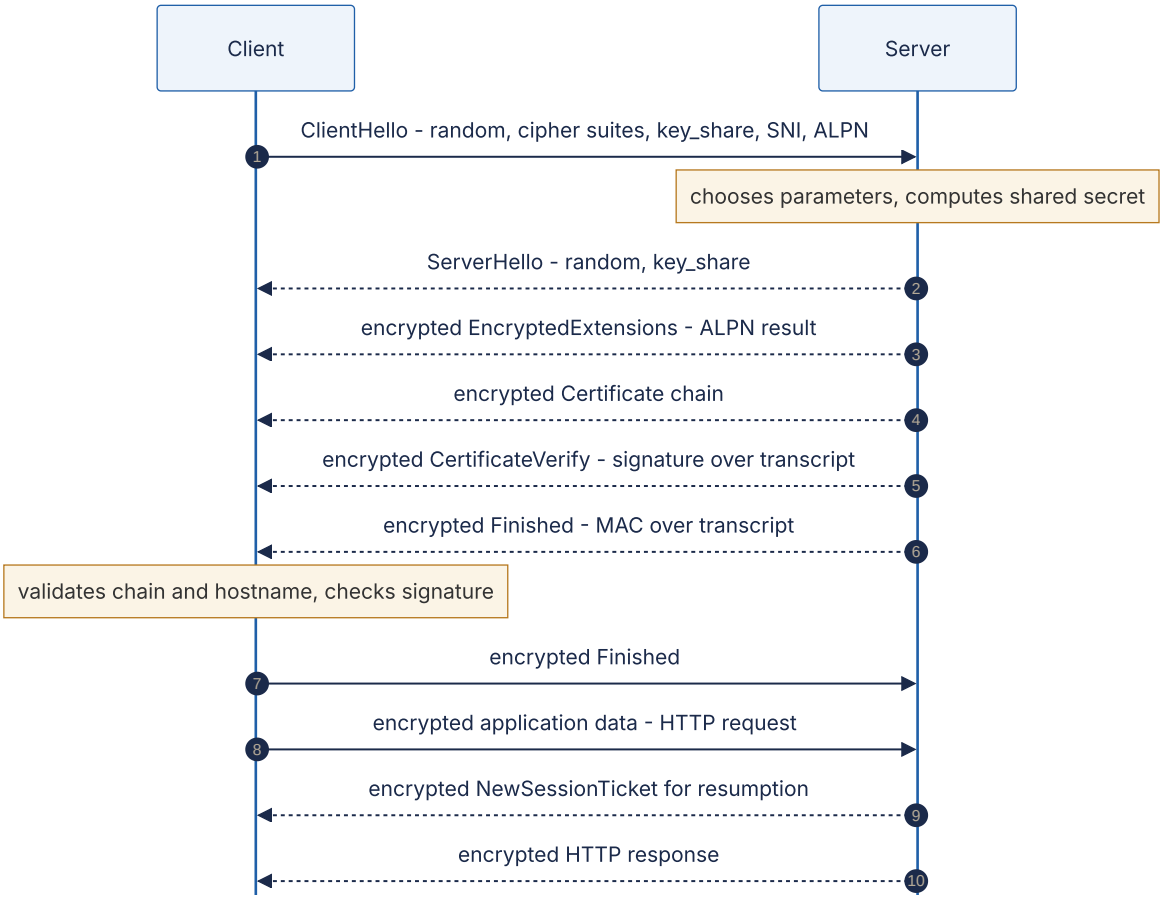


**Figure 7.9 — WebSocket opening handshake and framing.**

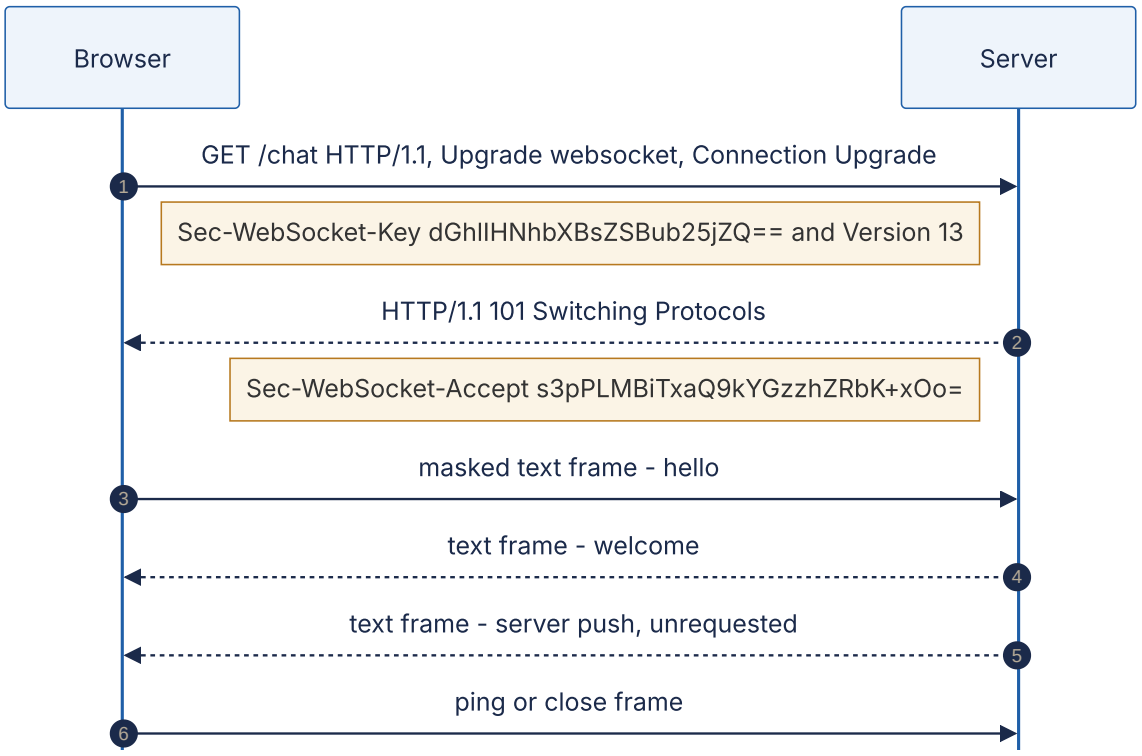



**Figure 7.10 — Forward proxy versus reverse proxy.**

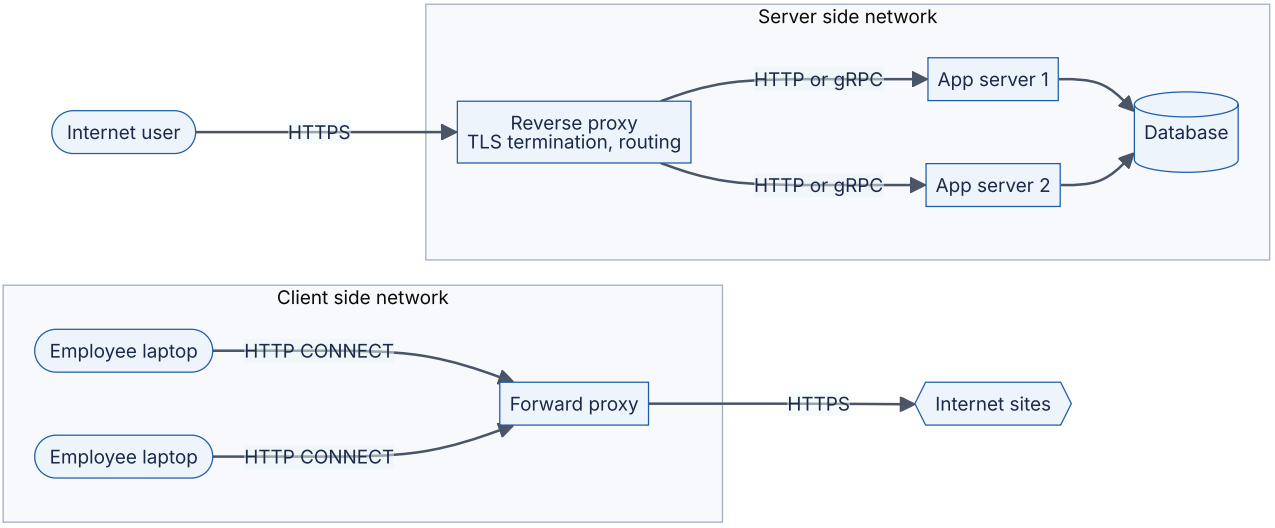



**Figure 7.11 — Source NAT for an outbound connection.**

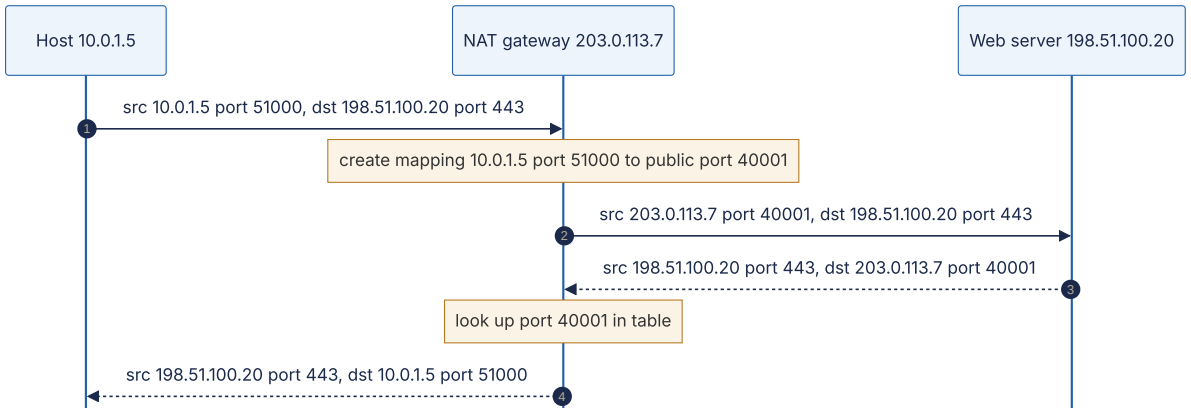



**Figure 7.12 — A leaf-spine fabric with ECMP.**

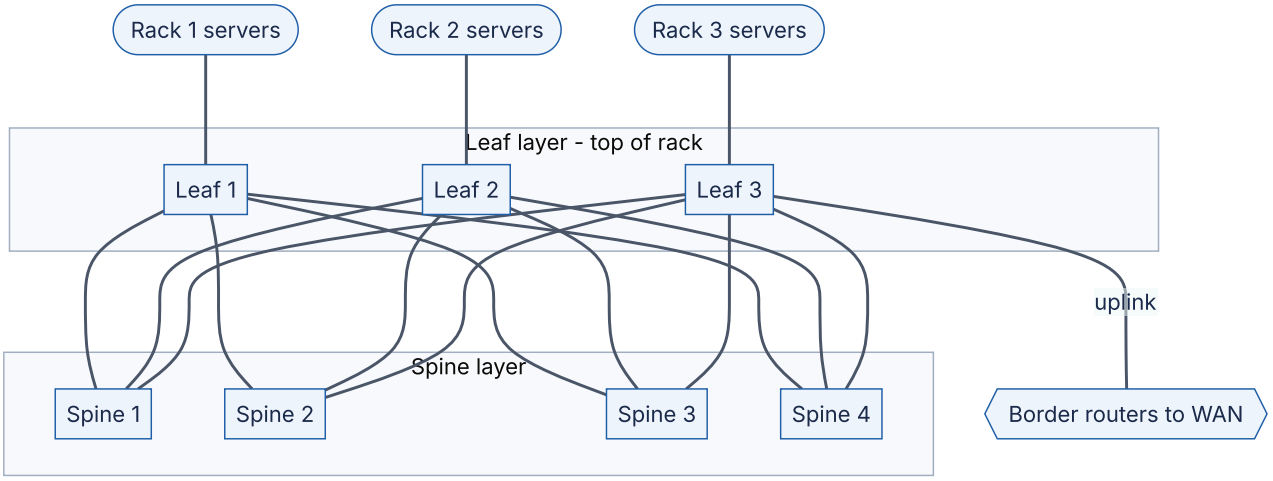
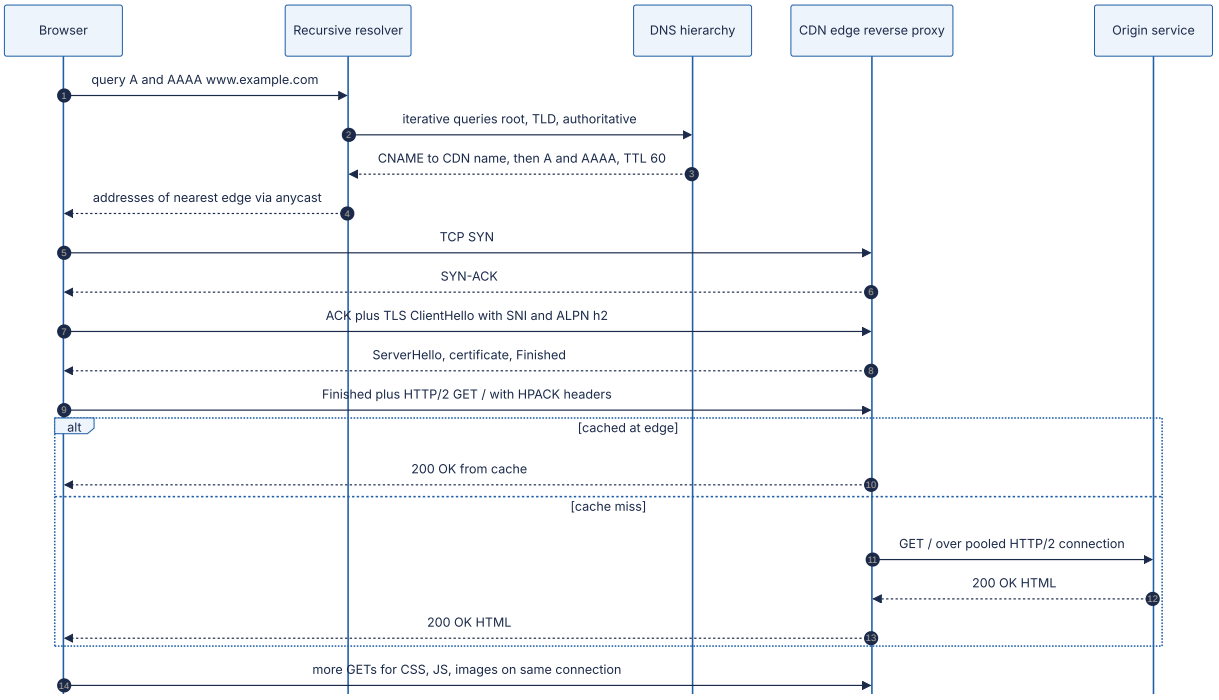



**Figure 7.13 — Loading https://www.example.com/ end to end.**



# Chapter 8 — Web Application Fundamentals

**Figure 8.1 — Three-tier client-server architecture.**

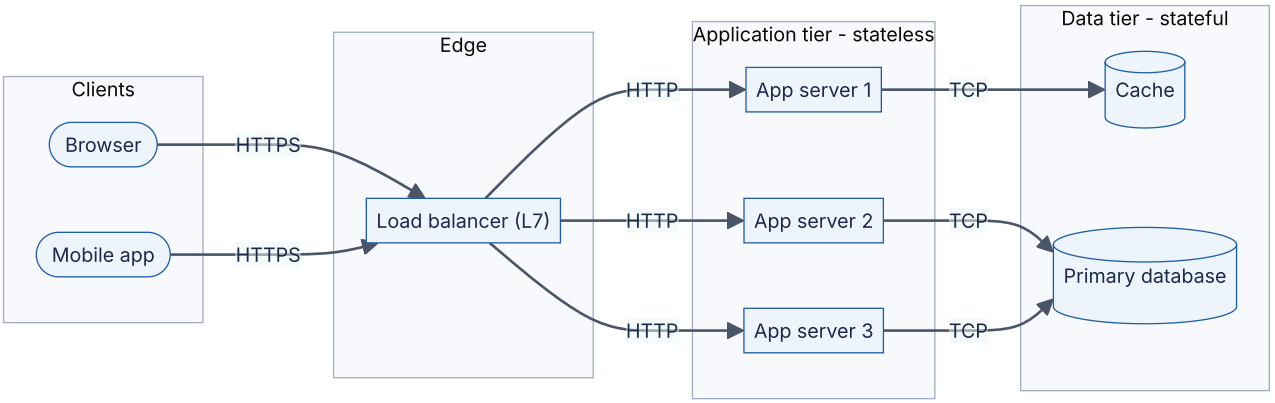



**Figure 8.2 — Backend-for-frontend aggregation.**

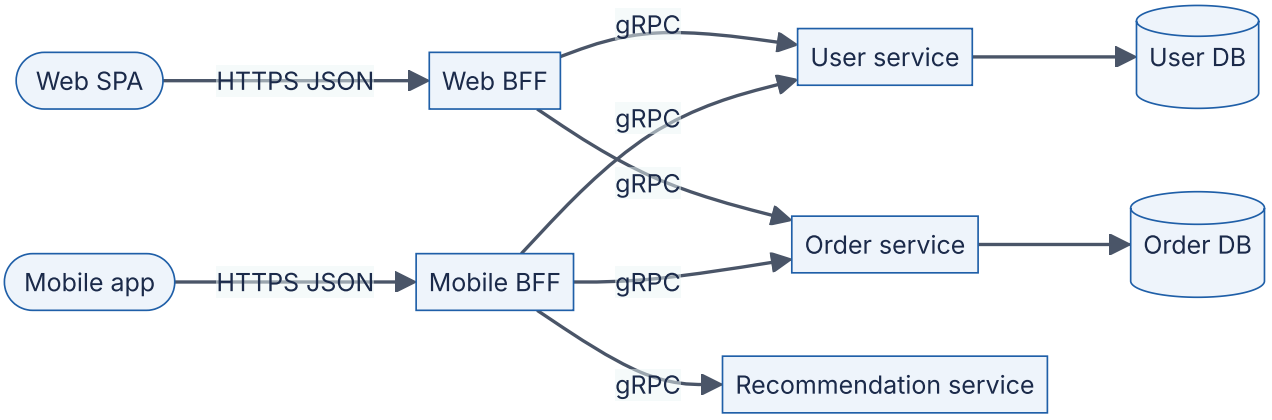



**Figure 8.3 — End-to-end lifecycle of one HTTPS API request.**

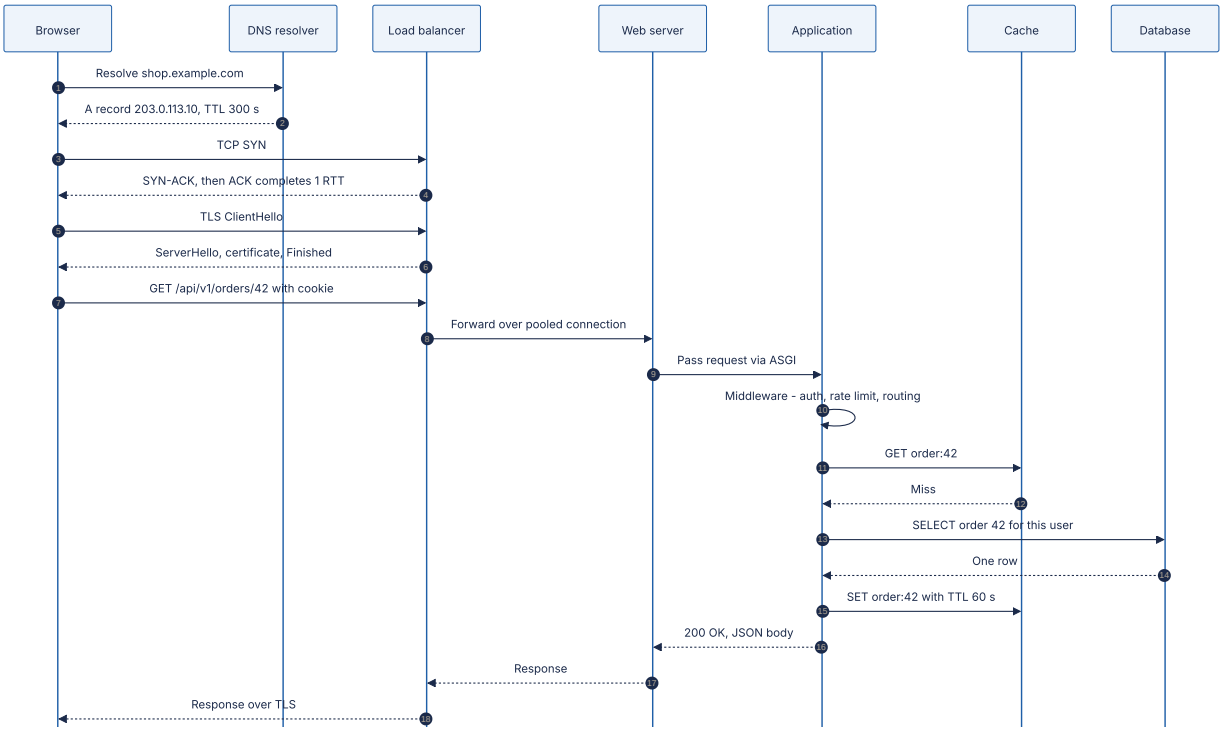



**Figure 8.4 — N+1 resolution versus DataLoader batching.**

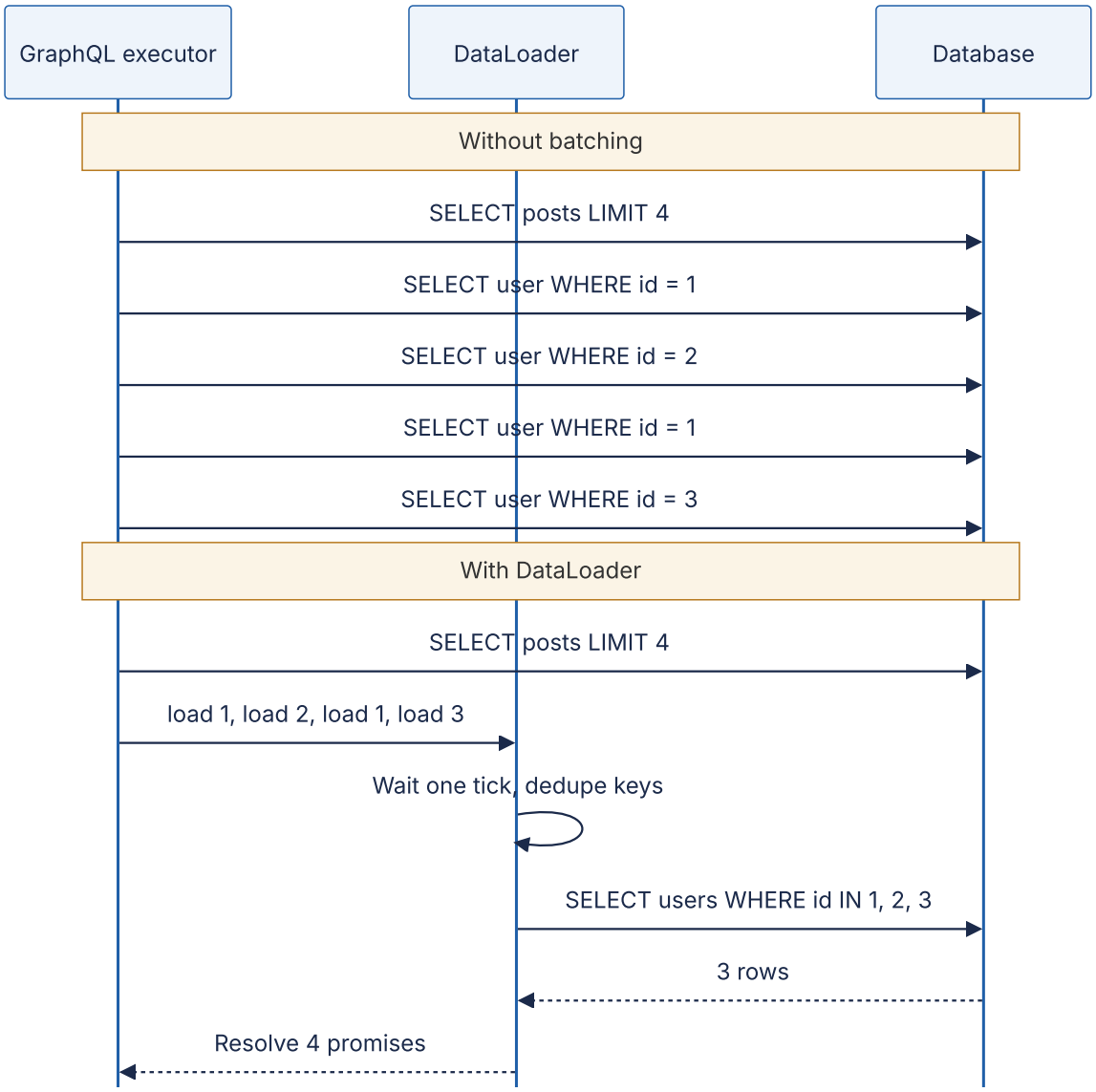



**Figure 8.5 — gRPC call mapped onto an HTTP/2 connection.**

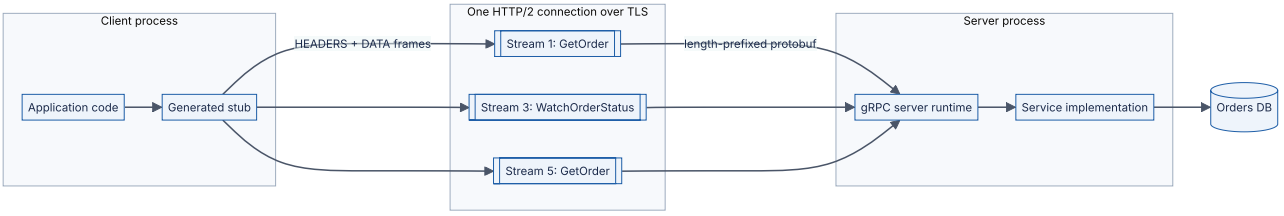



**Figure 8.6 — Deadline propagation through a call chain.**

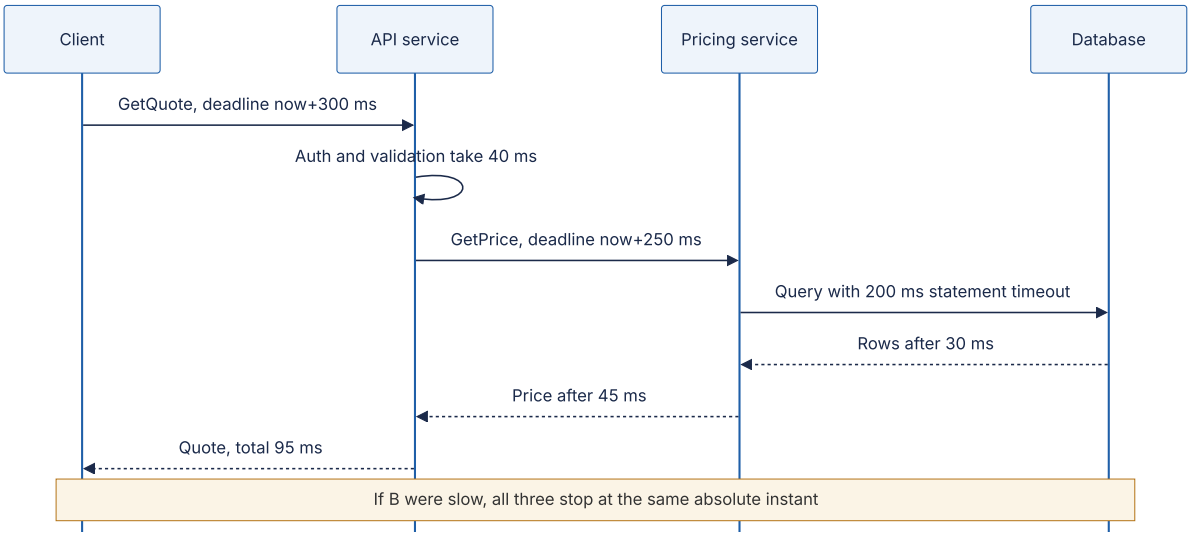



**Figure 8.7 — Mixed versions during a rolling deployment.**

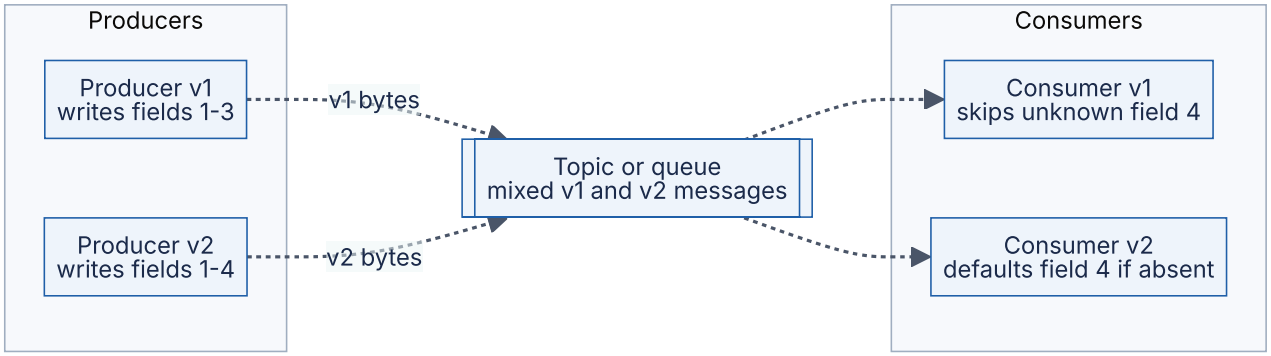



**Figure 8.8 — Server-side session lookup.**

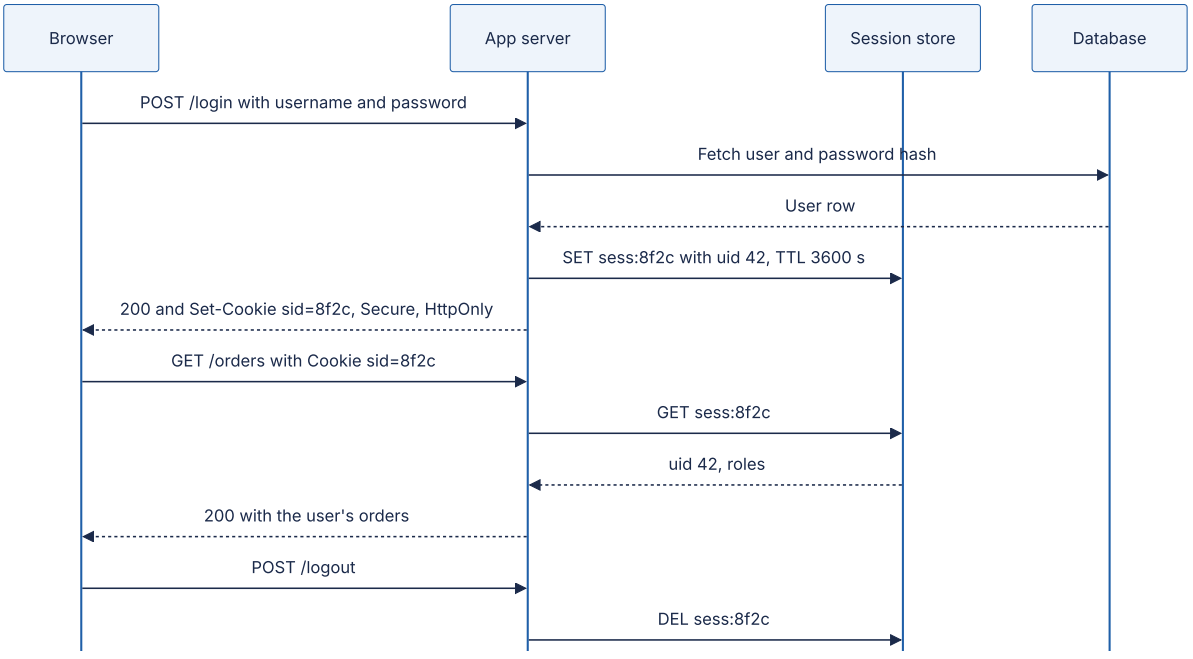



**Figure 8.9 — Relationship graph for a Zanzibar-style check.**

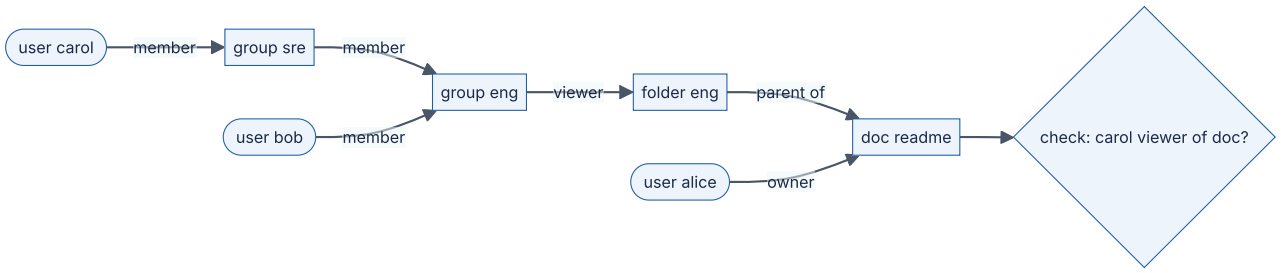
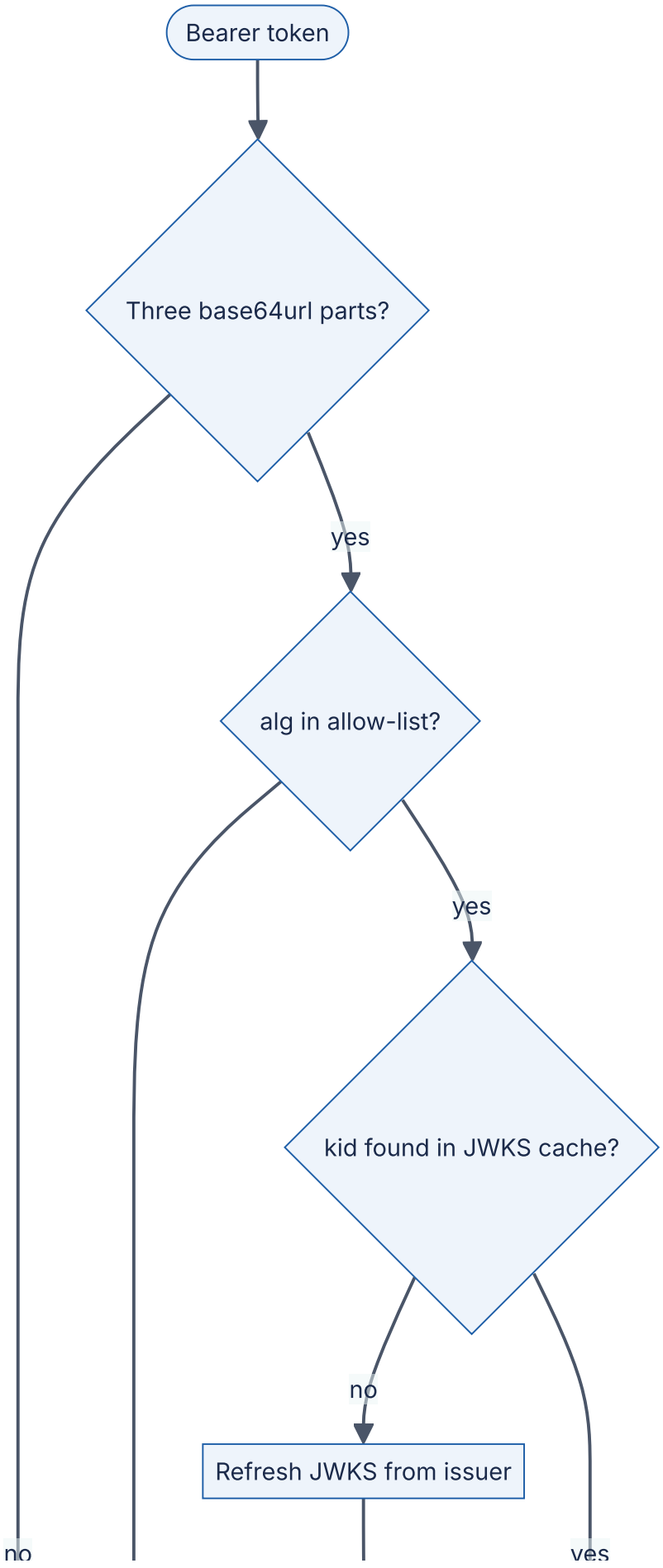



**Figure 8.10 — JWT validation pipeline.**


Bearer token
Three base64url parts?
yes
alg in allow-list?
yes
kid found in JWKS cache?
no
Refresh JWKS from issuer
no
yes

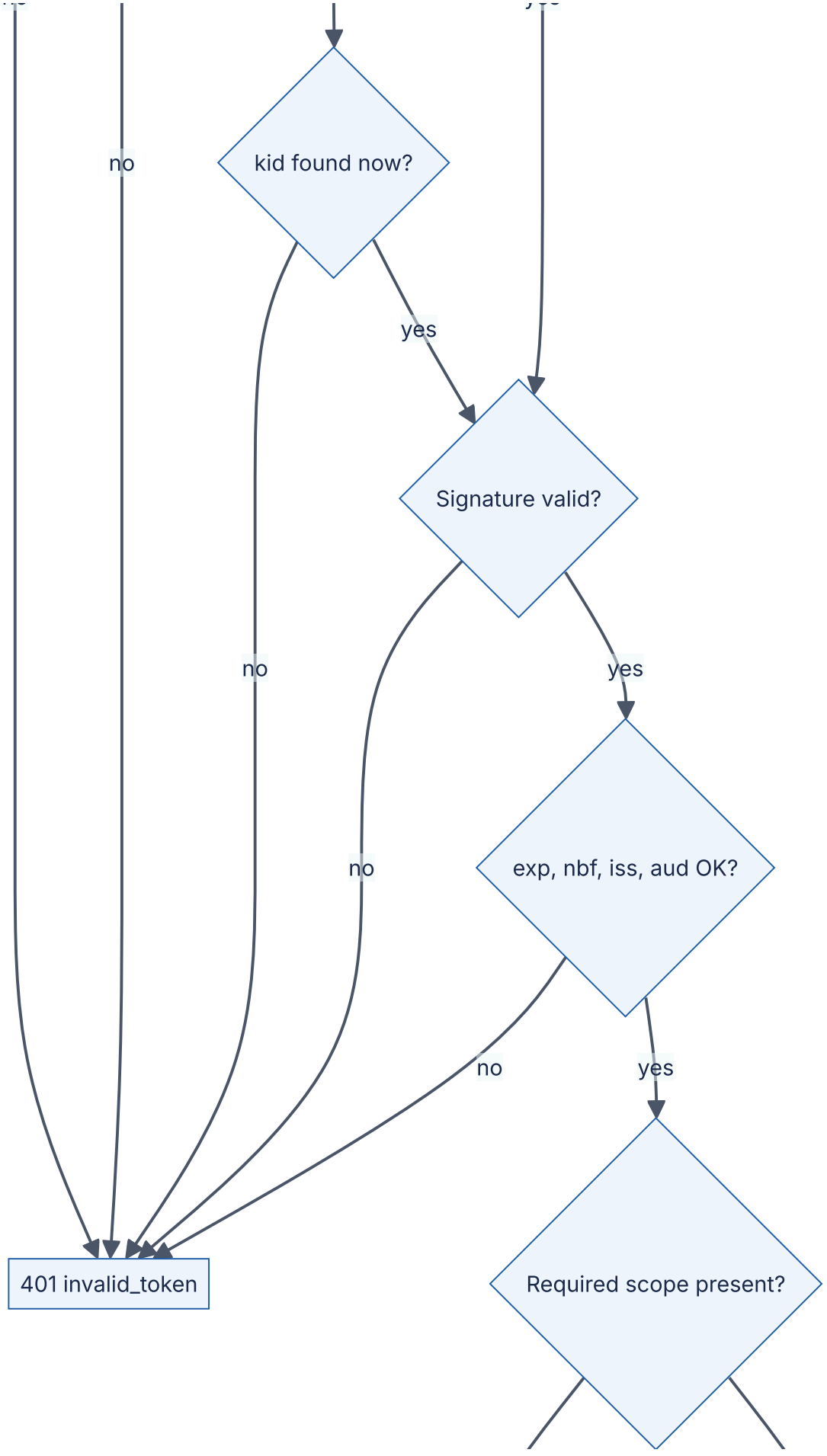

no
kid found now?
yes
Signature valid?
no
yes
no
exp, nbf, iss, aud OK?
no
yes
401 invalid_token
Required scope present?




**Figure 8.11 — OAuth 2.0 authorization code flow with PKCE.**

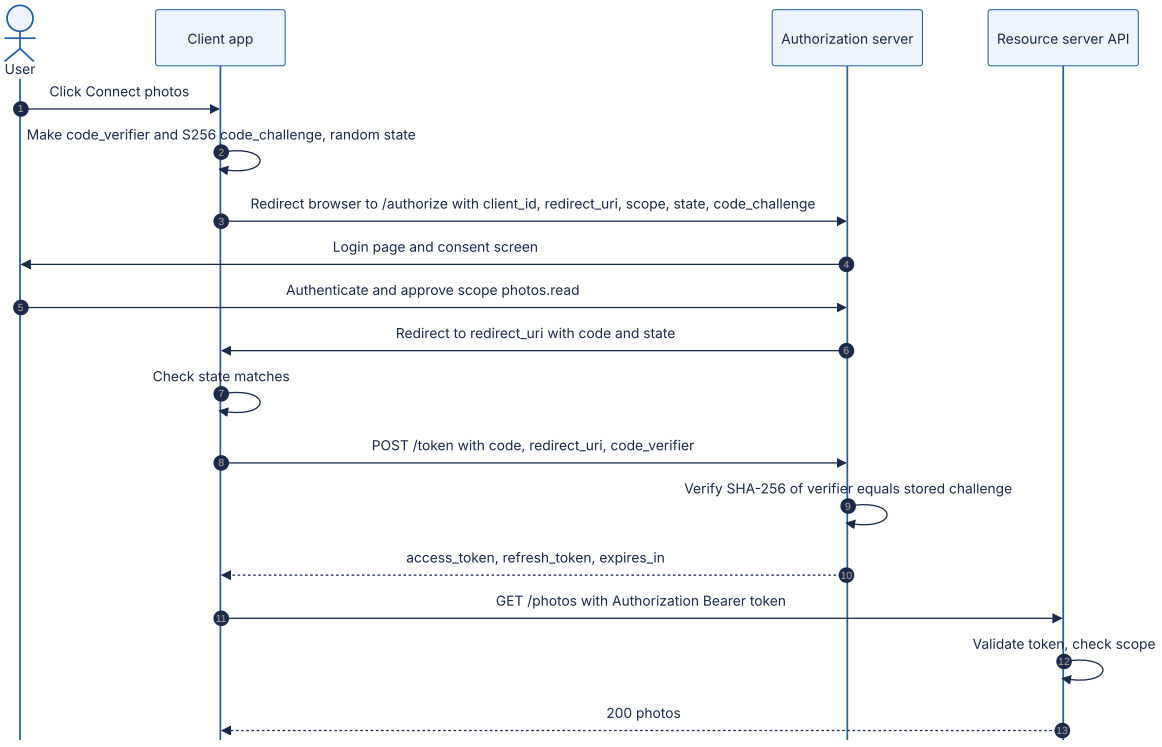
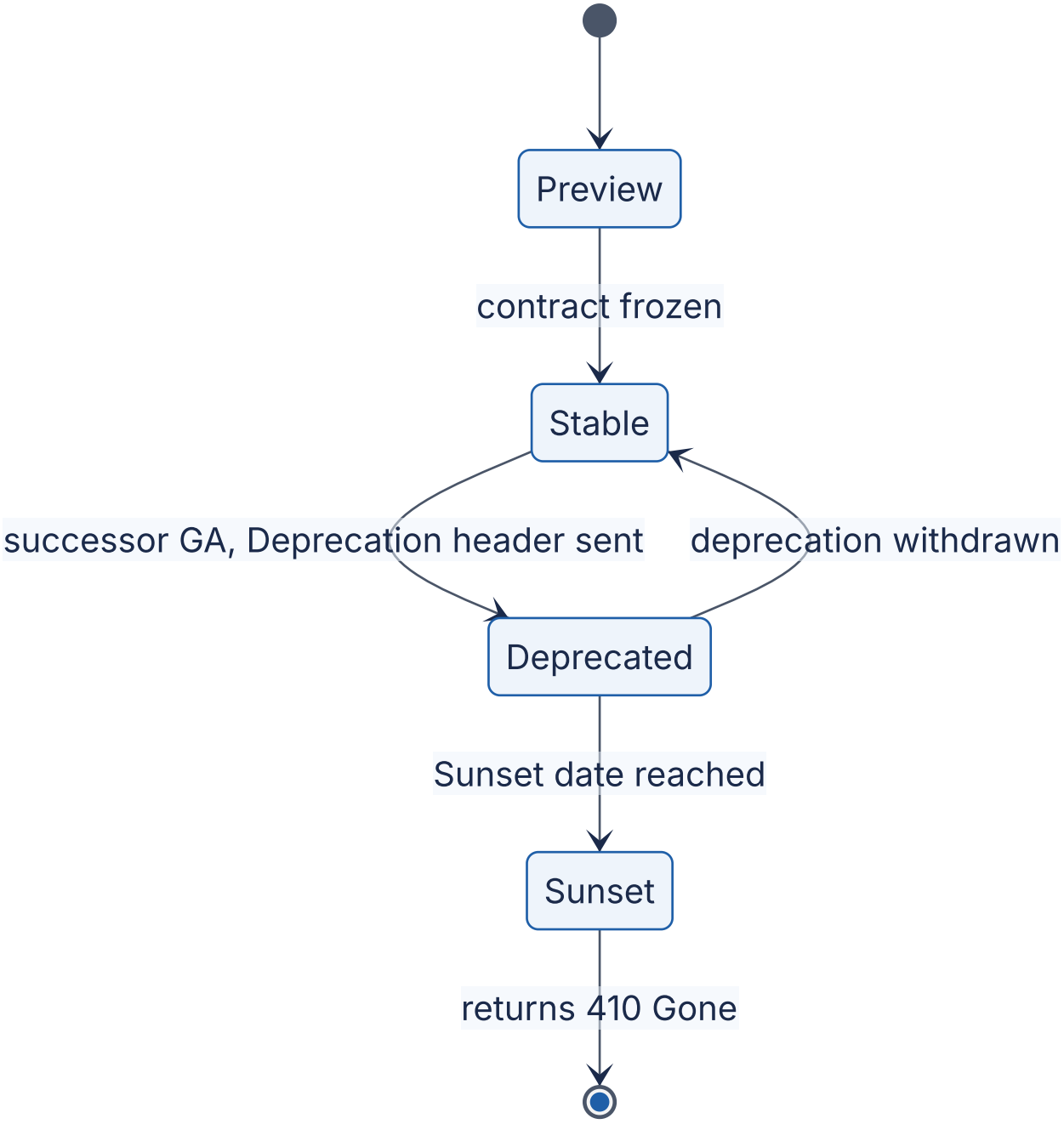



**Figure 8.12 — Lifecycle of an API version.**




## Chapter 9 — Real-Time Communication

**Figure 9.1 — Short polling.**

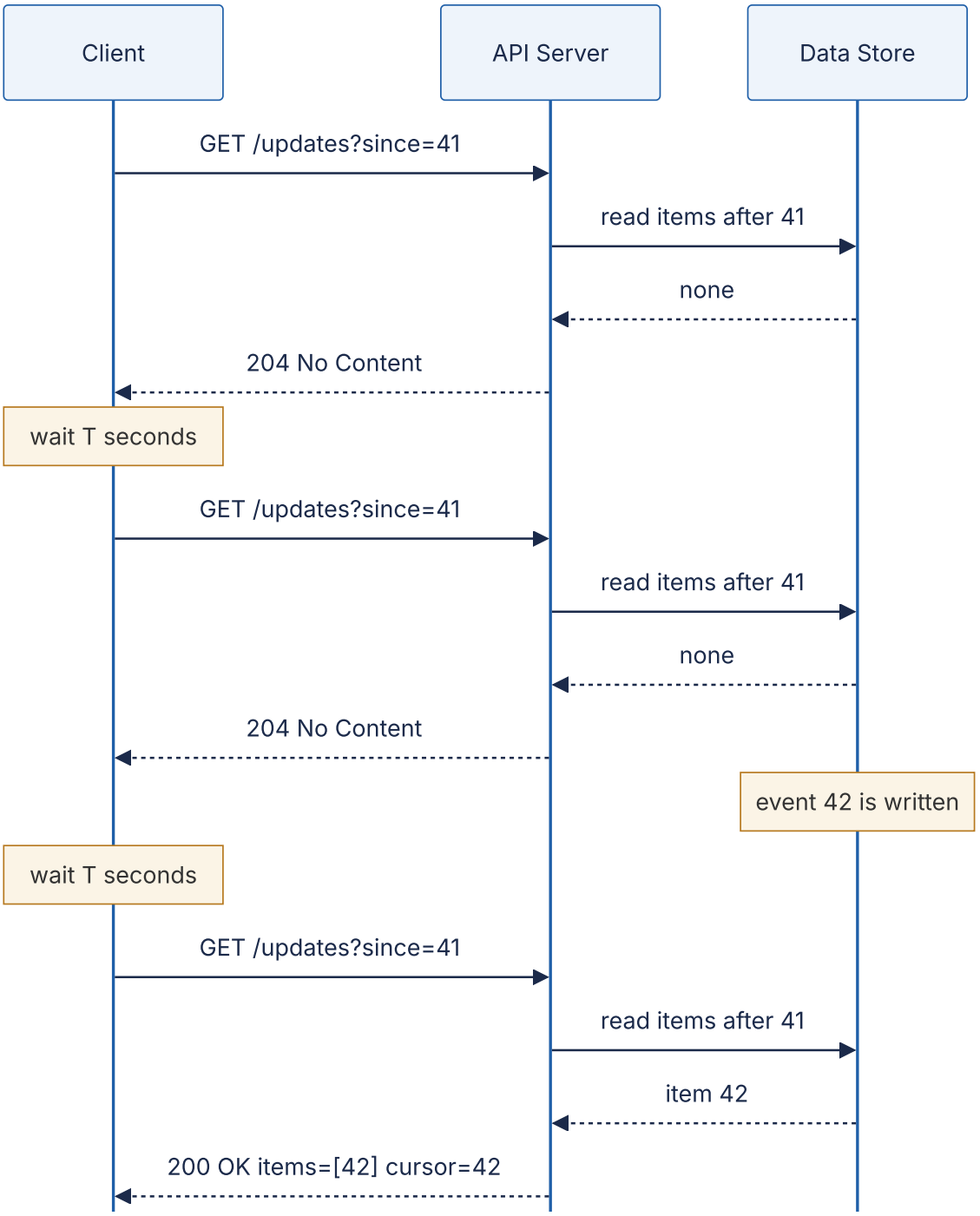



**Figure 9.2 — Long polling with cursors and a hold timeout.**

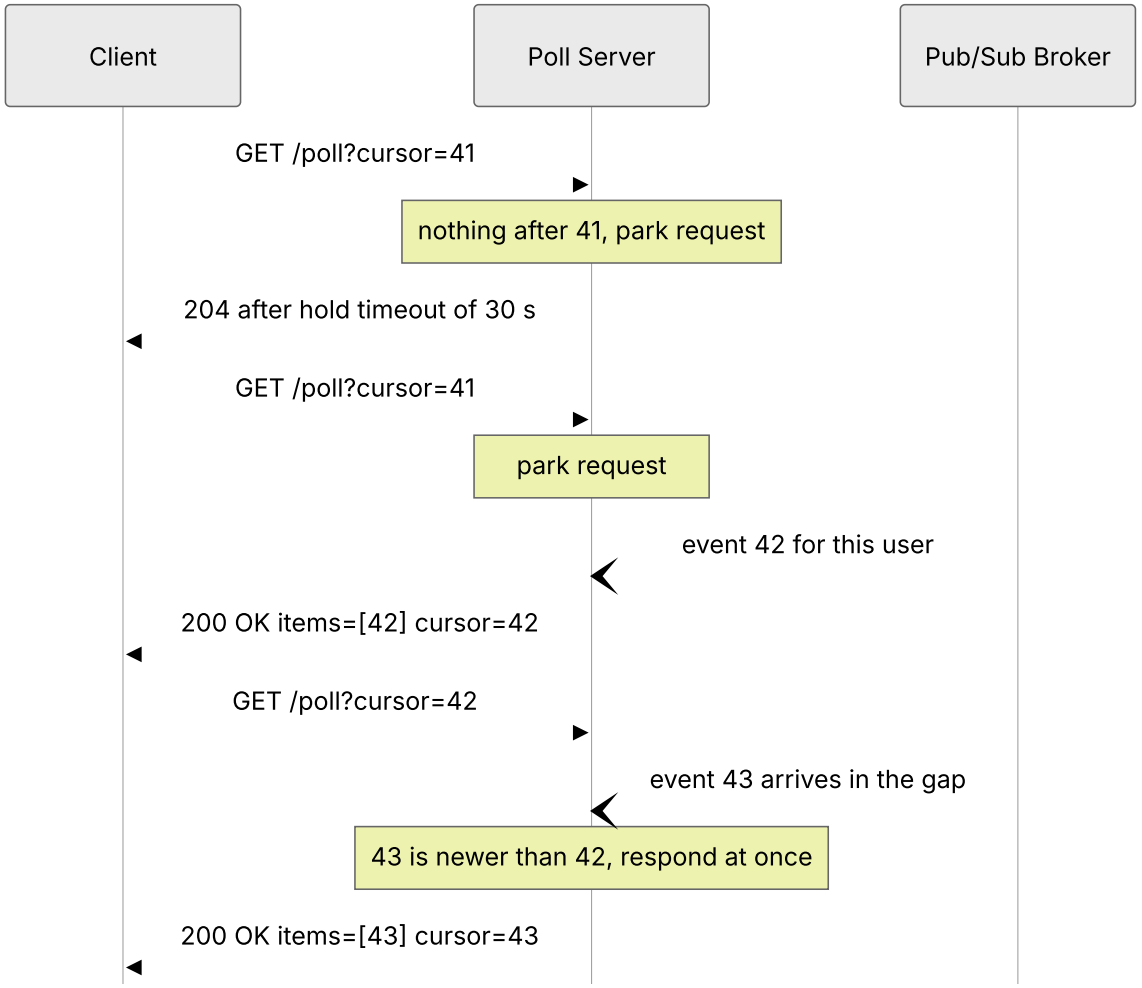



**Figure 9.3 — SSE streaming, disconnection and resumption with `Last-Event-ID`.**

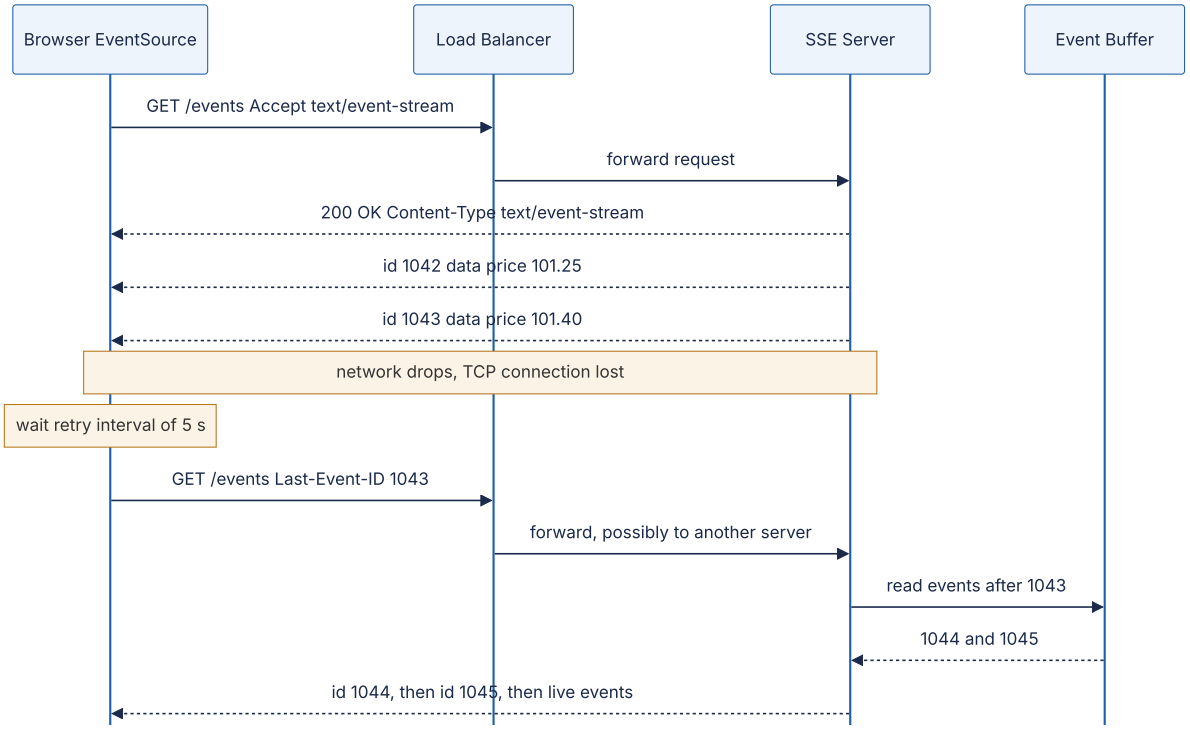


**Figure 9.4 — WebSocket opening handshake, message exchange, heartbeat and closing handshake.**

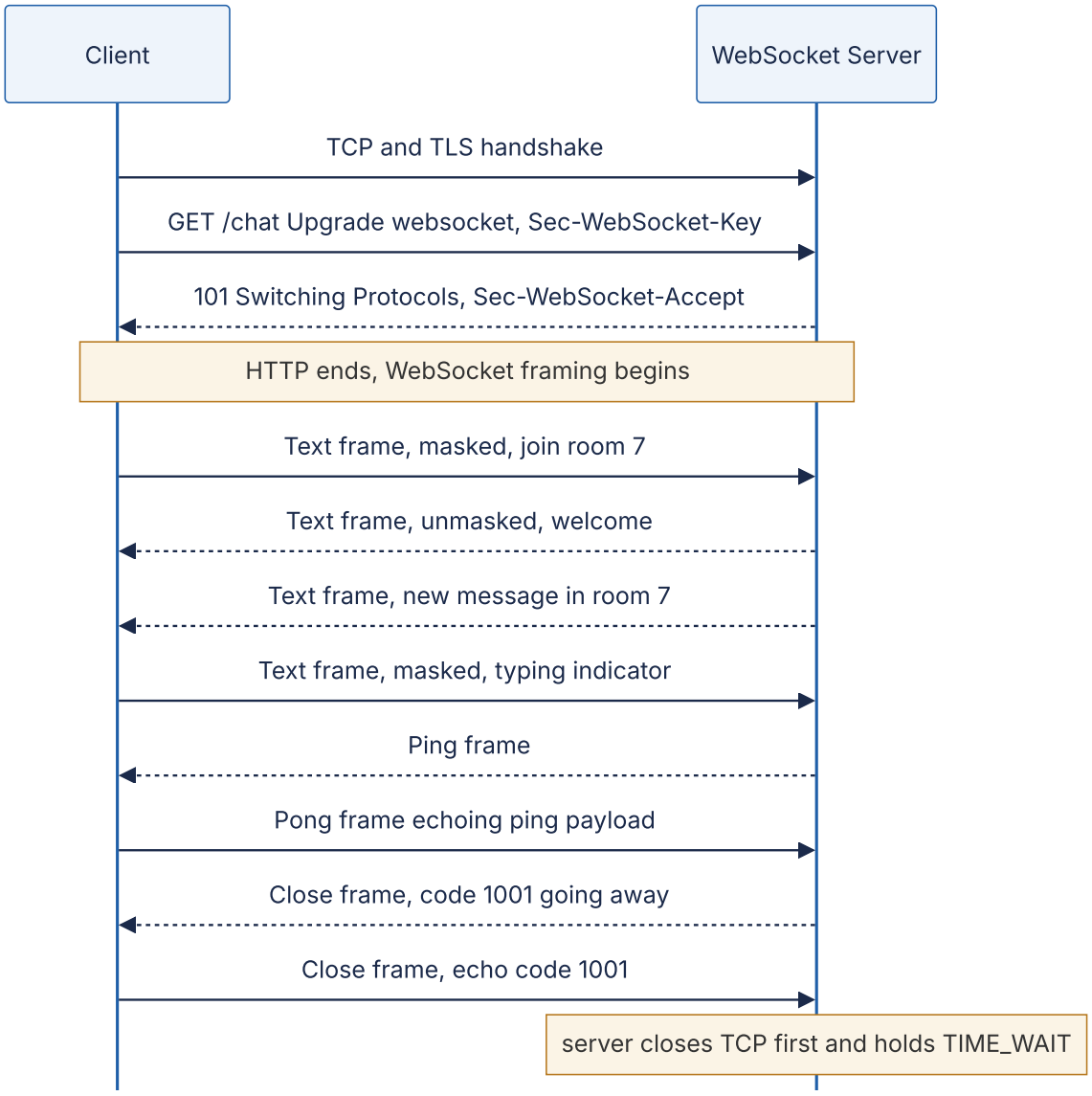



**Figure 9.5 — Lifecycle of a WebSocket connection as seen by a gateway.**

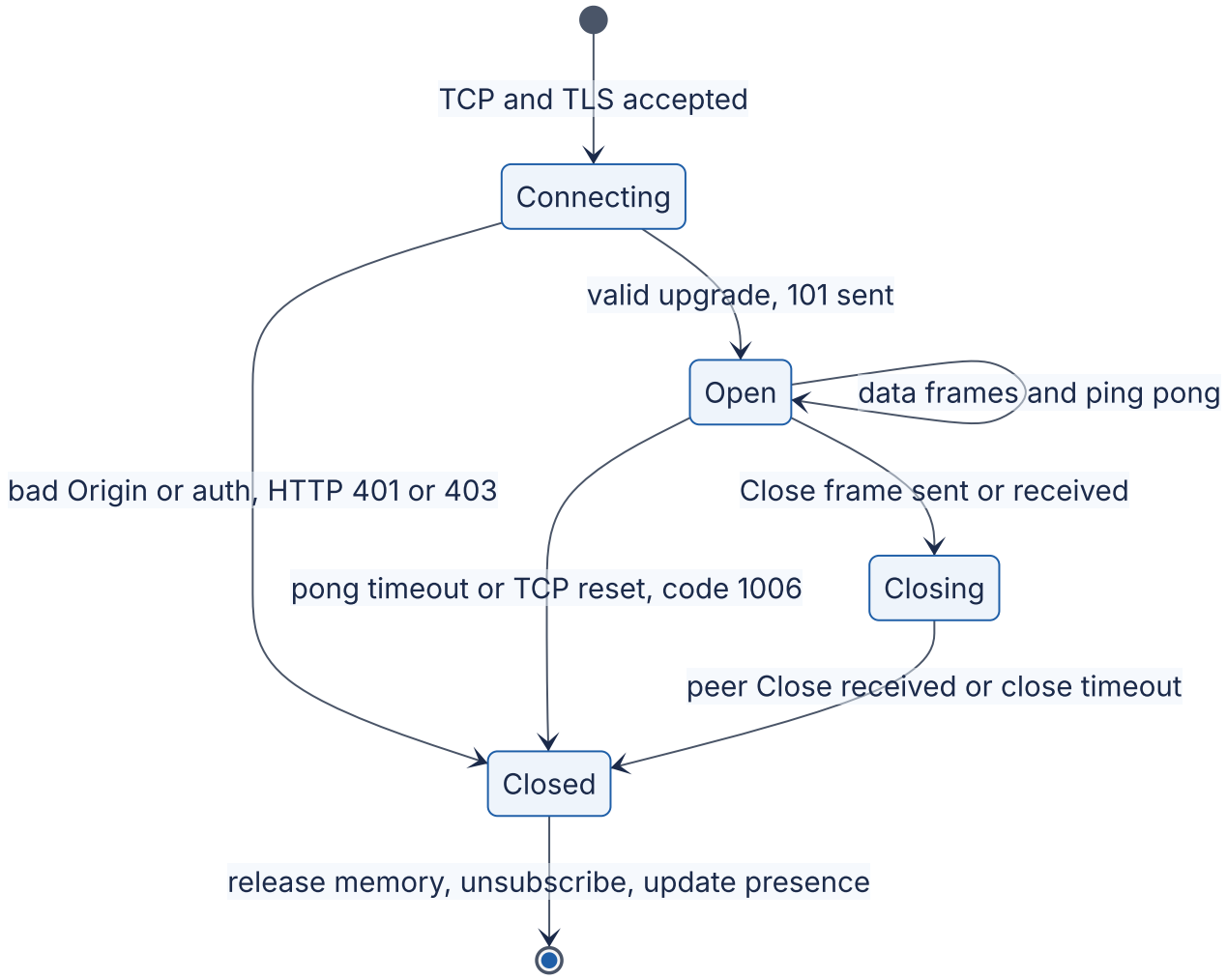



**Figure 9.6 — WebRTC session establishment: signaling, STUN, ICE and TURN fallback.**

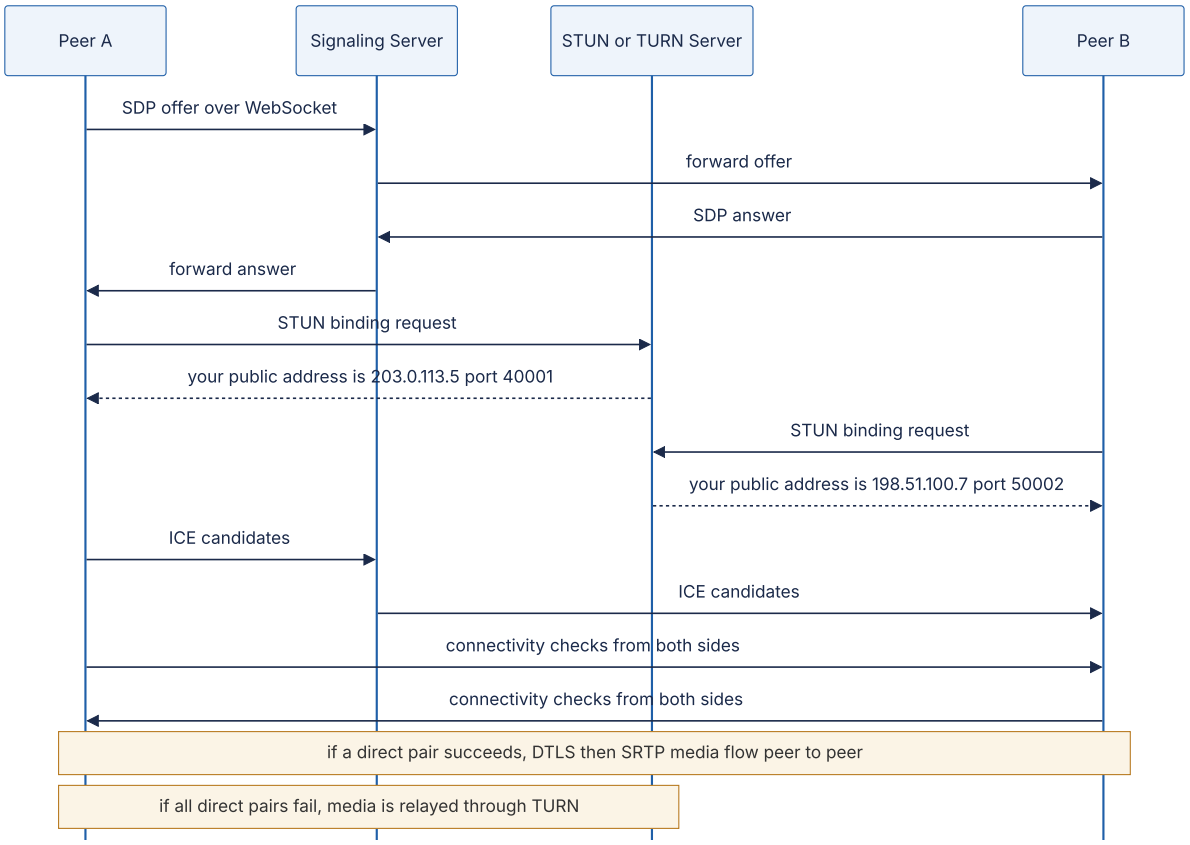



**Figure 9.7 — Mesh, SFU and MCU topologies for a four-person call.**

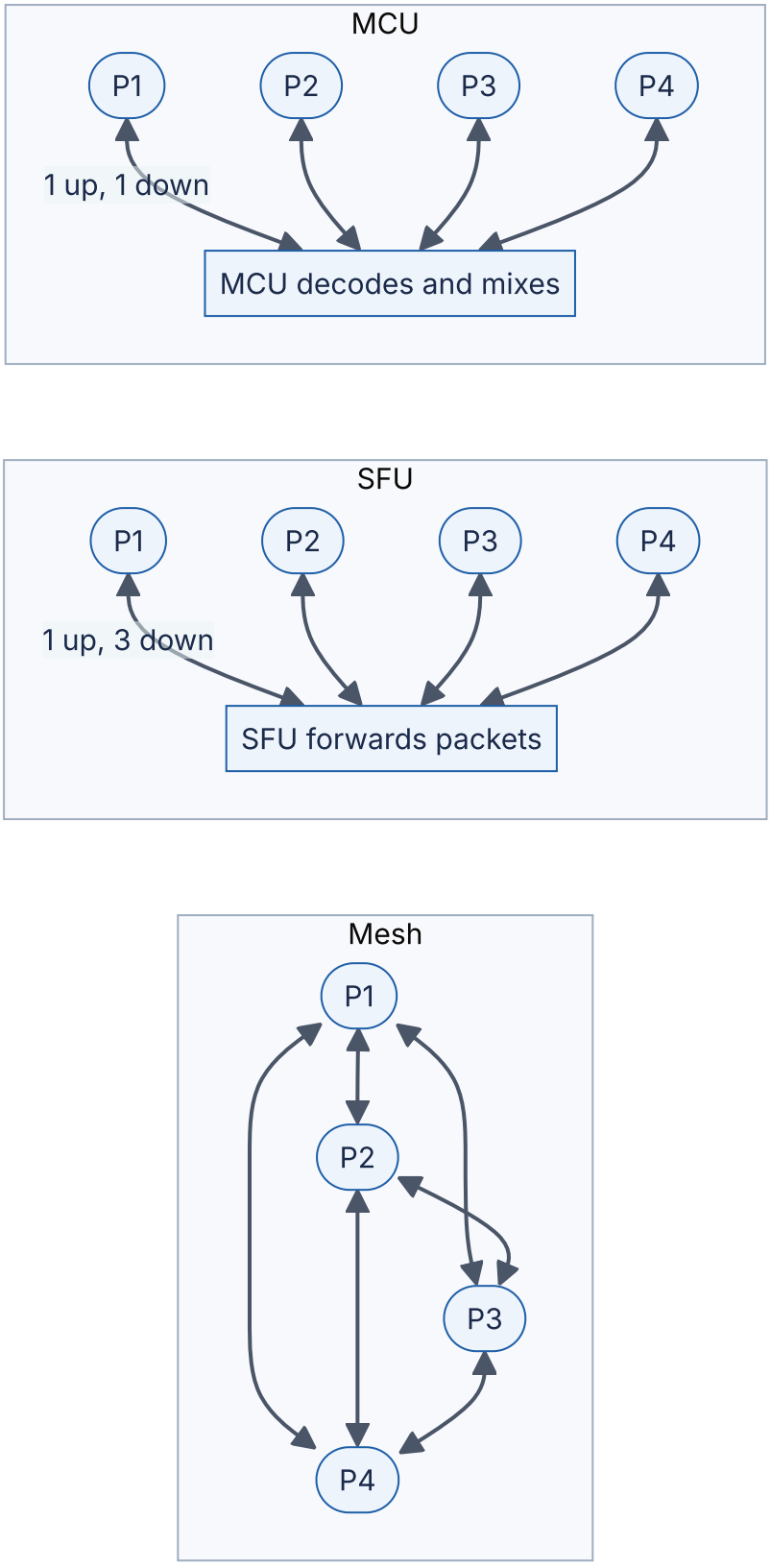



**Figure 9.8 — Mobile push architecture.**

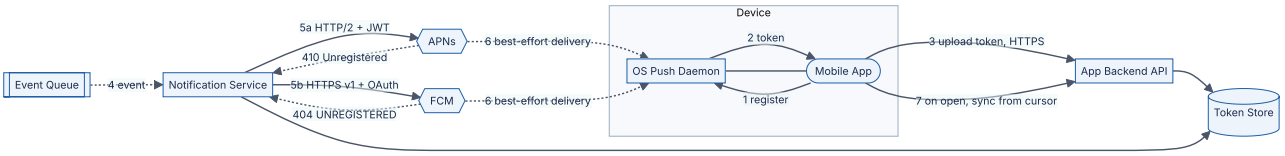



**Figure 9.9 — Architecture of a connection gateway fleet.**

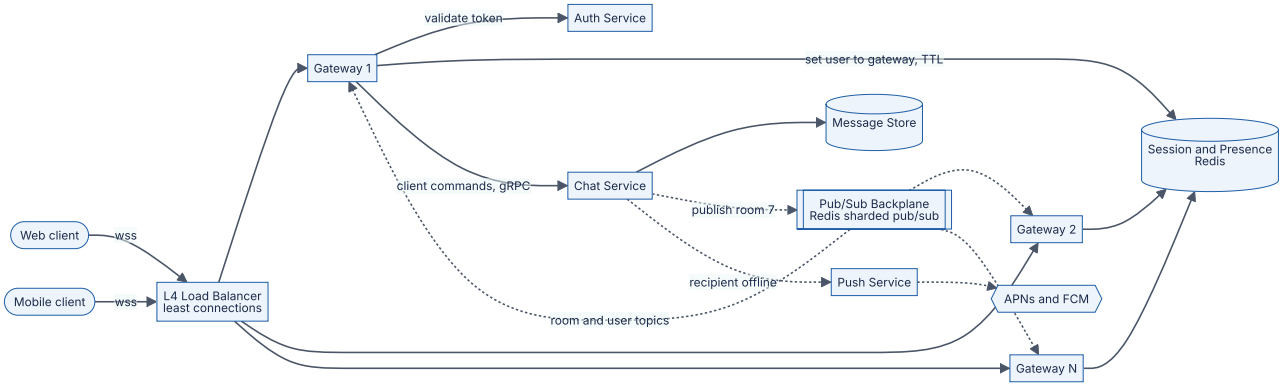
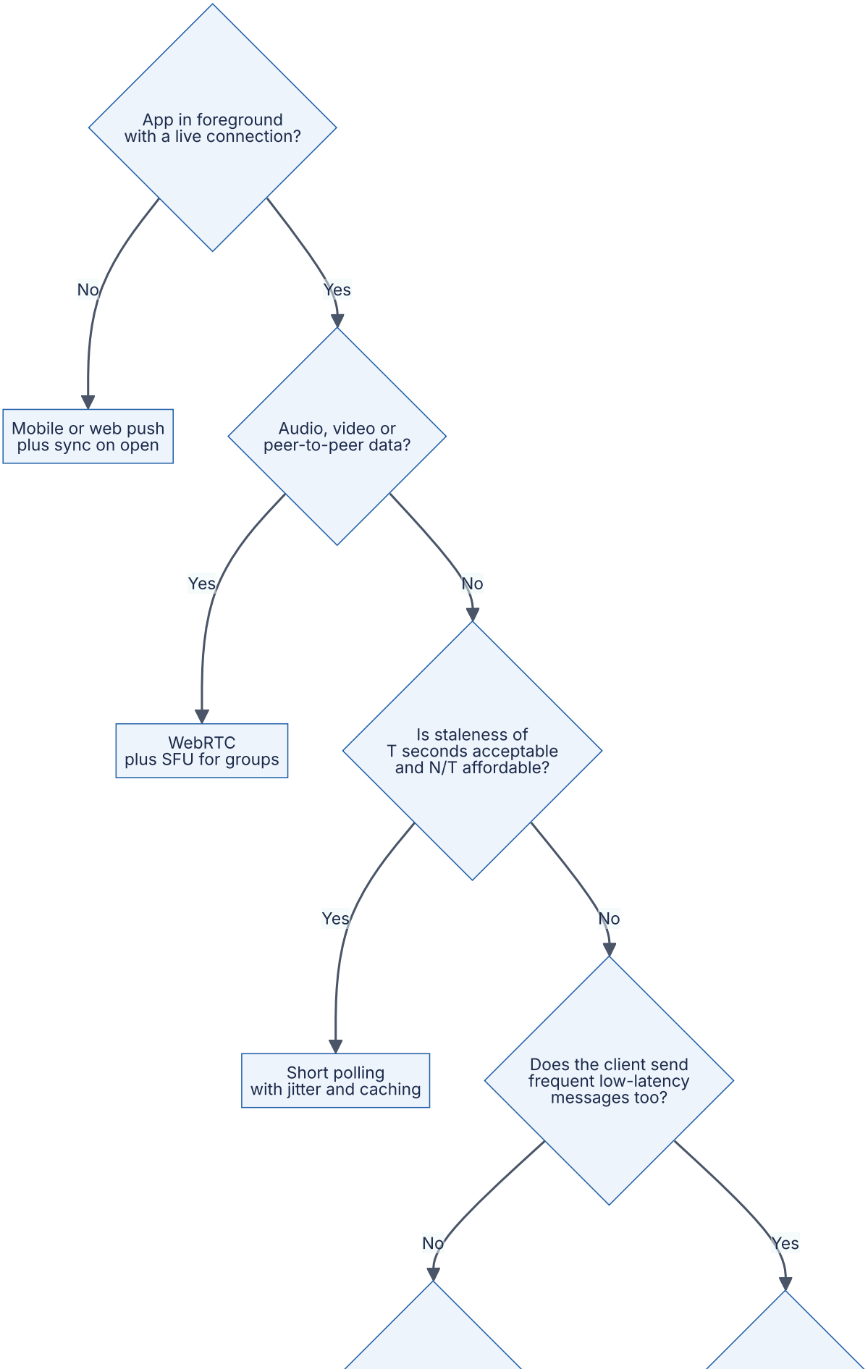
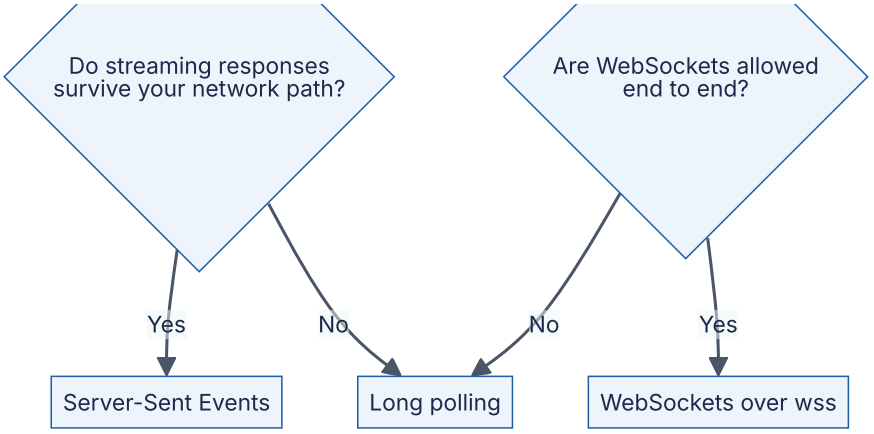



**Figure 9.10 — Decision flowchart for choosing a real-time technique.**


App in foreground with a live connection?
No
Yes
Mobile or web push plus sync on open
Audio, video or peer-to-peer data?
Yes
No
WebRTC plus SFU for groups
Is staleness of T seconds acceptable and N/T affordable?
Yes
No
Short polling with jitter and caching
Does the client send frequent low-latency messages too?
No
Yes




# Chapter 10 — Security Fundamentals for System Designers

**Figure 10.1 — Data-flow diagram of a photo-sharing web application with trust boundaries.**

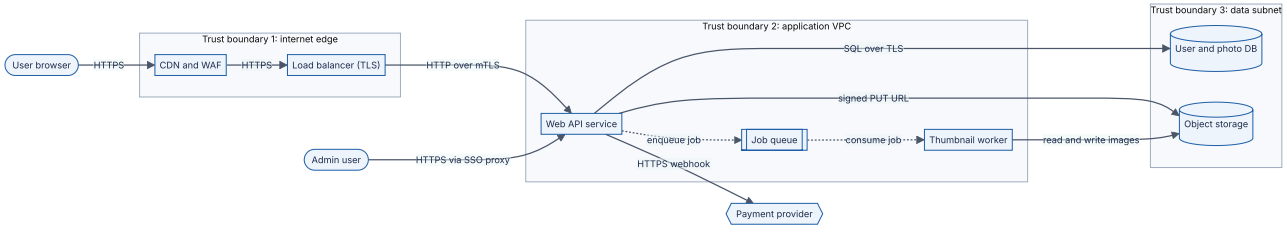



**Figure 10.2 — Simplified TLS 1.3 handshake: ephemeral key exchange plus signature.**

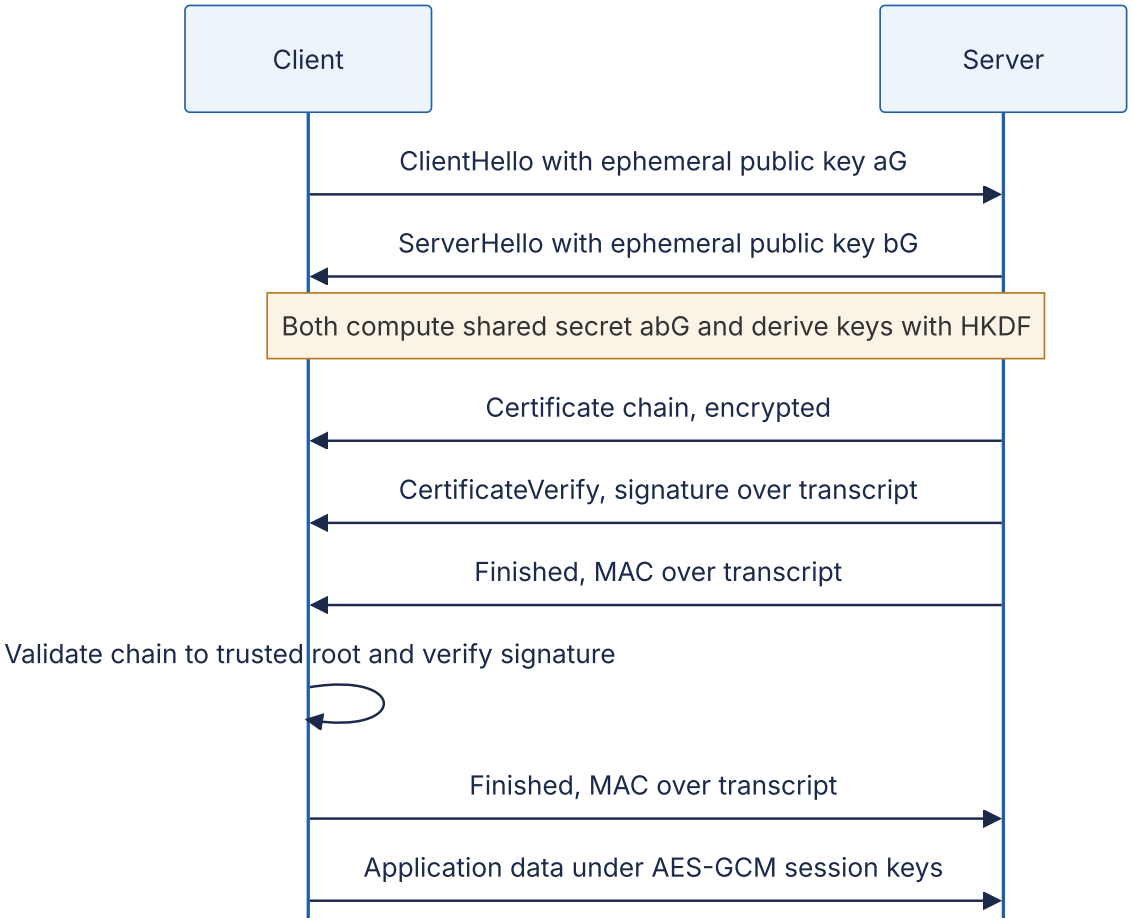



**Figure 10.3 — HMAC request signing with replay protection.**

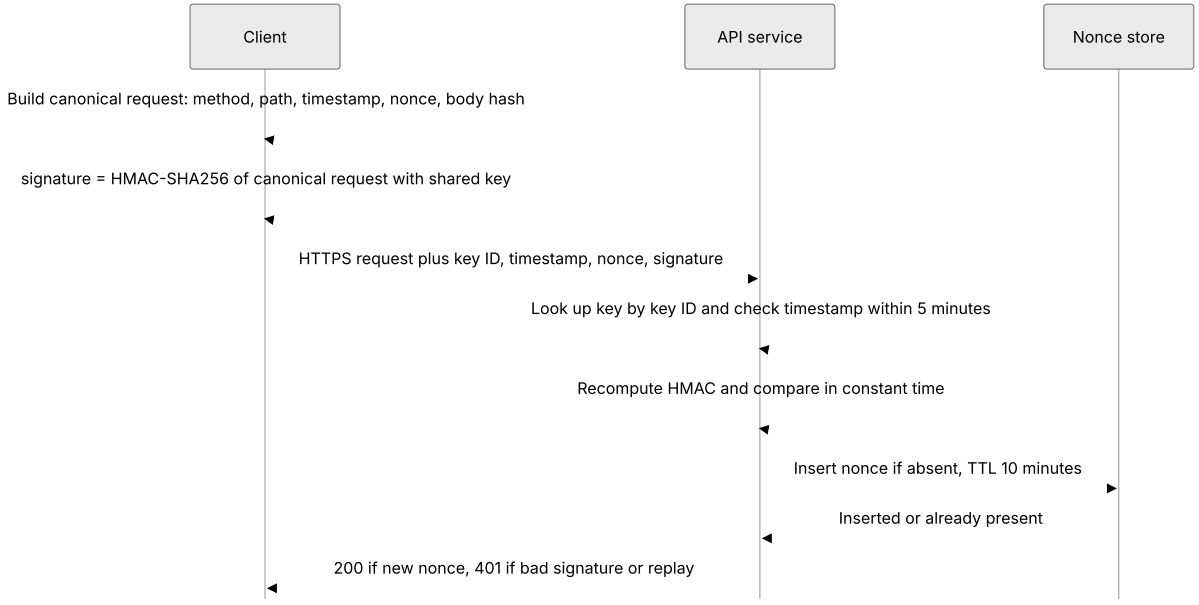



**Figure 10.4 — Certificate chain of trust and Certificate Transparency.**

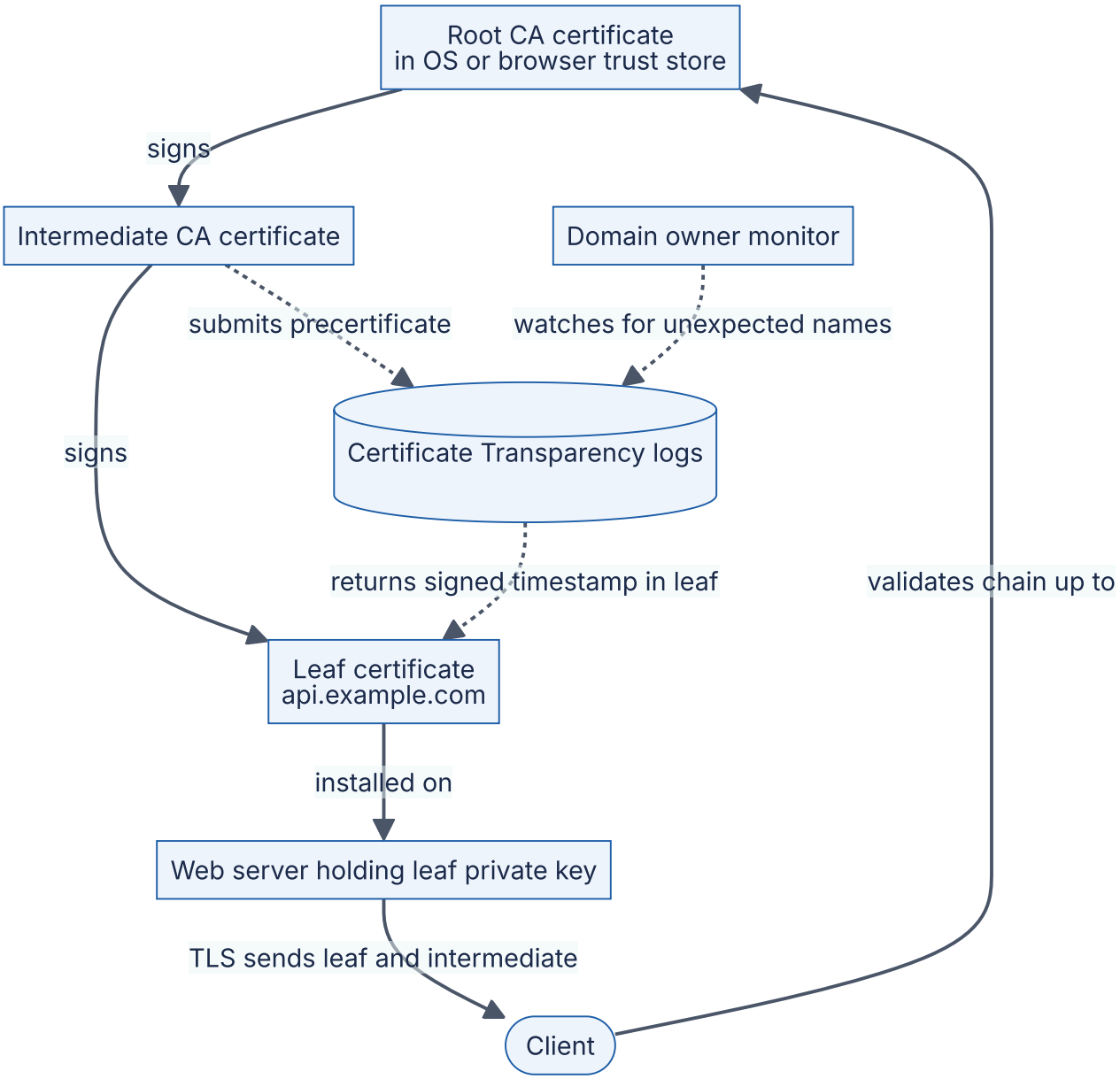



**Figure 10.5 — Registration and login with salted Argon2id and a pepper.**

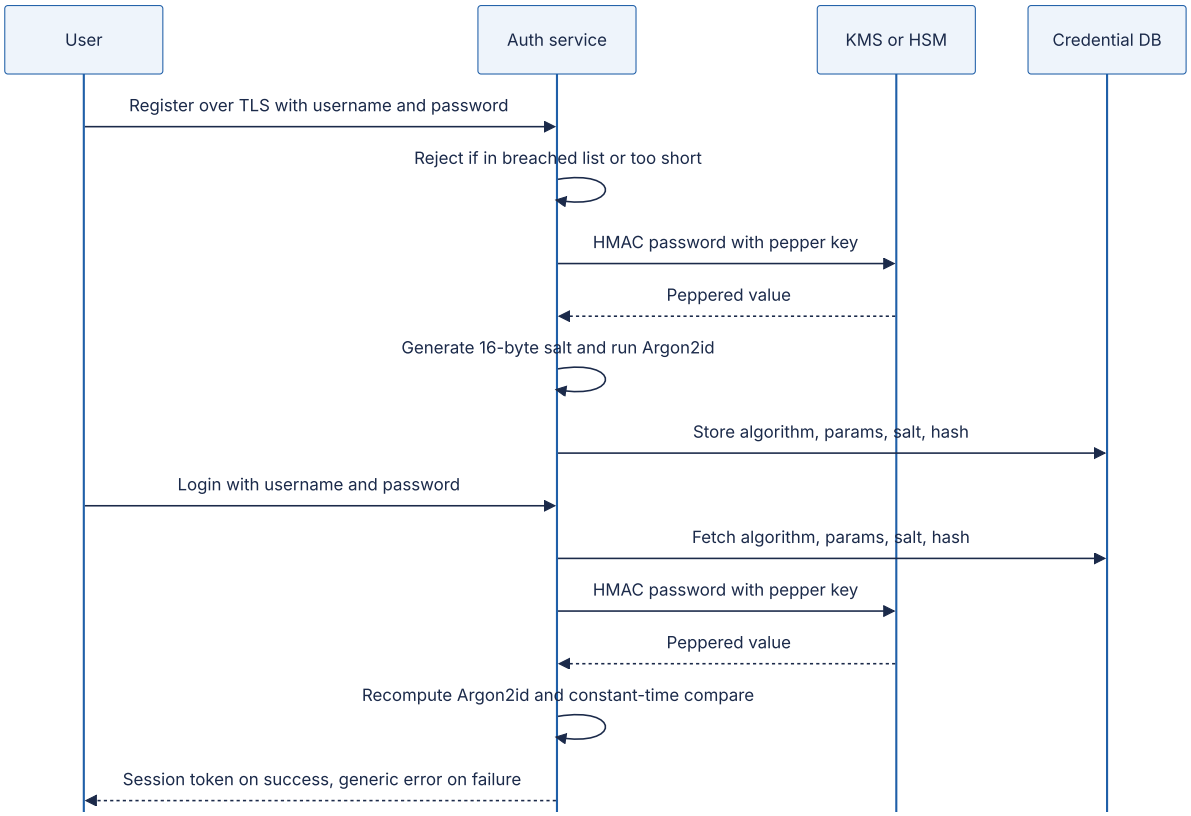



**Figure 10.6 — A workload obtaining dynamic database credentials from a vault.**

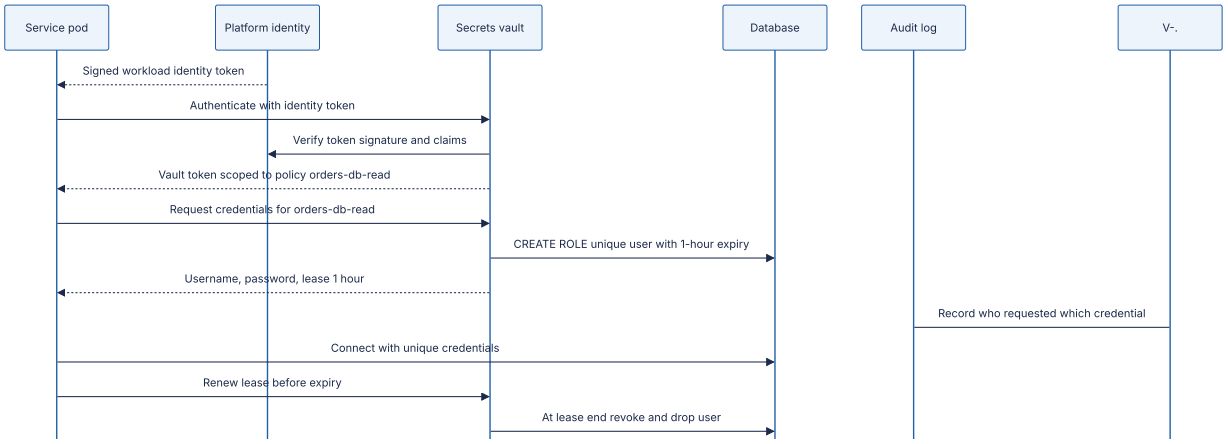



**Figure 10.7 — Envelope encryption: write and read paths.**

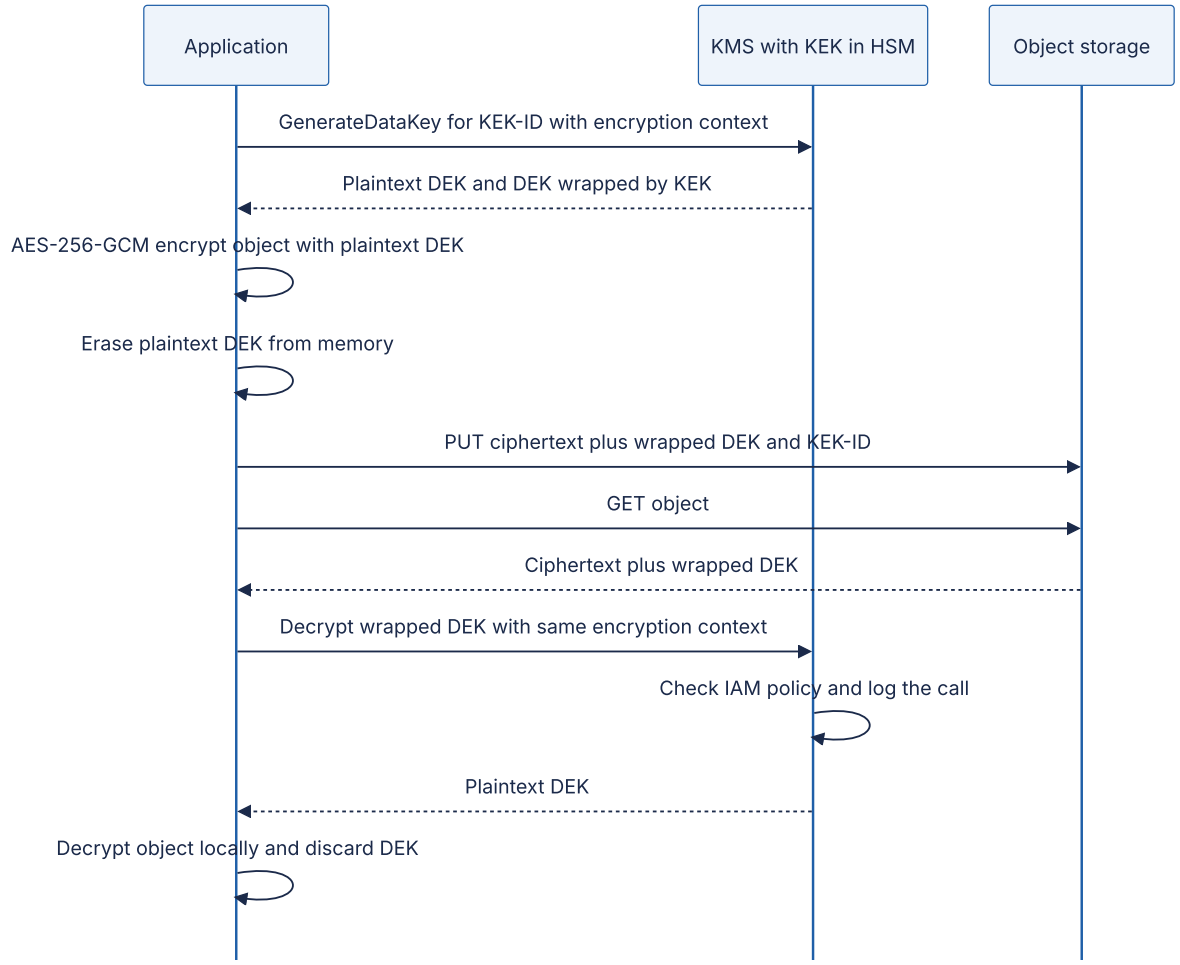



**Figure 10.8 — Zero-trust access in the style of BeyondCorp.**

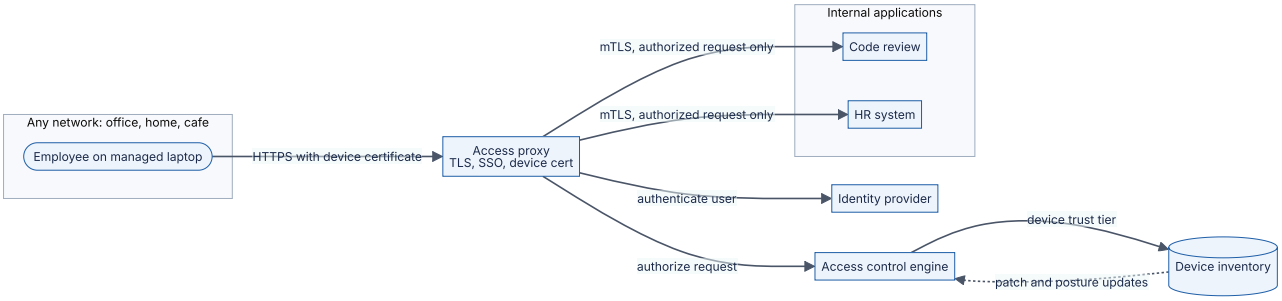



**Figure 10.9 — DDoS mitigation in layers.**

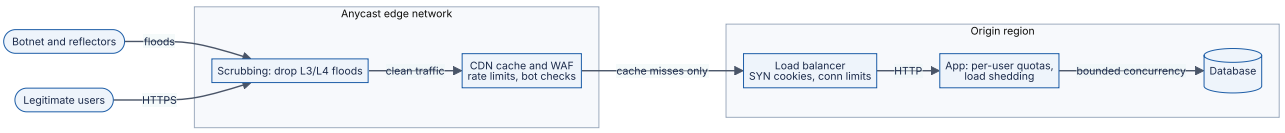
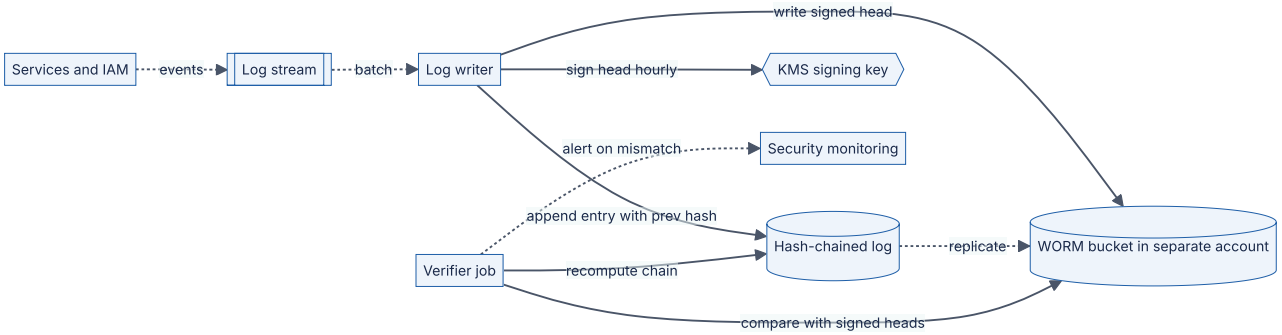



**Figure 10.10 — Tamper-evident audit logging pipeline.**


Services and IAM
events
Log stream
batch
Log writer
write signed head
sign head hourly
KMS signing key
alert on mismatch
Security monitoring
append entry with prev hash
Hash-chained log
replicate
WORM bucket in separate account
Verifier job
recompute chain
compare with signed heads